Game Programming with Unity

애니메이션 시스템
(Animation System)

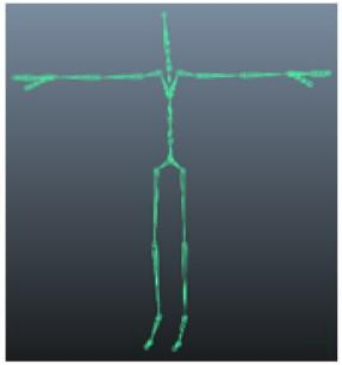
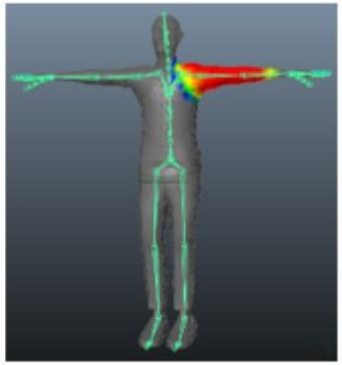
애니메이션 시스템

- **유니티 애니메이션 시스템(Unity Animation System)**
  - 레거시 애니메이션 시스템(Legacy Animation System)
    메카님 애니메이션 시스템 이전에 사용
    호환성을 위해 제공
  - 메카님 애니메이션 시스템(Mecanim Animation System)
    인간(Humanoid) 캐릭터에 대한 애니메이션의 쉬운 설정과 관리
    애니메이션 리타겟팅(Retargeting) 지원
    애니메이터 윈도우와 애니메이션 미리보기 윈도우 제공

Animation Clip
Body Mask
Animation Curves

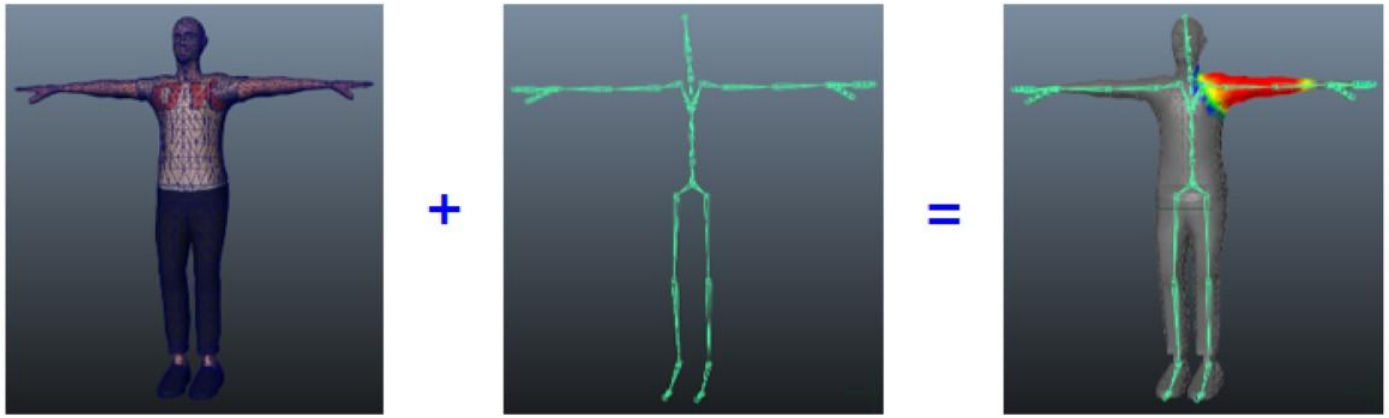
Avatar
Retargeting
Rigging
Skinning
Muscle Definition
T-pose
Bind-pose
Human Template

Animator Component
Root Motion
Animator Controller
Animation Layer
Animation State Machine
Animation Blend Tree
Animation Parameters
Inverse Kinematics (IK)
Animation Component

애니메이션 시스템

- **3D 모델 애니메이션(3D Model Animation)**
  - 캐릭터 모델(Generic/Humanoid Meshes)
    애니메이션을 위하여 리깅과 스키닝을 지원하는 메쉬 모델이 필요
    ① 모델링(Modeling)
    캐릭터 모델 메쉬(삼각형), 3DSMax, Maya, Blender
    메쉬의 크기에 주의(1m 큐브와 비교), 가능하면 T-포즈로 모델링
    ② 리깅(Rigging)
    뼈대(Joint, Skeleton) 계층 구조, 움직임을 제어
    사람의 경우 보통 엉덩이(Hip)가 계층 구조의 루트가 되도록 함
    최소 15개 이상의 뼈대가 필요
    ③ 스키닝(Skinning)
    메쉬와 뼈대를 연결
    소프트 바인드(하나의 정점에 영향을 주는 뼈대는 최대 4개까지만 가능



Unity3.5 애니메이션 시스템(Legacy): 애니메이션 컴포넌트, Animation 클래스
Unity4.0+ 애니메이션 시스템(Mecamin): 애니메이터 컴포넌트, Animator 클래스, 아바타, 리타겟팅

애니메이션 시스템

- **유니티 애니메이션 시스템(Animation System)**
  - 애니메이션 시스템은 스키닝 캐릭터를 지원(정점당 1/2/4 뼈대)
  - 애니메이션 컴포넌트에 애니메이션 클립을 지정하고 스크립트로 재생할 수 있음
    Animation Blending, Additive Animation, Animation Mixing, Layers
  - 캐릭터 모델링 최적화

① 각 캐릭터를 위하여 하나의 스킨 메쉬 렌더러를 사용할 것
유니티는 가시성 컬링과 바운딩 볼륨 갱신을 사용하여 애니메이션을 최적화
(하나의 애니메이션 컴포넌트와 하나의 스킨 메쉬 렌더러를 사용할 때만 활성화)

② 가능한 적은 수의 재질을 사용할 것
각 메쉬의 재질의 개수를 적게 사용

③ 가능한 적은 수의 뼈대(Bone)을 사용할 것

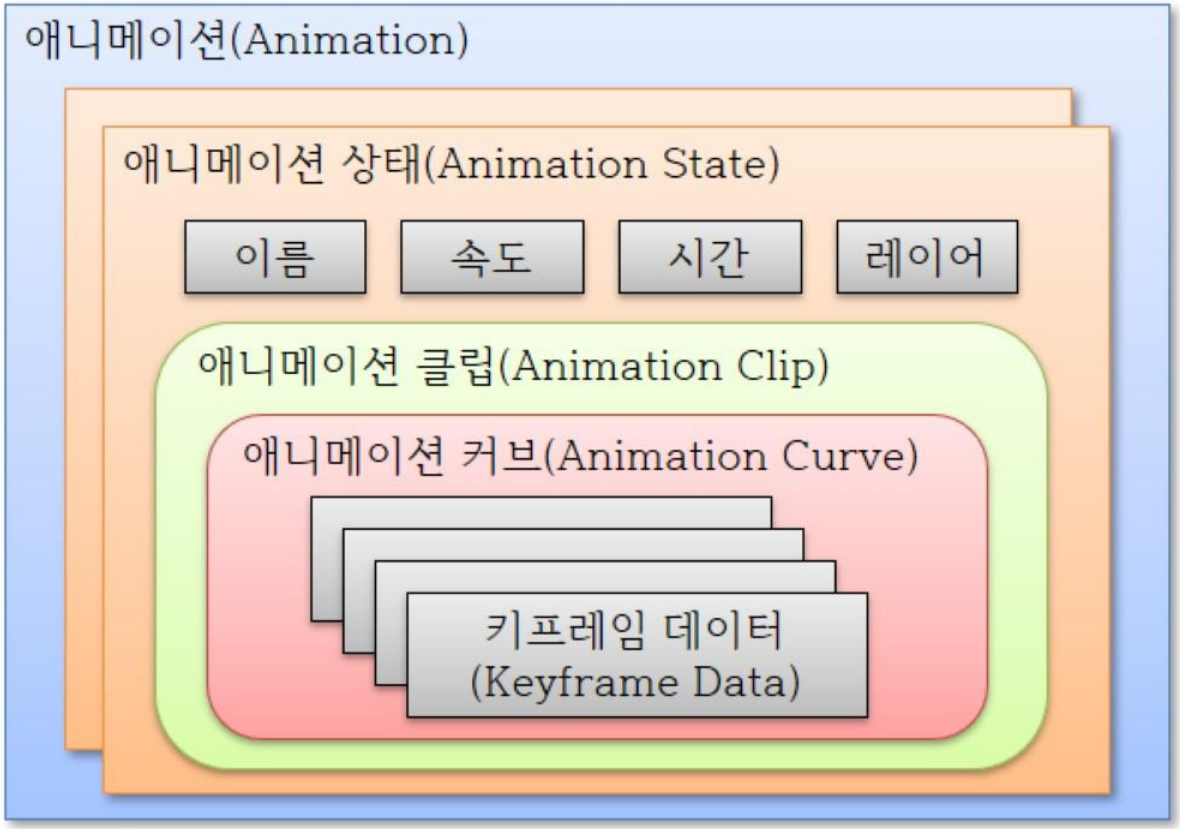
④ 다각형(면)의 개수
다각형의 개수는 캐릭터 품질과 플랫폼에 따라 달라짐
모바일 디바이스: 300~1500개의 다각형
PC: 1500~4000의 다각형
Half Life 2: 2500~5000개의 삼각형, PS3/XBOX 360: 5000~7000개의 삼각형

⑤ FK(Forward Kinematics)와 IK(Inverse Kinematics)를 분리할 것
애니메이션이 임포트될 때 모델의 IK 노드는 FK로 바뀜
유니티에서 IK 노드는 전혀 필요하지 않게됨
모델링을 할 때 IK와 FK 계층구조를 구분하여 IK를 쉽게 제거할 수 있도록 할 것

캐릭터의 실제 애니메이션 제어는 스크립트를 사용해야 함

애니메이션 시스템

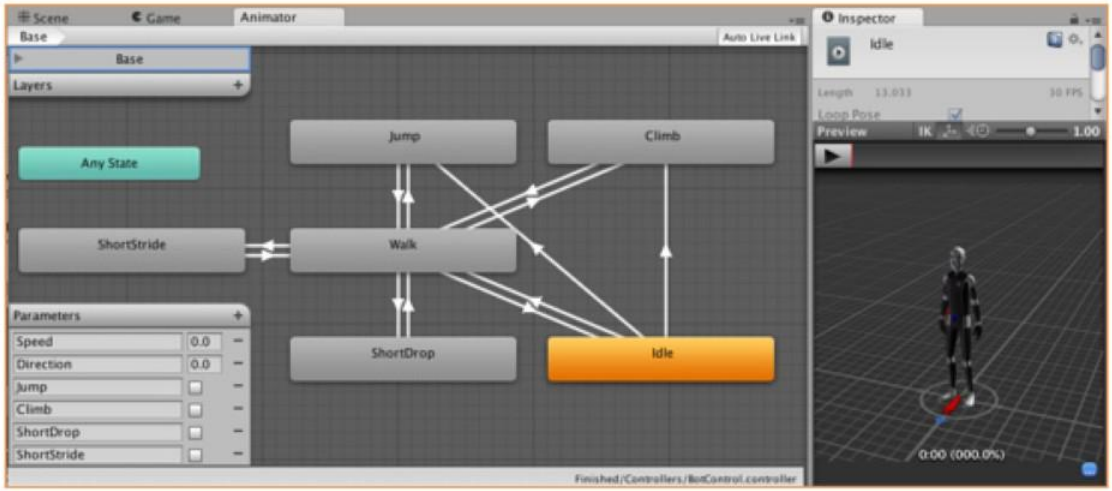
- 애니메이션 시스템(Animation System)



Animation	애니메이션을 관리하고 애니메이션을 실행
Animation State	하나의 애니메이션의 상태 정보와 애니메이션 클립 정보를 관리
Animation Clip	애니메이션 데이터
Clip Property	변화되는 속성(애니메이션 커브를 가짐)
Animation Curve	시간에 대한 움직임 정보(키프레임들의 집합)
Keyframe	위치(시간)

애니메이션 시스템

- **메카님 애니메이션 시스템(Mecanim Animation System)**
  - 기능
    인간 캐릭터(Humanoid)의 애니메이션 설정을 쉽게 처리
    애니메이션 리타겟팅(Retargeting)
    애니메이션 클립 설정을 간단하게 처리
    애니메이션들의 복잡한 상호작용의 관리(비주얼 프로그래밍 도구 사용)



  - 작업 순서
    ① 애셋 임포트
    ② 캐릭터 설정
    인간 캐릭터: 리타겟팅, 아바타(Avatar) 생성과 설정, 근육(Muscle) 정의
    일반 캐릭터: 동물, 리타겟팅이 가능하지 않음
    ③ 애니메이션 설정
    애니메이션 클립, 상태 기계, 블렌드 트리, 파라메터 설정, 애니메이션 스크립팅

애니메이션 시스템

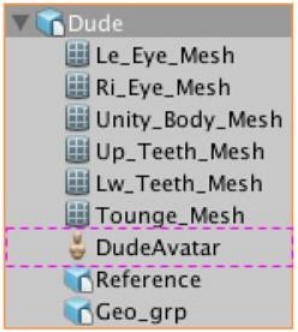
- **메카님 애니메이션 시스템(용어)**

Animation Clip	애니메이션 데이터("Idle", "Walk")	sub-Asset
Body Mask	뼈대에 포함 또는 제거할 부분, 애니메이션 레이어에 사용	Asset (.mask)
Animation Curves	애니메이션 클립에 부착	
Avatar	하나의 뼈대를 다른 뼈대로 리타겟팅하기 위한 인터페이스	sub-Asset
Retargeting	한 모델의 애니메이션을 다른 모델에 적용함	Asset (.mask)
Rigging	캐릭터의 메쉬를 위한 뼈대의 계층구조를 만드는 과정	Process
Skinning	캐릭터의 스킨 메쉬에 뼈대를 연결하는 과정	Process
Muscle Definition	캐릭터의 뼈대에 대한 더 직관적인 제어를 할 수 있도록 함	Process
T-pose	T-포즈, 아바타를 만들기 위하여 필요	
Bind-pose	캐릭터가 모델링되는 포즈	
Human template	미리 정의한 뼈대 매핑, FBX 파일에서 아바타로 뼈대를 일치	Asset(.ht)
Animator Component	메카님을 사용하는 모델의 컴포넌트	Component
Root Motion	캐릭터 루트의 모션	
Animator Controller	애니메이션 레이어를 통하여 애니메이션을 제어	Asset (.controller)
Animation Layer	애니메이션 상태 기계를 포함	
Animation State Machine	애니메이션 상태의 상호작용을 제어하는 그래프 각 상태는 애니메이션 블렌드 트리 또는 애니메이션 클립을 참조	
Animation Blend Tree	애니메이션 클립들의 연속적인 블렌딩을 하기 위하여 사용	
Animation Parameters	애니메이터 컨트롤러를 제어	
Inverse Kinematics (IK)	객체들에 기반하여 캐릭터의 신체를 제어	
Animation Component	메카님을 사용하지 않는 컴포넌트	Component

애니메이션 시스템

- **메카님 애니메이션 시스템(Mecanim Animation System)**
  - 인간 캐릭터 애니메이션(Humanoid Character Animation)
    메카님 애니메이션 시스템은 인간 뼈대 구조에 잘 적용하도록 구성됨
    아바타(Avatar): 메카님의 뼈대 구조와 모델의 뼈대 구조를 일치시키는 것
  - 모델을 위한 아바타 생성하기
    - 아바타 생성(Creating Avatar)
    - 아바타 설정(Configuring Avatar)
    - 근육 설정(Muscle Setup)
    - 아바타 몸체 매스크(Avatar Body Mask)
    - 리타겟팅(Retargeting)
    - 역운동학(Inverse Kinematics)
  - 아바타 생성(Creating Avatar)
    모델 파일(FBX, COLLADA 등)을 임포트한 후 리그(Rig) 유형을 설정
    "Humanoid"를 선택하고 "Apply"를 클릭, 성공하면 아바타 애셋이 추가됨







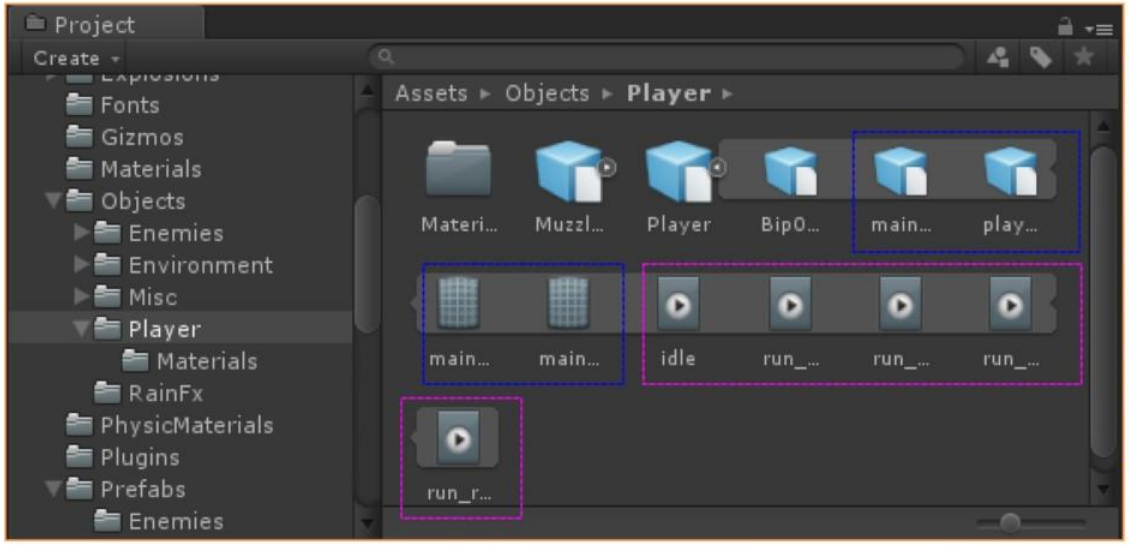


애니메이션 시스템

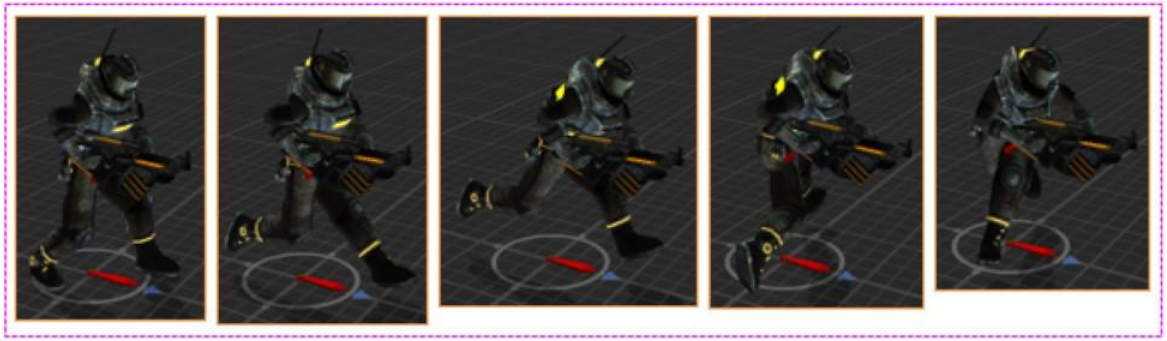
- **3D 모델(3D Model) 임포트**
  - 모델이 임포트되면 여러 개의 객체(계층)로 표현됨
    메쉬(Mesh)와 애니메이션 데이터를 포함하는 객체
    임포트된 객체는 모델 프리팹(Model Prefab)임
    여러 개의 메쉬 객체를 포함
    애니메이션 데이터(Animation Clip)를 포함







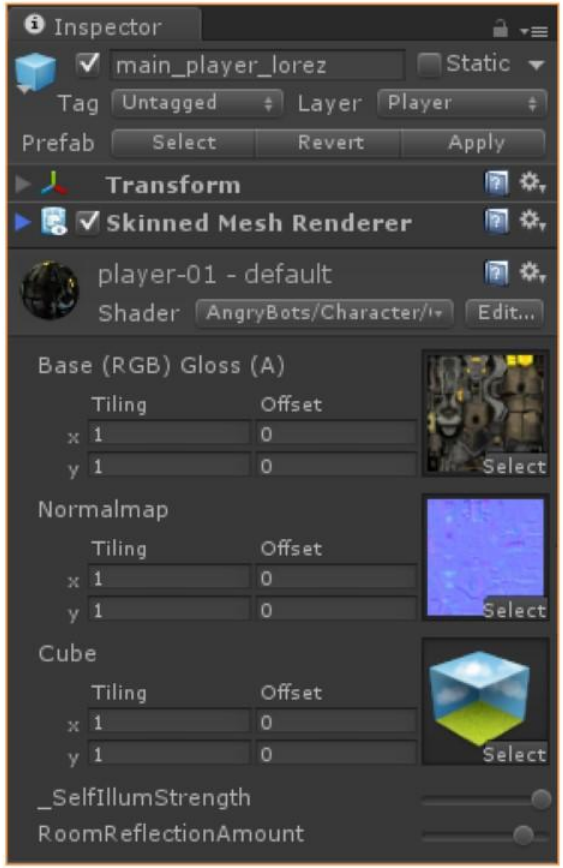




애니메이션 시스템

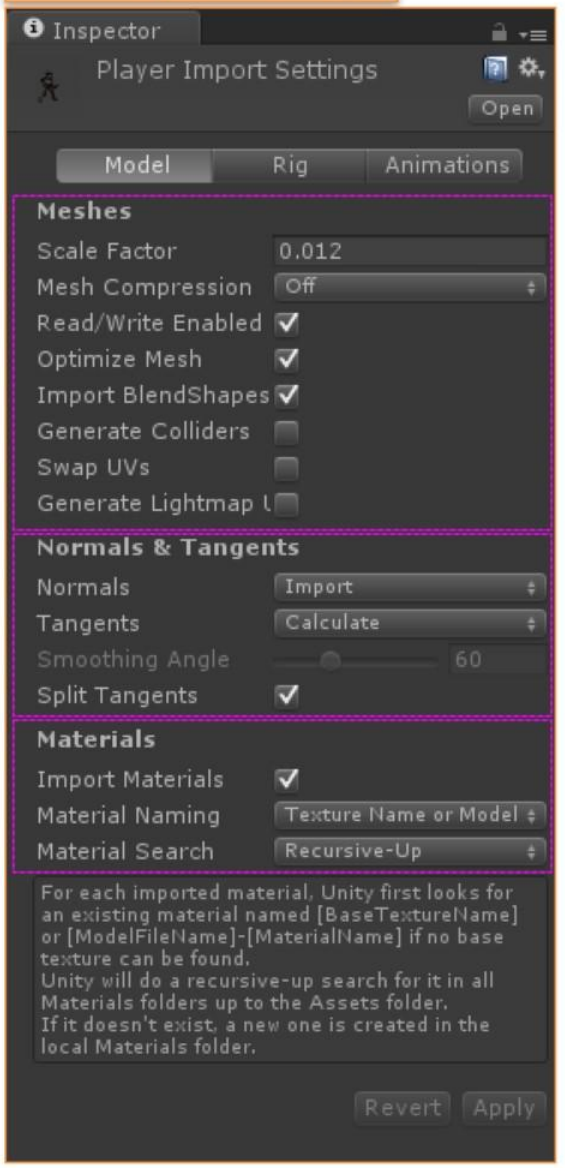
- **3D 모델(3D Model) 임포트**
  - 스킨 메쉬 렌더러(Skinned Mesh Renderer) 컴포넌트 애니메이션 데이터를 포함하는 모델의 렌더러
  - FBX 임포터 설정(FBX Importer Settings)
    - Model: 3D 모델(Mesh) 임포트 설정
    - Rig: 모델이 애니메이션을 지원하도록 설정
    - Animations: 애니메이션 클립을 임포트하도록 설정







FBX Import Settings



애니메이션 시스템

- 3D 모델(3D Model) 임포트 – Model(Mesh) 설정

Scale Factor	모델 크기, (fbx/max/jas/c4d)=0.01, (mb/ma/lxo/dxf/blend/dae)=1, 3ds=0.1
Mesh Compression	메쉬의 파일을 압축(Off, Low, Medium, High)
Read/Write Enabled	실행시 메쉬를 변경할 수 있음
Optimize Mesh	메쉬를 최적화
Import BlendShapes	모델의 BlendShapes을 임포트
Generate Colliders	메쉬 충돌체를 추가, 움직이는 메쉬에는 사용하지 말 것
Swap UVs	첫 번째 UV 채널과 두 번째 채널을 바꿈
Generate Lightmap UVs	라이트매핑을 위하여 두 번째 UV 채널을 생성
Normals, Tangents	법선(접선, 종법선) 벡터를 계산하는 방법(Import, Calculate, None)
Smoothing Angle	에지의 정도
Split Tangents	법선 맵 조명의 틈이 생기면 활성화, 보통 캐릭터에만 적용됨
Import Materials	비활성화되면 재질을 생성할 필요가 없음(기본적으로 디퓨즈 재질을 사용)
Material Naming	유니티 재질(텍스쳐)를 찾는 방법
Material Search	재질을 찾는 위치

By Base Texture Name	유니티에서 재질의 이름은 임포트된 재질의 **디퓨즈 텍스쳐의 이름**을 사용 디퓨즈 텍스쳐가 재질에 설정되지 않으면 유니티는 임포트된 재질의 이름을 사용
From Model's Material	**임포트된 재질의 이름**을 유니티 재질의 이름으로 사용
Model Name + Model's Material	임포트된 재질의 이름과 결합된 모델 파일의 이름을 유니티 재질의 이름으로 사용

Local Materials Folder	로컬 재질 폴더(Materials, 모델 파일과 같은 폴더)에서만 재질을 찾음
Recursive-Up	애셋 폴더까지 올라가면서 모든 부모 폴더의 재질 폴더에서 재질을 찾음
Everywhere	모든 프로젝트 폴더에서 재질을 찾음

애니메이션 시스템

- **스킨 메쉬 렌더러(Skinned Mesh Renderer) 컴포넌트**
  - 스키닝 메쉬를 임포트하면 스킨 메쉬 렌더러 컴포넌트가 자동으로 추가됨

Cast Shadows	이 메쉬가 그림자를 생성
Receive Shadows	이 메쉬에 그림자를 그림
Materials	모델을 렌더링할 재질의 리스트
Use Light Probes	라이트 프로브를 사용
Quality	정점에 영향을 주는 본의 최대 개수
Update When Offscreen	화면에 보이지 않을 때 애니메이션을 갱신
Mesh	스킨 메쉬
Root Bone	계층구조의 루트 뼈대
Bounds	화면에 보이는 가를 계산하기 위한 바운딩 박스

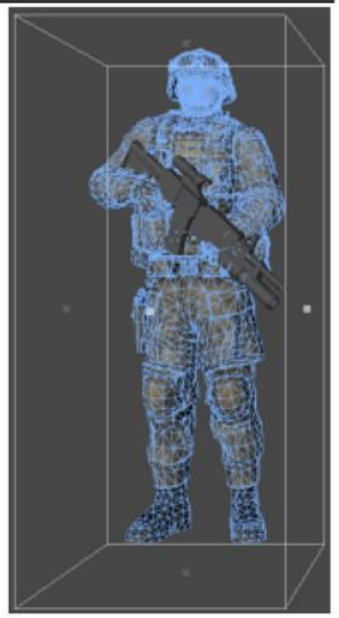
Auto
1 Bone
2 Bones
4 Bones



Edit ▶ Project Settings ▶ Quality ▶ Blend Weights

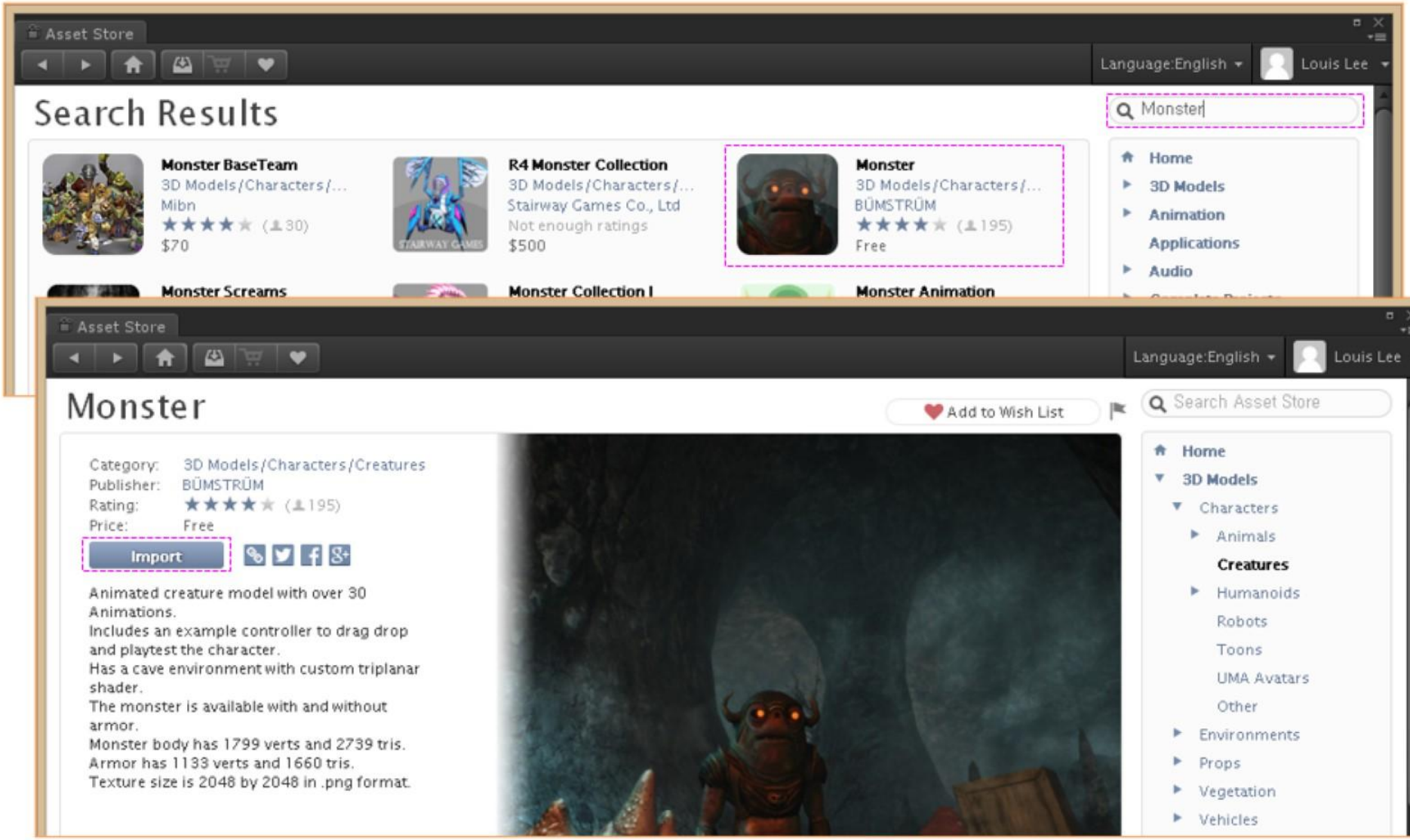
- **스킨 메쉬 렌더러(SkinnedMeshRenderer) 클래스**

bones	뼈대들의 배열, Transform[]
rootBone	루트 뼈대, Transform
quality	하나의 정점에 영향을 주는 본의 최대 개수, SkinQuality
sharedMesh	스킨 메쉬
updateWhenOffscreen	화면에 보이지 않을 때 애니메이션을 갱신
localBounds	스킨 메쉬의 AABB(지역 좌표계)
BakeMesh	스킨 메쉬 렌더러의 스냅 샷을 생성하여 메쉬에 저장



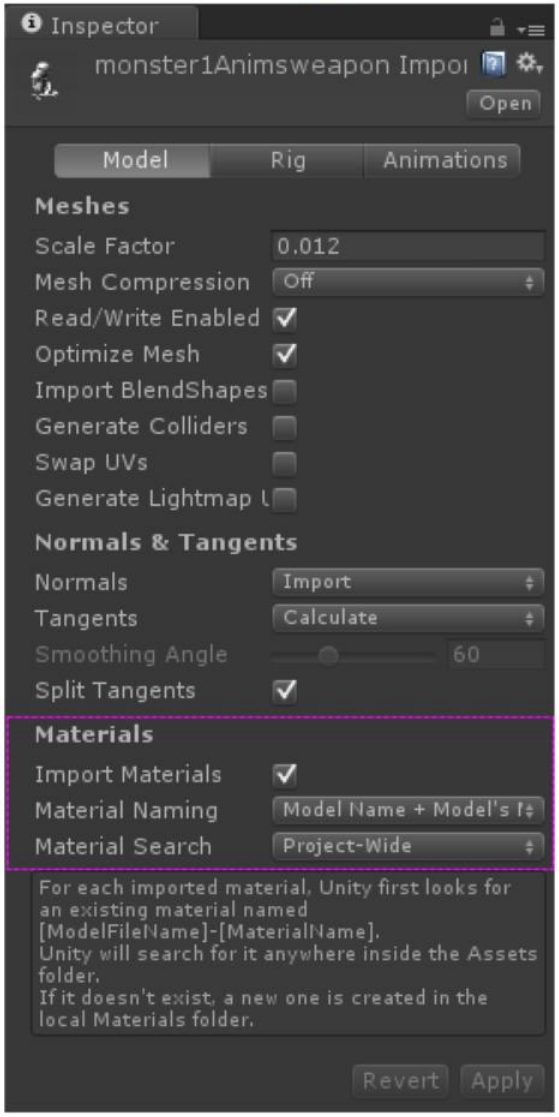
애니메이션 시스템

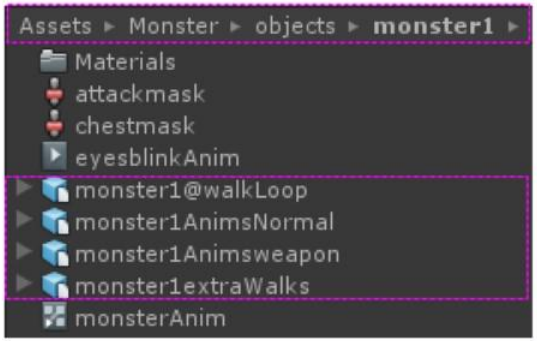
- 3D 모델(3D Model) 임포트 – Model(Mesh) 설정
  - 애셋 스토어(Asset Store): **Window ▶ Asset Store**

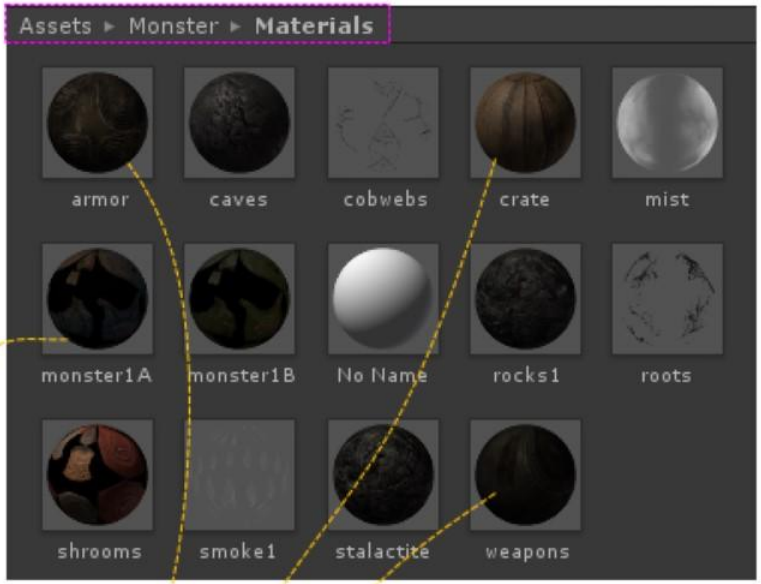


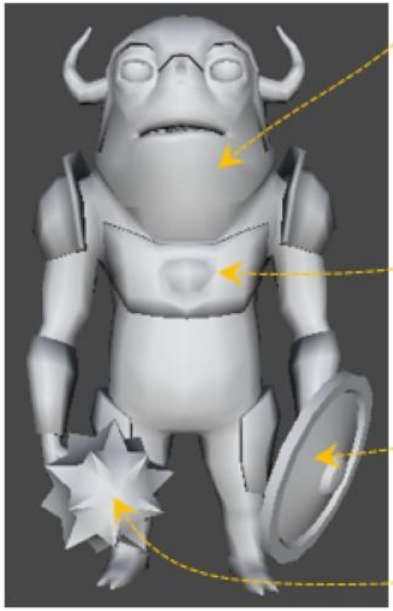
애니메이션 시스템

- 3D 모델(3D Model) 임포트 – Model(Mesh) 설정





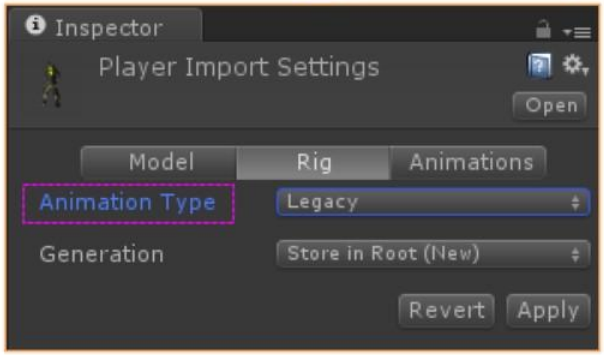


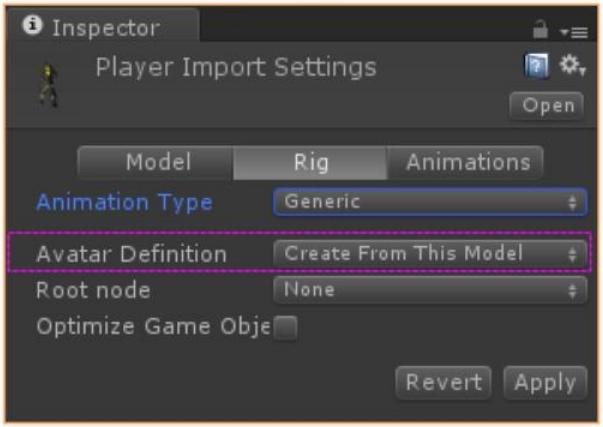


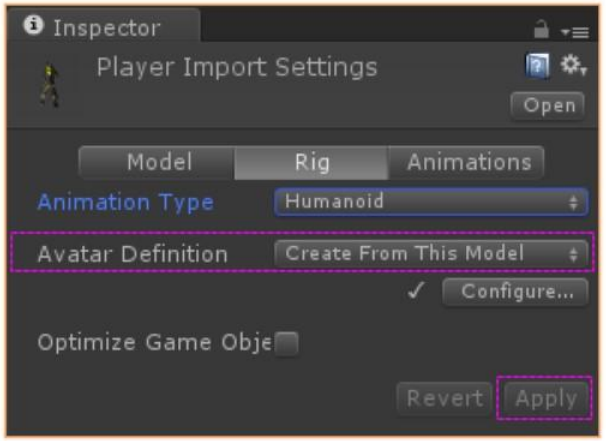


애니메이션 시스템

- 3D 모델(3D Model) 임포트 – Rig 설정







Animation Type

None	애니메이션 없음
Legacy	3.x 애니메이션
Generic	일반적인 메카님 애니메이션
Humanoid	인간 메카님 애니메이션

Avatar Definition

Create from this model	이 모델에서 아바타를 생성
Copy from other Avatar	다른 모델의 아바타 설정을 사용

아바타(Avatar): 하나의 뼈대 구조를 다른 뼈대 구조와 일치(Retargeting)하기 위한 인터페이스(매핑)

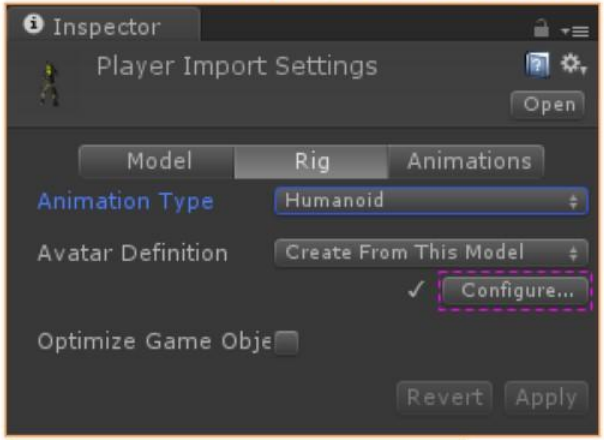
Animation Type	애니메이션의 유형(None, Legacy, Generic, Humanoid)
Root node	Generic 애니메이션 뼈대 계층 구조의 루트 노드를 설정
Avatar Definition	아바타 정의(뼈대 구조)를 어디에서 가져오는가
Configure...	자동적인 아바타 생성 **"Humanoid"를 선택하고 Apply를 누르면 뼈대 구조를 자동적으로 일치시키려고 함** 성공하면 옆에 √ 표시되고 "Avatar" 애셋이 모델 애셋에 추가됨 실패하면 옆에 x 표시, 이 버튼을 눌러서 세부적인 뼈대 매핑을 할 수 있음



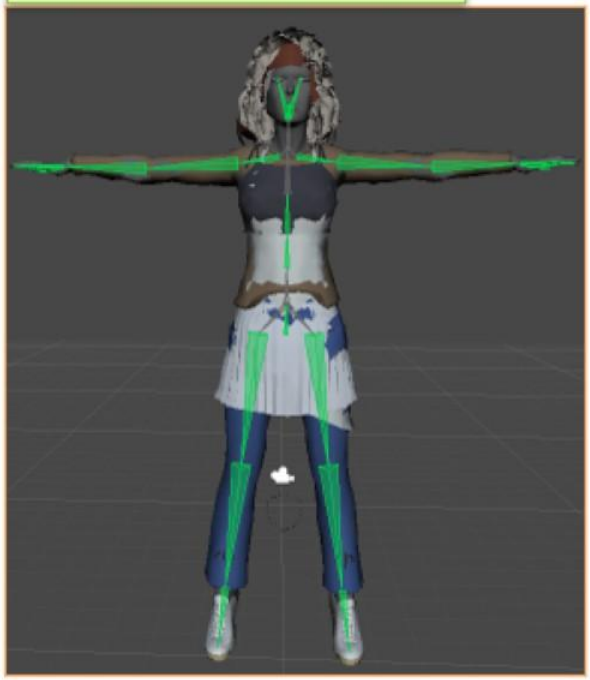
애니메이션 시스템

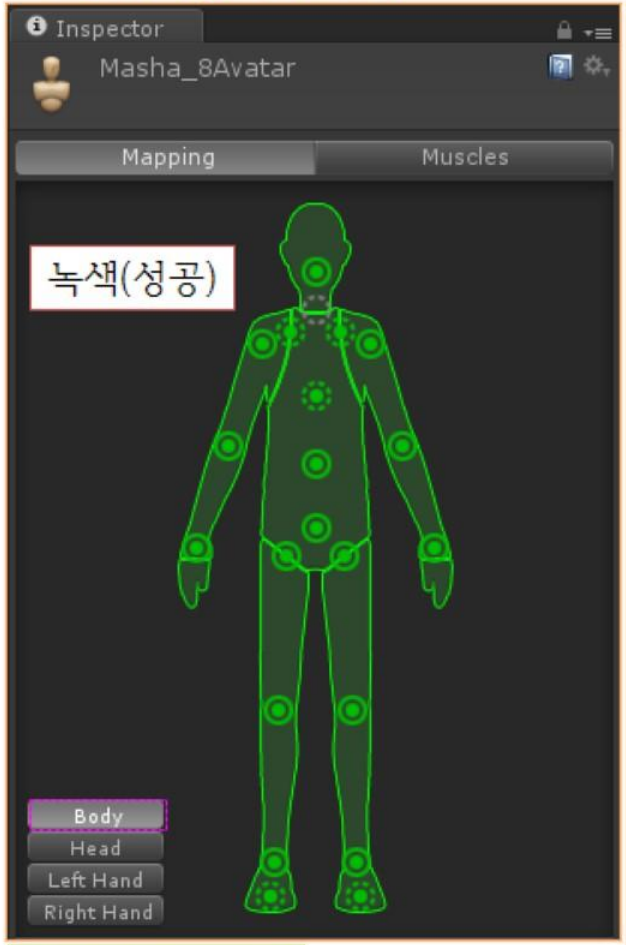
- 아바타 수동 생성(Avatar Configuration)

15개의 관절: 뼈대(Bone)

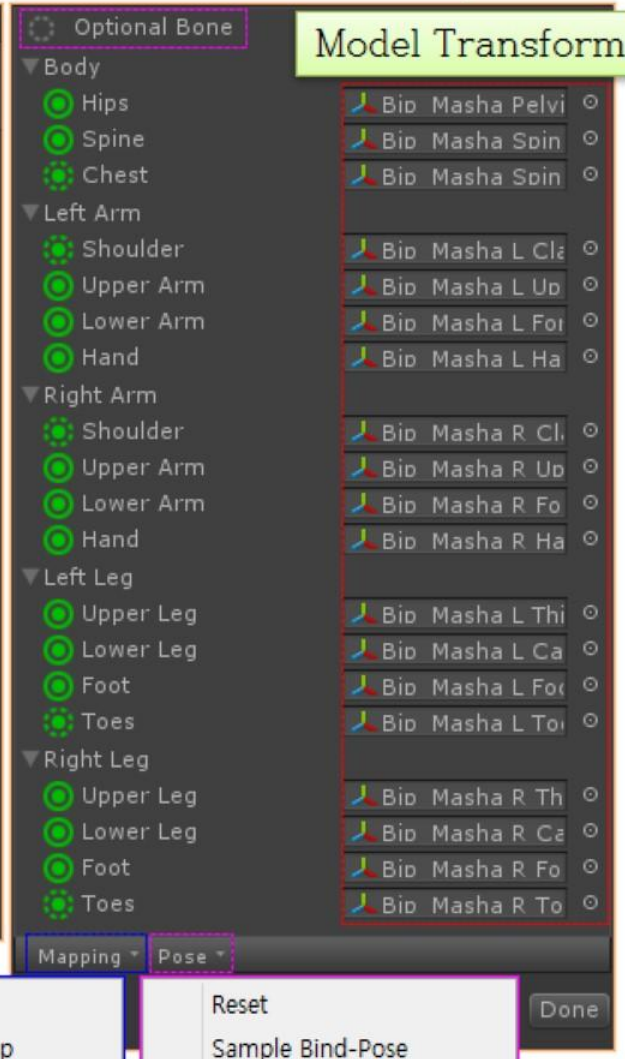


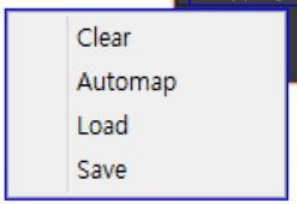
계층 뷰: T-포즈(T-Pose)

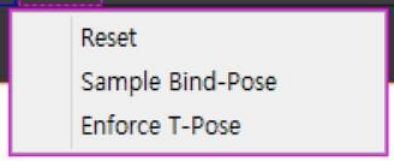




아바타(Avatar)







애니메이션 시스템

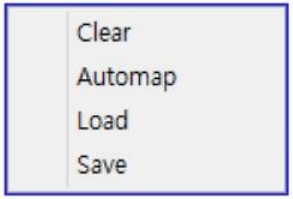
- **아바타 수동 생성(Avatar Configuration)**



설정 윈도우를 닫음

자동적으로 뼈대의 구조를 일치하지 못한 경우 다음 순서로 시도
① Sample Bind-Pose: 바인드 포즈(Bind-Pose) 샘플링
모델링할 때의 포즈와 가장 유사한 모델을 얻기 위함
② "Automap"
초기 포즈에서 뼈대 매핑을 생성
실패하면 수동으로 뼈대를 설정할 수 있음(계층 뷰에서 드래그)
매핑이 성공하면 녹색, 실패하면 적색으로 표시
③ "Enforce T-Pose"
모델을 기본 포즈인 T-포즈에 유사하도록 만듬

- 캐릭터의 뼈대 구조가 메카님의 뼈대 구조와 일치해야 함
모델링할 때 뼈대의 이름을 아바타의 뼈대 이름과 유사하게 지을 것
("LeftArm", "RightForearm")
- 캐릭터의 모델이 T-포즈이어야 함
- 골반(Hip)이 뼈대 계층구조의 루트가 되어야 함
- 최소 15개의 뼈대가 필요함
- 아바타를 생성한 후 모델의 스케일을 변경하면 다시 매핑해야 함
Pose ▶Reset ▶Apply

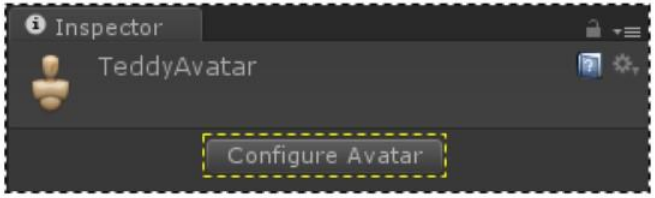


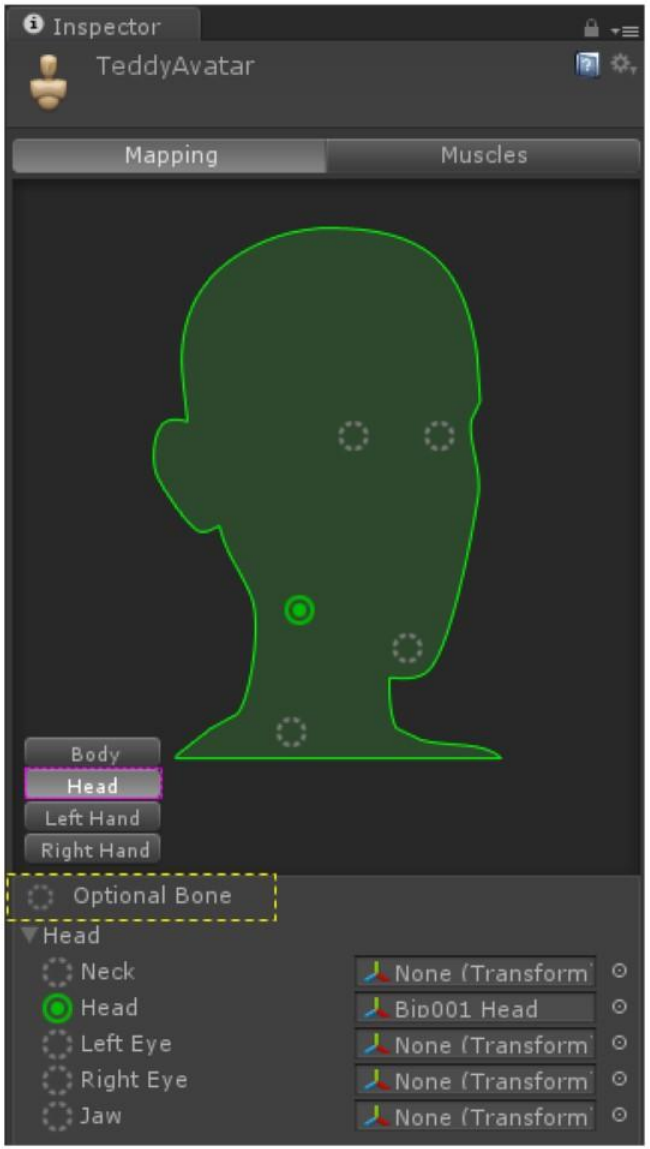


애니메이션 시스템

- **메카님 애니메이션 시스템(Mecanim Animation System)**
  - 아바타 설정(Configure Avatar)
    아바타가 설정되면 메카님이 뼈대 구조를 이해
    아바타에 대한 세부 설정을 할 수 있음
  - 뼈대 매핑
    몸통, 머리, 왼손, 오른손에 대한 뼈대 매핑

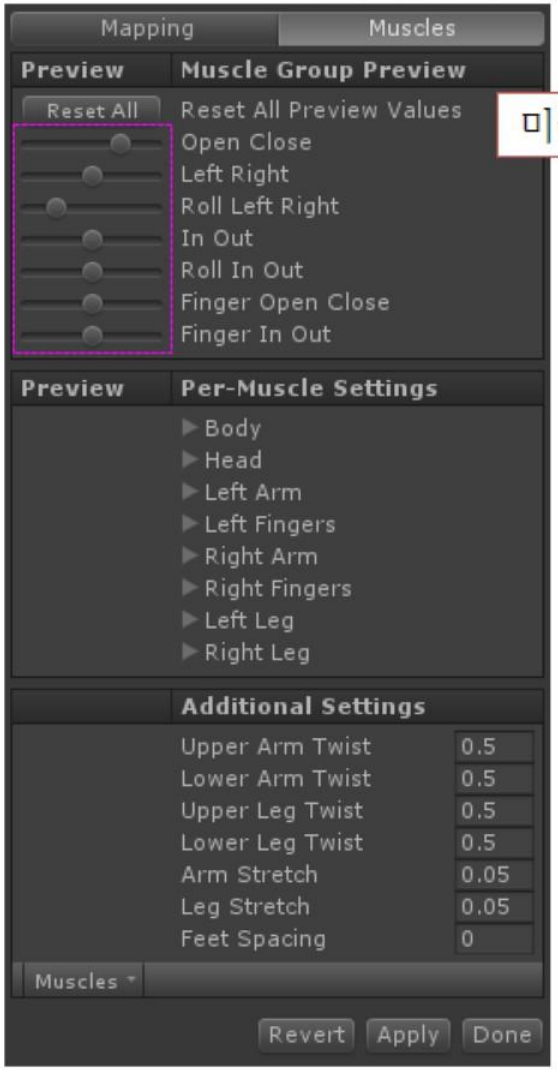




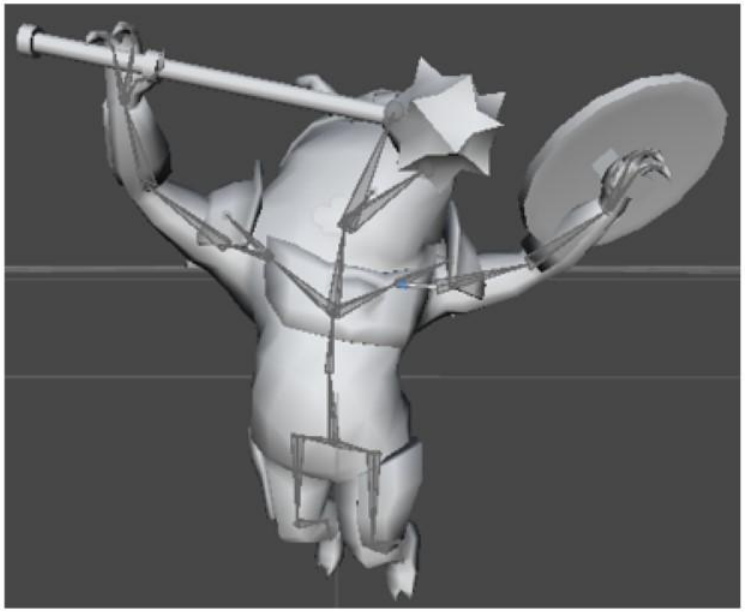


애니메이션 시스템

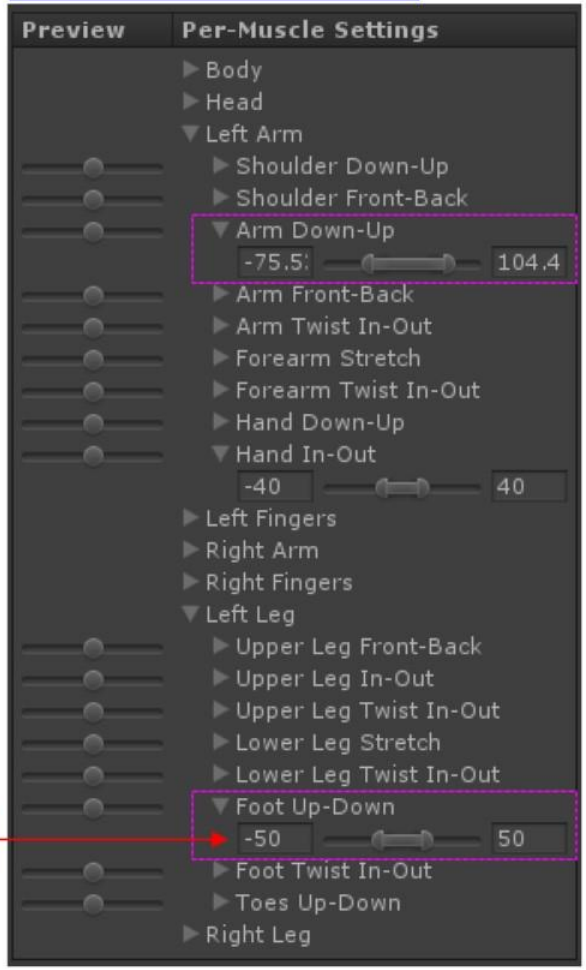
- **메카님 애니메이션 시스템(Mecanim Animation System)**
  - 근육(Muscle) 설정
    아바타 설정 후 근육을 사용하여 다른 뼈대들이 움직이는 범위를 제어할 수 있음



미리 정해진 근육들을 사용하여 움직임 확인



개별 근육에 대한 설정



관절의 회전 범위

애니메이션 시스템

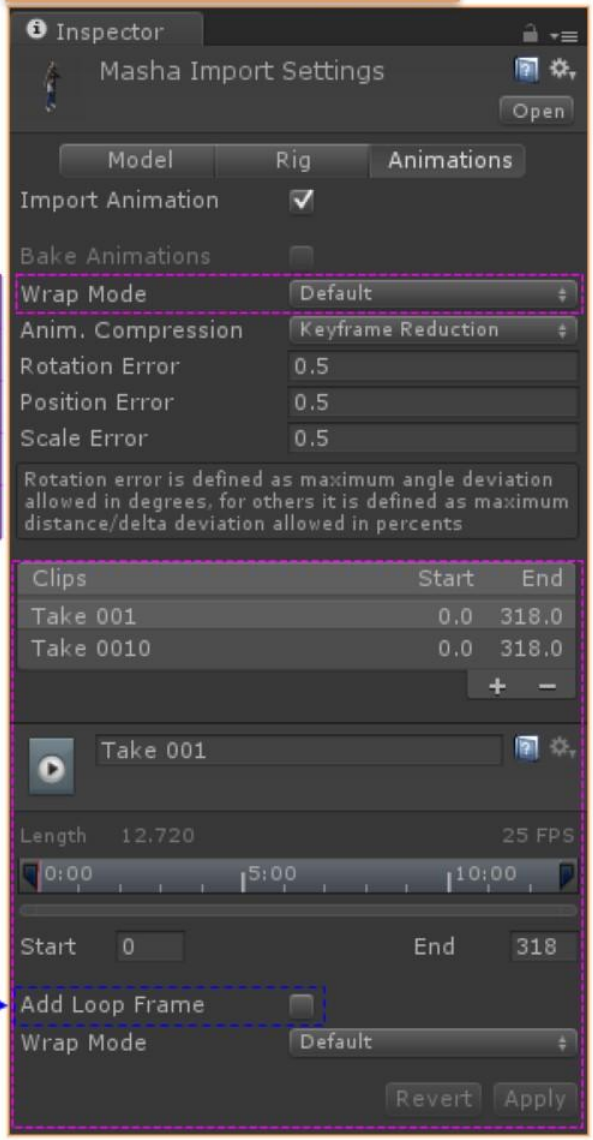
- 3D 모델(3D Model) 임포트 – Animations 설정

Animation Type: **Humanoid**



- Off
- Keyframe Reduction
- Optimal

Animation Type: **Legacy**



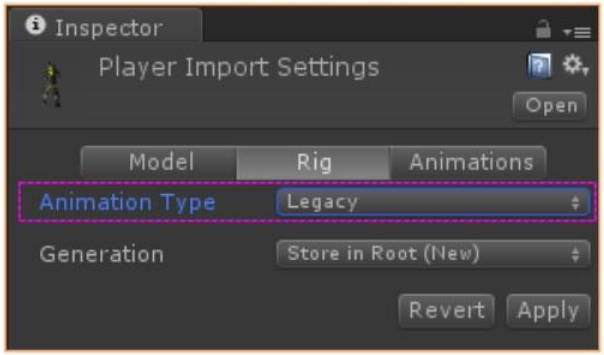
- Default
- Once
- Loop
- PingPong
- ClampForever

끝과 시작 키프레임과 일치하도록 함

Import Animation	애니메이션을 임포트하는가
Bake Animations	Maya, 3dsMax, Cinema4D 파일에만 사용
Anim. Compression	애니메이션 압축 방법
Rotation Error	두 개의 키프레임이 같게 취급되는 오차(각도)
Position Error	두 개의 키프레임이 같게 취급되는 오차(좌표의 백분율)
Scale Error	두 개의 키프레임이 같게 취급되는 오차(좌표의 백분율)

애니메이션 시스템

- 레거시(Legacy) 애니메이션 시스템





Import Animation	애니메이션을 임포트하는 여부
Wrap Mode	애니메이션 래핑 모드
Anim Compression	불필요한 부분을 제거하거나 압축하는 방법
Rotation error	회전 값의 최소 차이(각도) 이 값보다 작은 두 키프레임은 같은 것으로 취급
Position error	위치 값의 최소 차이(좌표값의 백분율) 이 값보다 작은 두 키프레임은 같은 것으로 취급
Scale error	크기 값의 최소 차이(좌표값의 백분율) 이 값보다 작은 두 키프레임은 같은 것으로 취급

Default	애니메이션 클립의 설정을 사용
Once	애니메이션을 한번 재생
Loop	애니메이션을 반복 재생
PingPong	핑퐁 모드(끝에서 역방향 재생)
ClampForever	끝에서 마지막 프레임을 계속 재생

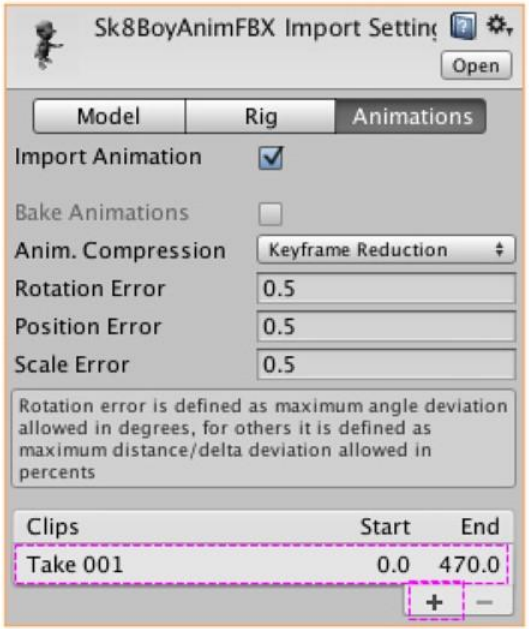
Off	압축하지 않음
Keyframe reduction	너무 작은 키프레임을 제거
Keyframe reduction and compression	클립 데이터 압축

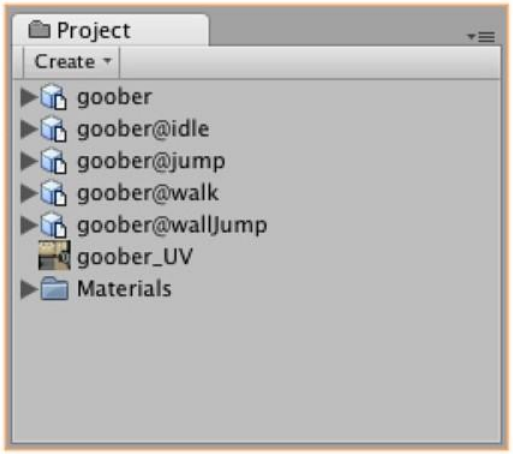
애니메이션 시스템

- **캐릭터 컨트롤러(Character Controller)**
  - 새로운 프로젝트 생성
    평면(Plane)을 생성(아주 크게 만들것)
  - **Assets ▶ Import Package ▶ Characters**
  - 프로젝트 뷰(Project View)
    Standard Assets/Characters/ThirdPersonCharacter/Prefabs
    "ThirdPersonController" 프리팹을 객체로 생성하고 프로젝트 실행
    "Ethan" 캐릭터 모델
  - "ThirdPersonCharacter" "ThirdPersonUserControl" 스크립트
    Standard Assets/Characters/ThirdPersonCharacter/Scripts
  - 캐릭터 모델 바꾸기
    ① 원하는 캐릭터 모델을 임포트
    "Humanoid"로 리깅(Rigging)
    게임 객체로 생성
    ② 계층 뷰에서 "ThirdPersonCharacter"의 모든 자식 객체를 삭제
    ③ 임포트한 캐릭터 모델을 "ThirdPersonCharacter"의 자식 객체로 만듬
    자식 객체의 위치(Position)를 (0, 0, 0)으로 설정
    ④ 임포트한 모델의 아바타(Avatar)를 "ThirdPersonCharacter"의 애니메이터
    컴포넌트의 아바타로 설정
    ⑤ 실행
  - 3인칭 카메라 구현 스크립트
    Standard Assets/Utility/(SmoothFollow | FollowTarget)

애니메이션 시스템

- **3D 모델(3D Model) 임포트 – Animations 설정**
  - 애니메이션 클립(Animation Clip)
    일반적으로 하나의 모델에는 여러 개별적인 동작이 포함되어 있음
    (예: 걷기, 달리기, 점프하기, 던지기, 등)
    이러한 개별적인 동작을 별도의 애니메이션 클립으로 임포트할 필요가 있음
    - 분리된 애니메이션을 가진 모델
      모델을 임포트할 때 애니메이션 클립을 자동으로 생성
      여러 개의 모델 파일에서 애니메이션을 임포트 가능
    - 분리되지 않은 애니메이션을 가진 모델
      모델을 임포트한 후 애니메이션 클립을 직접 생성



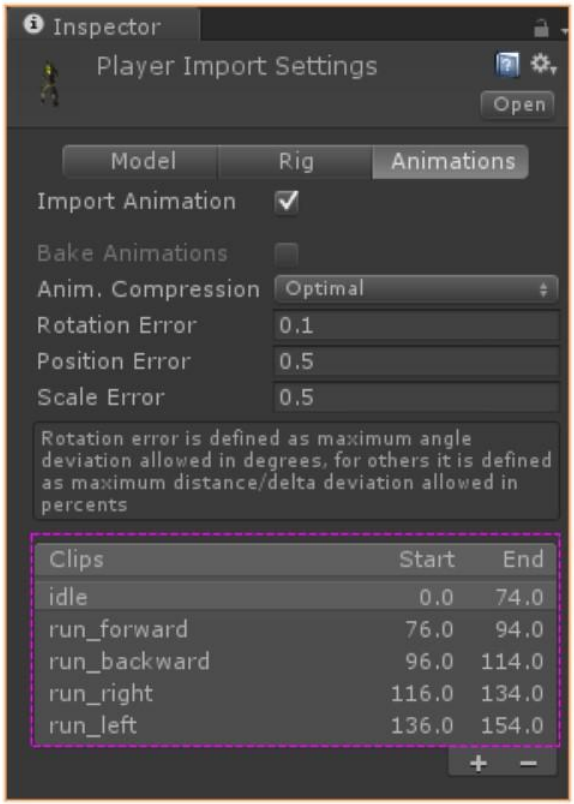




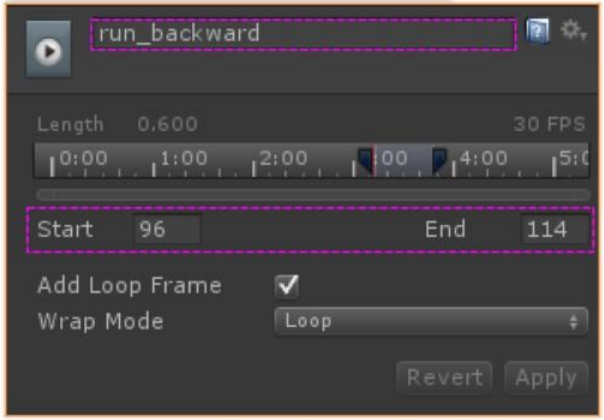
"modelName@animationName.fbx"

애니메이션 시스템

- **3D 모델(3D Model) 임포트**
  - 애니메이션 클립(Animation Clip)



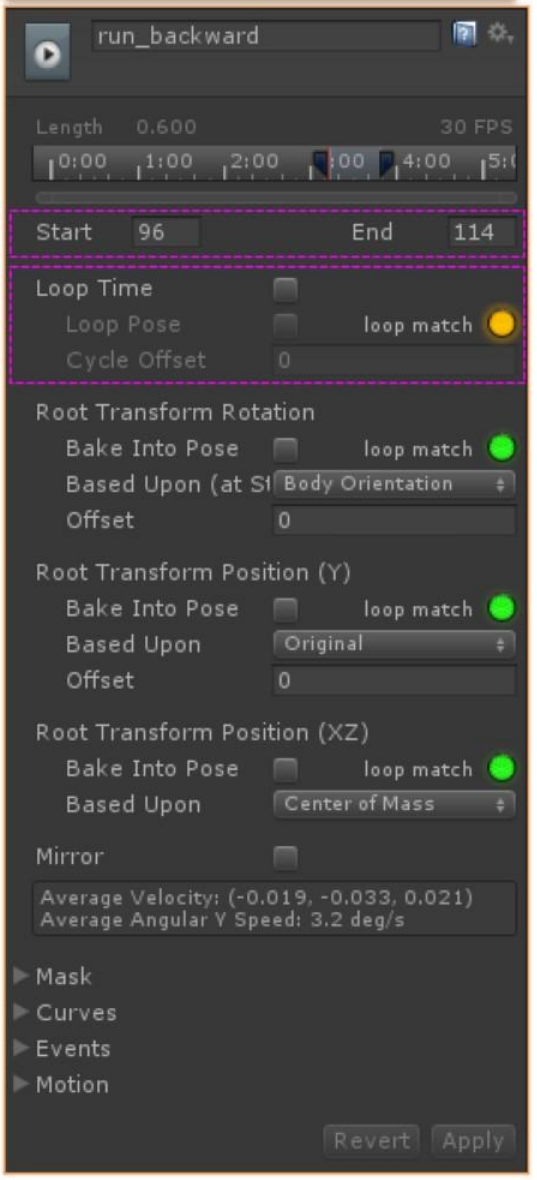
Animation Type: Legacy



Name	애니메이션 클립의 이름
Start	클립의 시작 프레임
End	클립의 끝 프레임
Add Loop Frame	끝과 시작이 같도록
Wrap Mode	클립의 랩 모드
Loop Time	애니메이션 클립의 반복
Loop Pose	반복을 부드럽게
Cycle Offset	반복하는 사이클의 오프셋
Mirror	왼쪽과 오른쪽을 미러링

Default
Once
Loop
PingPong
ClampForever

Animation Type: Humanoid



애니메이션 시스템

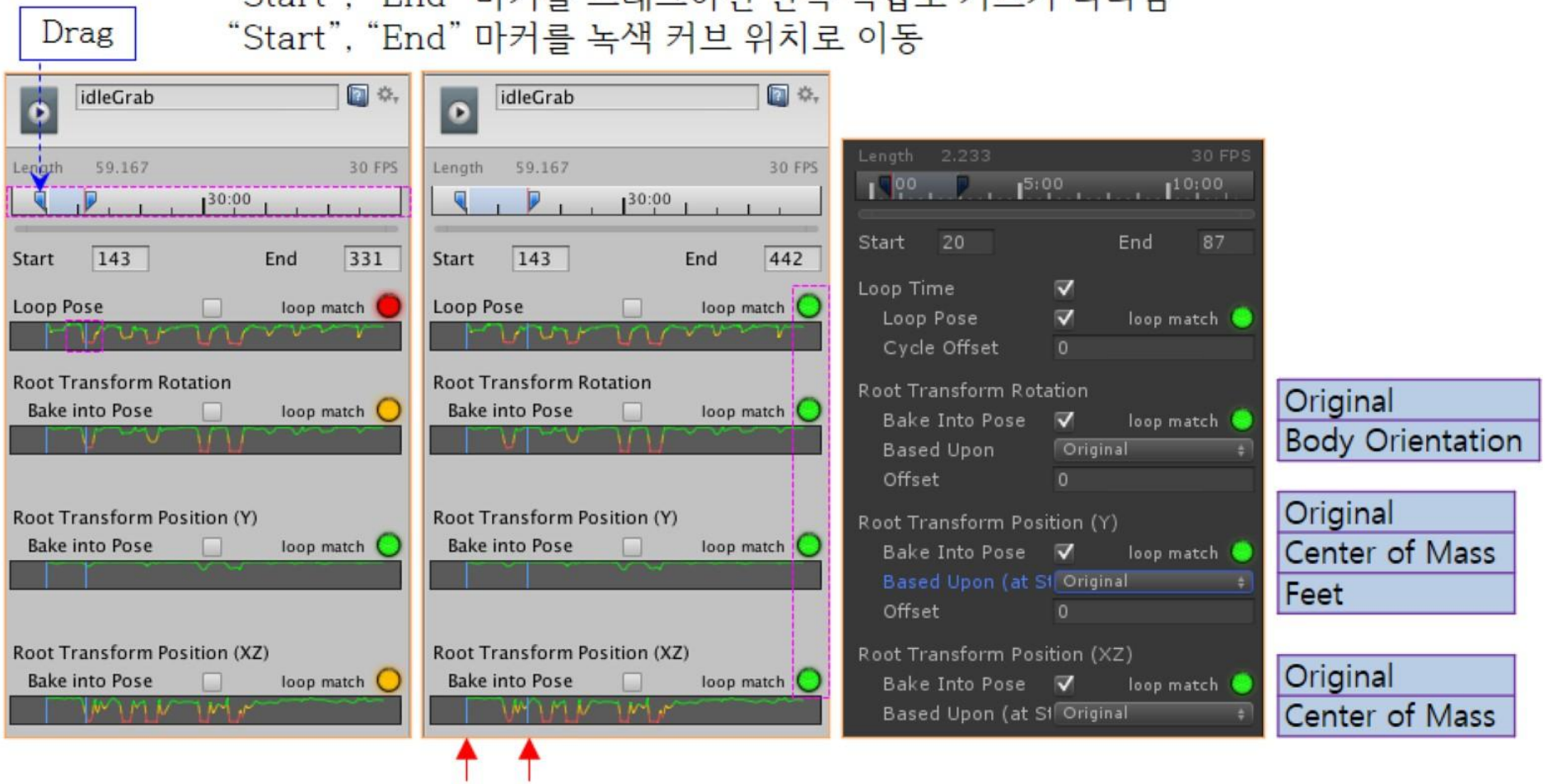
- **3D 모델(3D Model) 임포트** – Animations 설정

Start, End	애니메이션 클립의 시작/끝 프레임
Loop Pose	모션 루프를 이어줌
Cycle Offset	반복 애니메이션의 오프셋
Root Transform Rotation	
Bake into Pose	루트 회전을 뼈대들의 움직임으로 포함
Based Upon	어떤 루트 회전을 기반으로 하는가
- Original	원본의 회전을 유지
- Body Orientation	상체가 앞을 향하도록 유지
Offset	루트 회전에 대한 오프셋 각도
Root Transform Position (Y)	
Bake into Pose	수직 방향의 루트 움직임을 뼈대들의 움직임으로 포함
Based Upon	어떤 수직 방향의 루트 위치를 기반으로 하는가
- Original	원본의 수직 위치를 유지
- Center of Mass	무게중심이 루트 변환 위치와 일치하도록 유지
- Feet	발이 루트 변환 위치와 일치하도록 유지
Offset	수직 루트 위치에 대한 오프셋
Root Transform Position (XZ)	
Bake into Pose	수평 방향의 루트 움직임을 뼈대들의 움직임으로 포함
Based Upon	어떤 수평 방향의 루트 위치를 기반으로 하는가
- Original	원본의 수평 위치를 유지
- Center of Mass	무게중심이 루트 변환 위치와 일치하도록 유지
Mirror	애니메이션 클립의 왼쪽과 오른쪽을 같게 만듬
Body Mask	애니메이션 클립에 적용된 바디 매스크
Curves (Unity Pro)	파라메터-관련 커브



애니메이션 시스템

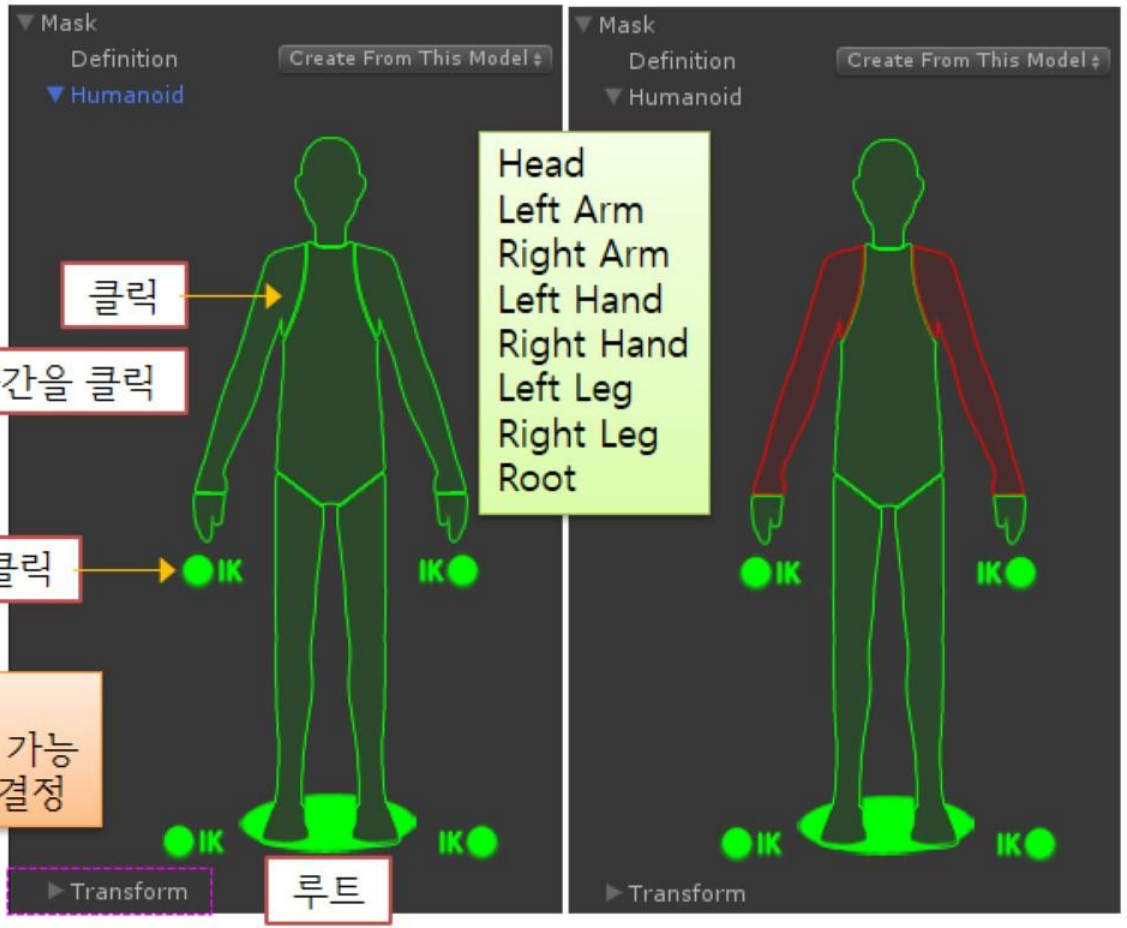
- **3D 모델(3D Model) 임포트 – Animations 설정**
  - 애니메이션 반복(Looping Animation Clip)
    일반적으로 애니메이션 동작이 잘 반복(Loop)되도록 하는 것이 필요
    (예: 걷기 동작의 시작과 끝이 유사한 포즈이어야 함)
    애니메이션 클립을 포즈, 회전, 위치에 기반하여 반복되도록 할 수 있음
    "Start", "End" 마커를 드래그하면 반복 적합도 커브가 나타남
    "Start", "End" 마커를 녹색 커브 위치로 이동



애니메이션 시스템

- **메카님 애니메이션 시스템(Mecanim Animation System)**
  - 아바타 바디 매스크(Body Mask)
    애니메이션 클립을 몸체의 특정 부위에 선택적으로 적용하도록 설정 가능
    현재 애니메이션 동작에 따라 움직이지 않을 부위를 선택(재생하여 확인)
    재사용할 수 있는 아바타 바디 매스크 애셋 생성
    Assets ▶ Create ▶ Avatar Mask

걷기 애니메이션(팔/다리를 움직임)
양손에 큰 물건을 들고 있다면?
양팔의 애니메이션을 적용하지 않음

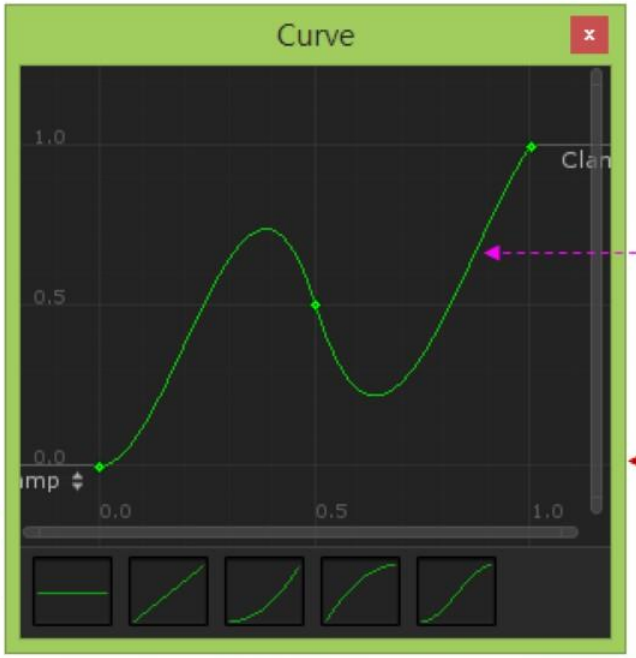


손과 발에 IK를 설정할 수 있음
(IK: Inverse Kinematics)
애니메이션 블렌딩에 IK 커브를 포함

변환 매스크(Transform Mask)
현재 애니메이션 클립에 적용될 변환을 선택 가능
어떤 애니메이션 커브가 임포트될 것인가를 결정

애니메이션 시스템

- **3D 모델(3D Model) 임포트** – Animations 설정
  - 메카님 애니메이션 커브(Animation Curve)
    애니메이션 클립에 애니메이션 커브를 연결할 수 있음
    애니메이션 커브 편집기(Curve Editor)



Wrap 모드

키 추가(Add Key)
오른쪽 마우스 클릭
Shift, Delete

Wrap 모드

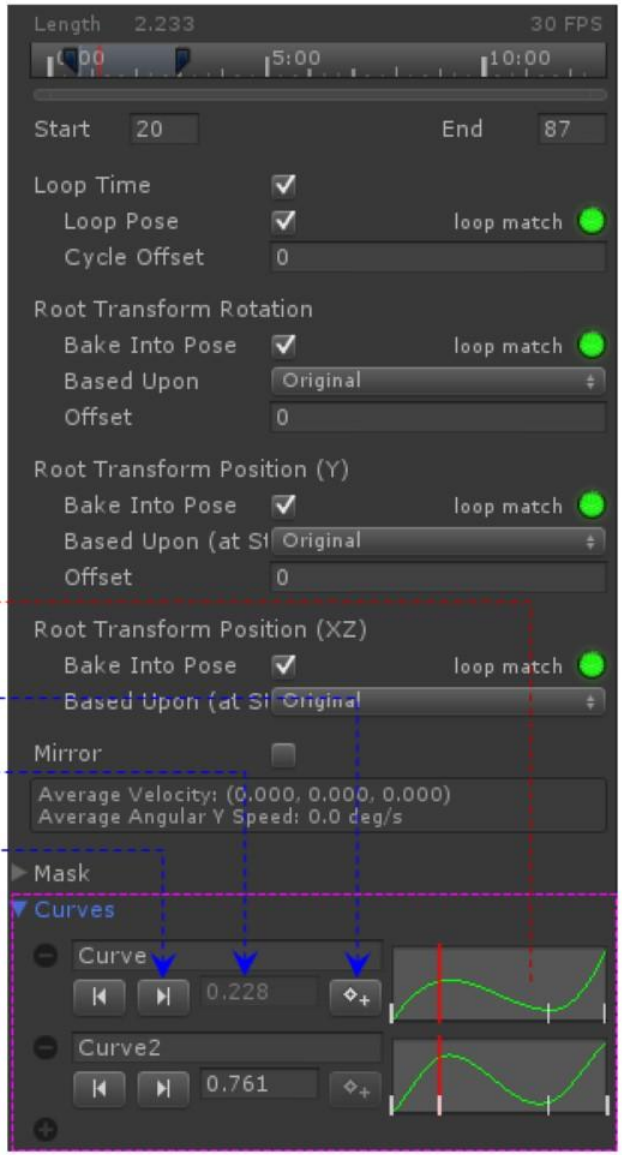
- Loop
- Ping Pong
- Clamp

x-축: 정규화된 시간(0.0~1.0)

키프레임 추가

커브의 y-값

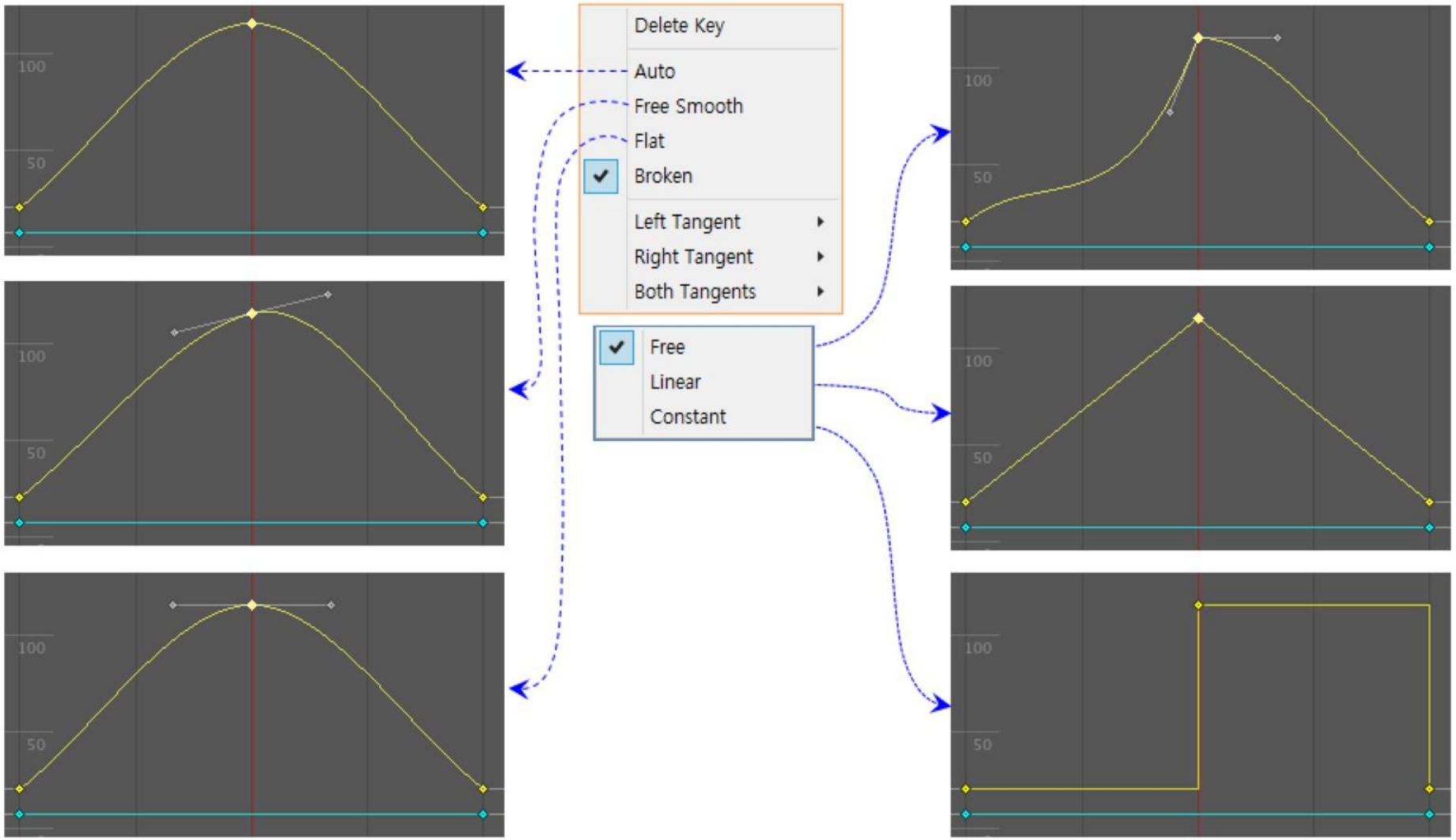
키 이동(Previous/Next)



- 커브 이름이 애니메이터 컨트롤러의 파라메터 이름과 같으면 파라메터의 값은 Getxxx() 함수가 호출되는 시간에 대하여 커브에 따라 반환됨
- 여러 애니메이션 클립의 커브가 같은 파라메터 이름을 가지면 파라메터의 값은 여러 커브의 값을 블렌딩하여 결정
- 파라메터를 위한 커브가 존재하지 않으면 파라메터의 기본값을 사용

애니메이션 시스템

- **3D 모델(3D Model) 임포트** – Animations 설정
  - 애니메이션 커브 편집기(Curve Editor)
    키를 마우스 오른쪽 버튼 클릭



애니메이션 시스템

- **애니메이션 뷰(Animation View)**
  - 애니메이션 속성 편집
    - 편집할 수 있는 애니메이션 속성
      - 실수(Float)
      - 색상(Color)
      - 벡터(Vector2, Vector3, Vector4)
      - 쿼터니언(Quaternion)

    배열, 구조체, 객체, 스크립트 컴포넌트의 bool은 지원하지 않음
    유니티 컴포넌트의 bool은 지원
  - 애니메이션 속성의 예
    - 조명(Light)의 깜박임, 번쩍임: 색상(Color), 밝기(Intensity)
    - 바람 소리, 엔진 소리, 물 소리: 오디오 소스의 피치(Pitch), 볼륨(Volume)
    - 벨트의 이동, 흐르는 물: 재질(Material)의 텍스쳐 오프셋
    - 불꽃, 분수: 파티클의 상태, 속도
    - 스크립트 컴포넌트의 변수
  - 애니메이션 샘플링(Sampling)
    - 애니메이션을 제작할 때 프레임 레이트(고정): 60Hz
    - 게임의 프레임 레이트(가변): 56.72Hz, 83.14Hz

    결과적으로 엔진은 가변 프레임 레이트로 애니메이션 커브를 샘플링해야 함
    (애니메이션을 반복할 때마다 애니메이션은 조금씩 다르게 보일 가능성이 있음)
  - 애니메이션 게임 로직

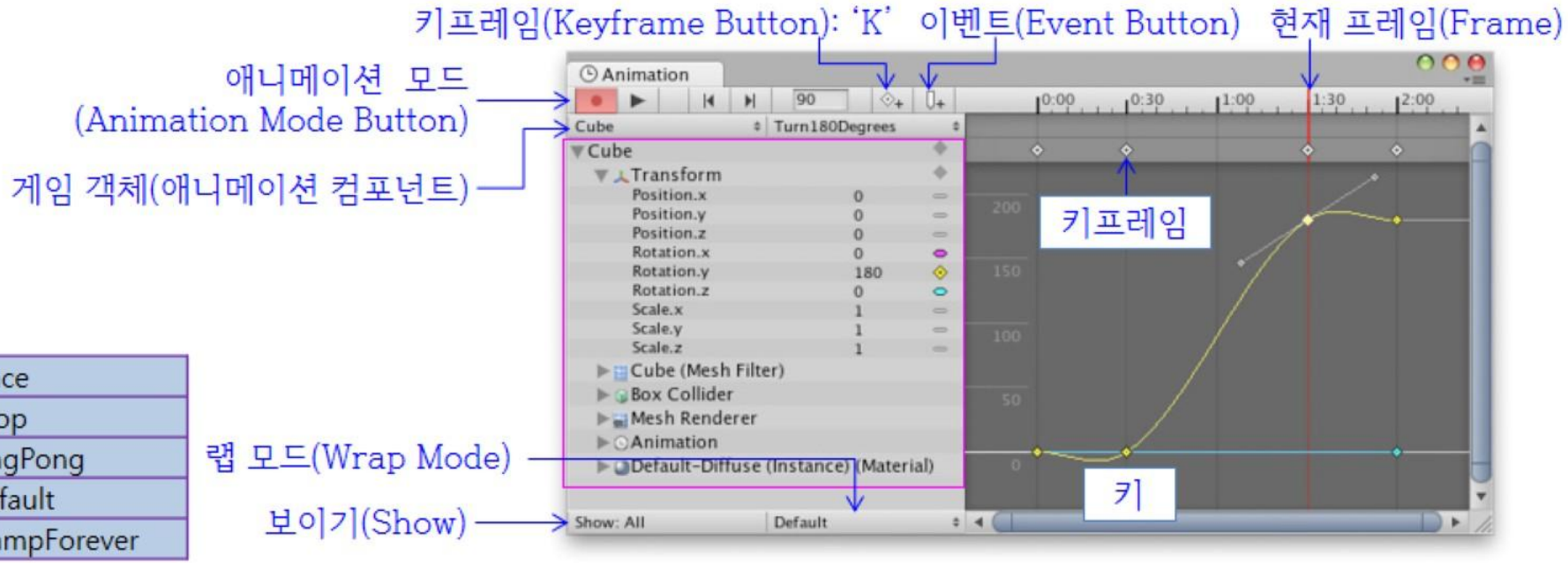
    특정 시간에 애니메이션 속성 값을 비교하는 코드는 잘 동작하지 않을 것임
    (예: 프레임 마다 30도씩 0도에서 180까지 회전하는 객체가 90도일 때 ...)

애니메이션 시스템

- **애니메이션 뷰(Animation View)**
  - 선택한 게임 객체를 위한 애니메이션 클립을 생성하거나 수정할 수 있음
  - Window ▶ Animation

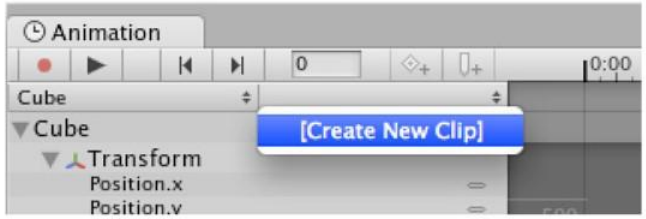
    애니메이션 가능 속성(Animatable Properties)

    컴포넌트(Component), 재질(Material), 자식 게임 객체(Child Game Objects)



  - 애니메이션하기 위해서 애니메이션 컴포넌트(Animation Component)가 필요

    애니메이션 클립을 생성할 때 또는 애니메이션 모드로 들어갈 때 자동으로 생성
  - 새로운 애니메이션 클립 생성



애니메이션 시스템

- **애니메이션 뷰(Animation View)**
  - 애니메이션 클립 편집
    - 애니메이션 모드: 애니메이션 모드 버튼 토글(Toggle)
      게임 객체의 변화가 애니메이션 클립의 데이터로 저장
    - 타임 라인(Time Line): 초:프레임



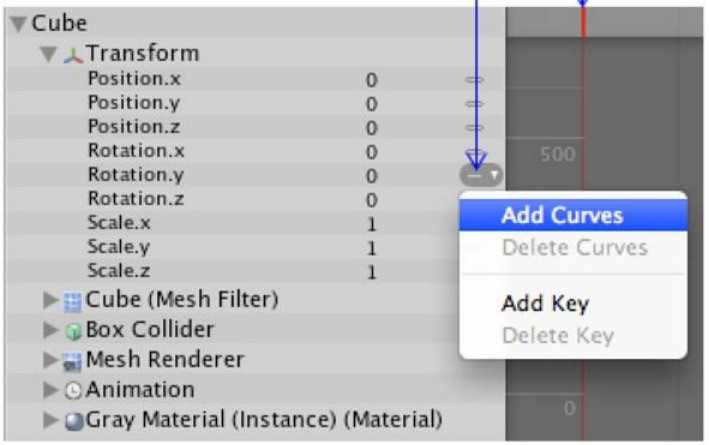
    - 키프레임 추가:
    - 재생:
      보이는 타임 라인
      전체 재생: 'F'

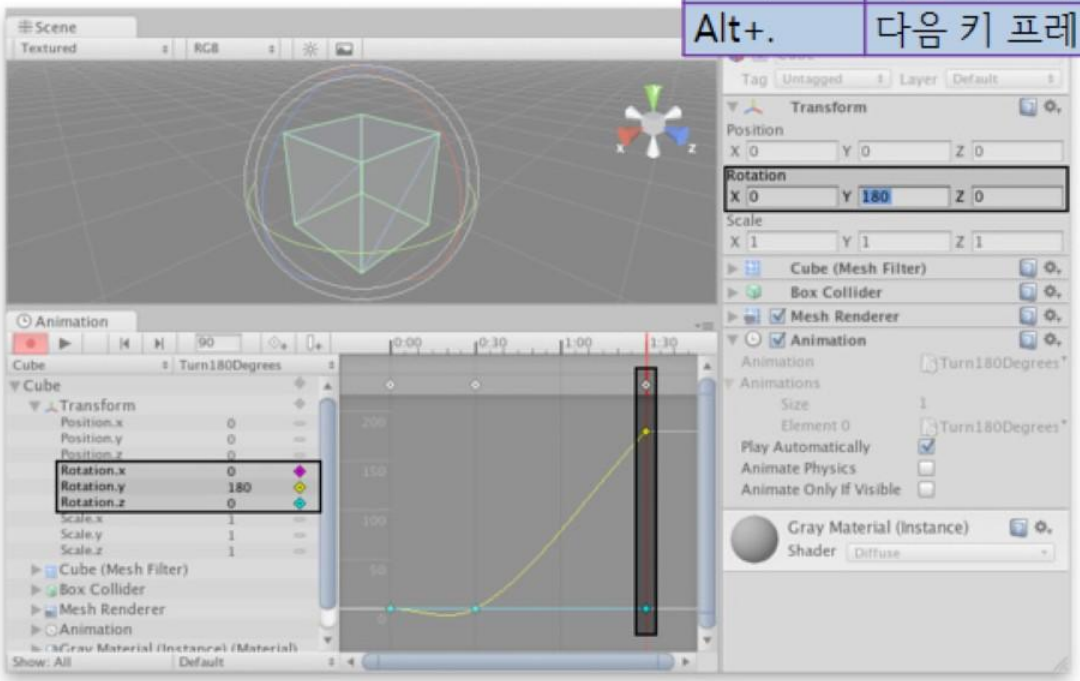
다음 프레임

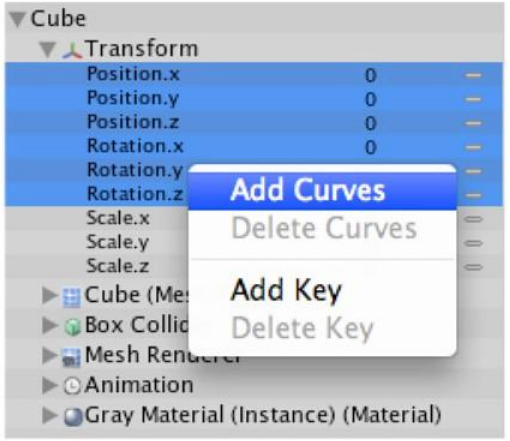
쉼표(,)	이전 프레임
마침표(.)	다음 프레임
Alt+,	이전 키 프레임
Alt+.	다음 키 프레임

현재 프레임(Frame)

키 인디케이터(Key Indicator)





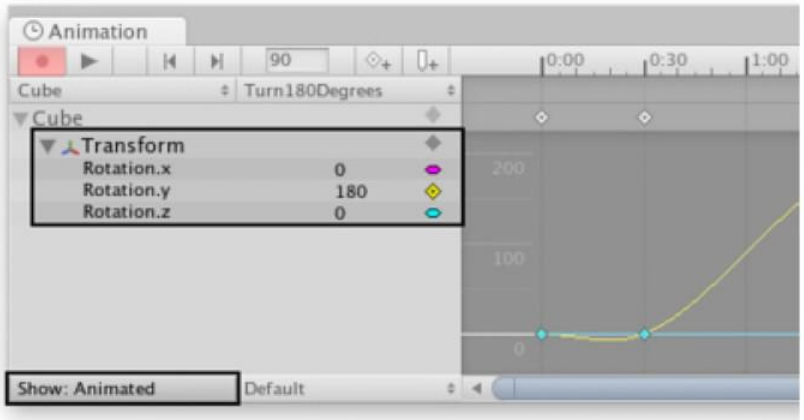


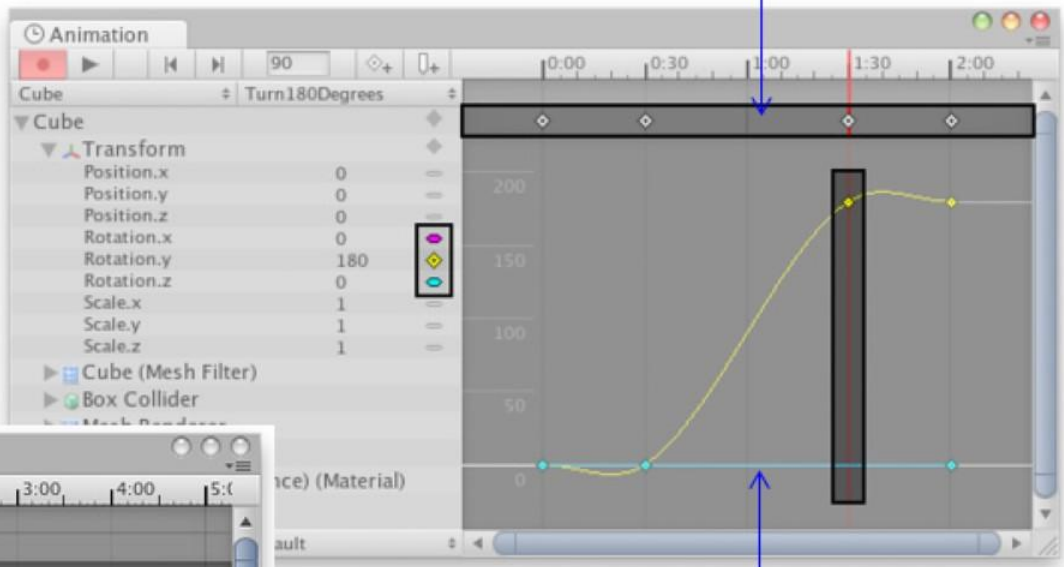
애니메이션 시스템

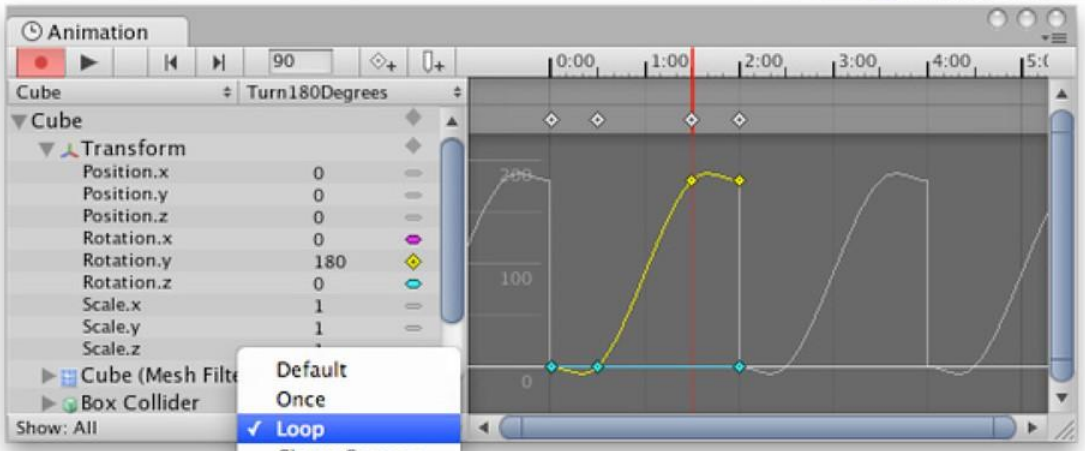
- **애니메이션 뷰(Animation View)**
  - 애니메이션 커브 편집
    - 키(Key): 커브의 제어점(Control Point)
    - 키프레임(Keyframe): 키를 가지고 있는 프레임

커브 편집: 키 선택(클릭, 토글) ▶ 드래그
여러 키 선택: Shift + 클릭, 사각형 영역 선택

키프레임 라인(Keyframe Line)







키프레임 추가

더블 클릭



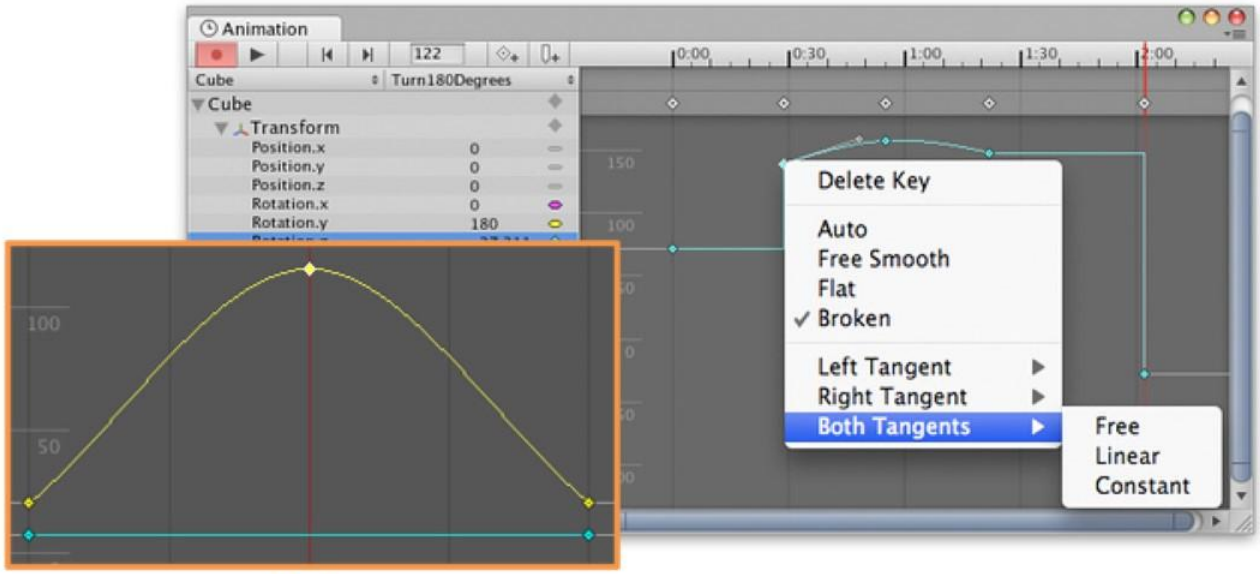
키프레임 삭제: Del

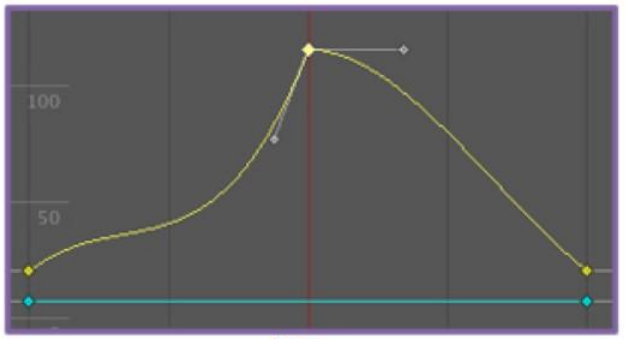
오른쪽 마우스 버튼
(Add Keyframe)
(Delete Keyframe)

포커스: F, 눈금 스냅: Control + 왼쪽 마우스 버튼 드래그
줌: 스크롤 휠, Alt + 오른쪽 마우스 버튼 드래그

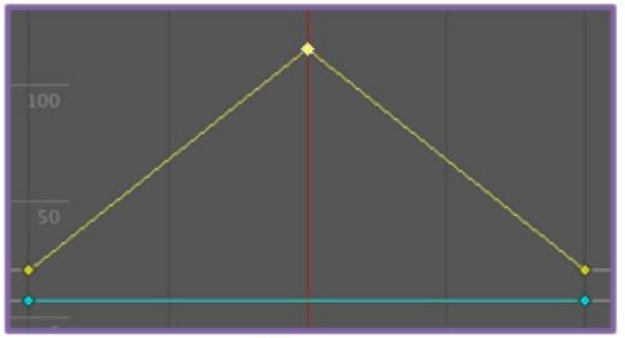
애니메이션 시스템

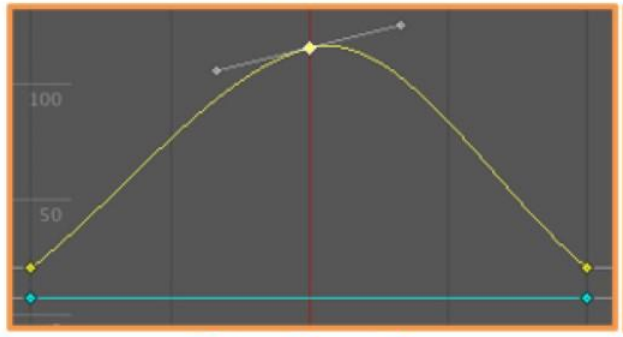
- **애니메이션 뷰(Animation View)**
  - 애니메이션 커브의 접선(Tangents) 편집
    하나의 키는 두 개의 접선을 가짐(왼쪽, 오른쪽)
    접선은 키들 사이의 커브의 모양을 제어함

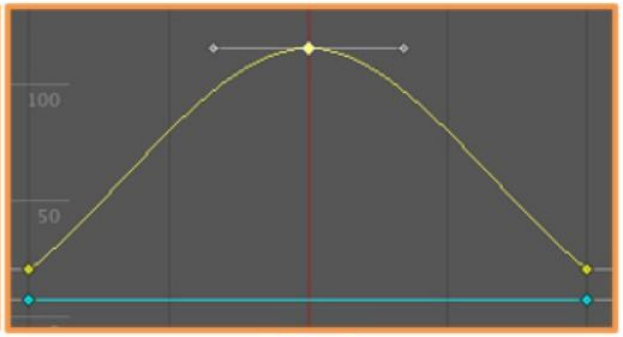


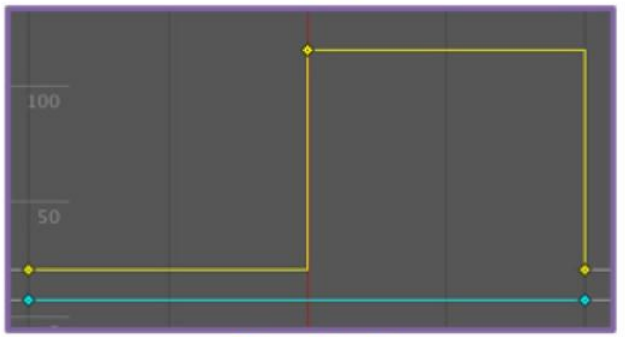

Free

Auto


Linear

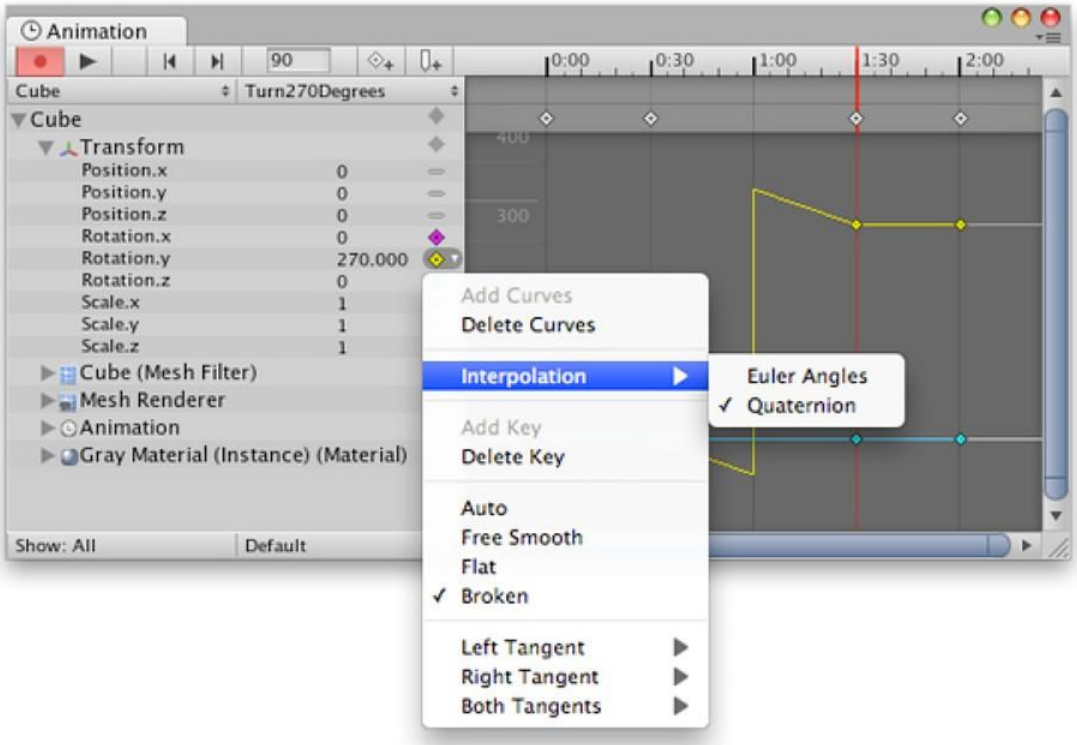

Free Smooth


Flat


Constant

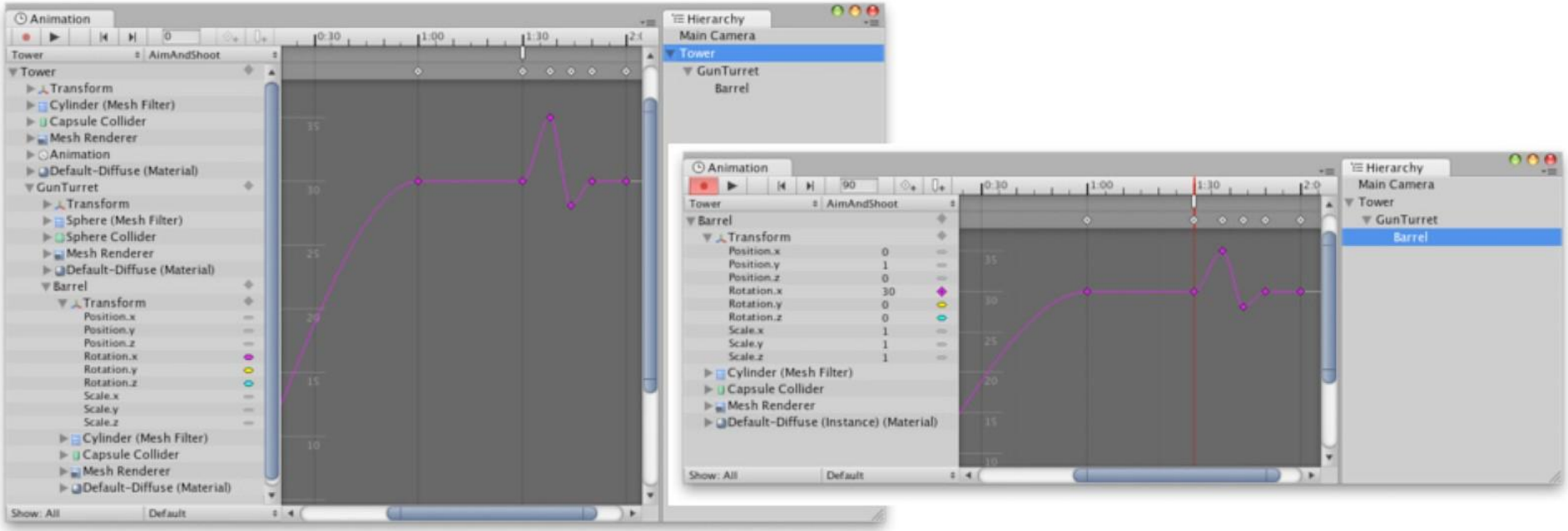
애니메이션 시스템

- **애니메이션 뷰(Animation View)**
  - 애니메이션 커브 편집
    유니티 내부적으로 회전은 쿼터니언으로 표현(단, 오일러 각도로 표시)
    두 개의 회전을 보간할 때 쿼터니언 또는 오일러 각도를 사용할 수 있음
  - 쿼터니언 보간(Quaternion Interpolation)
    두 개의 회전을 보간할 때 짧은 경로를 생성
    180도 보다 큰 회전을 표현 못함(사이에 키를 추가하여 표현해야 함)
  - 오일러 각도 보간(Euler Angles Interpolation)
    내부적으로 쿼터니언 표현으로 처리



애니메이션 시스템

- **애니메이션 뷰(Animation View)**
  - 계층 구조 객체의 애니메이션 커브 편집
    부모 객체의 애니메이션 컴포넌트를 사용하여 자식 객체를 애니메이션 할 수 있음

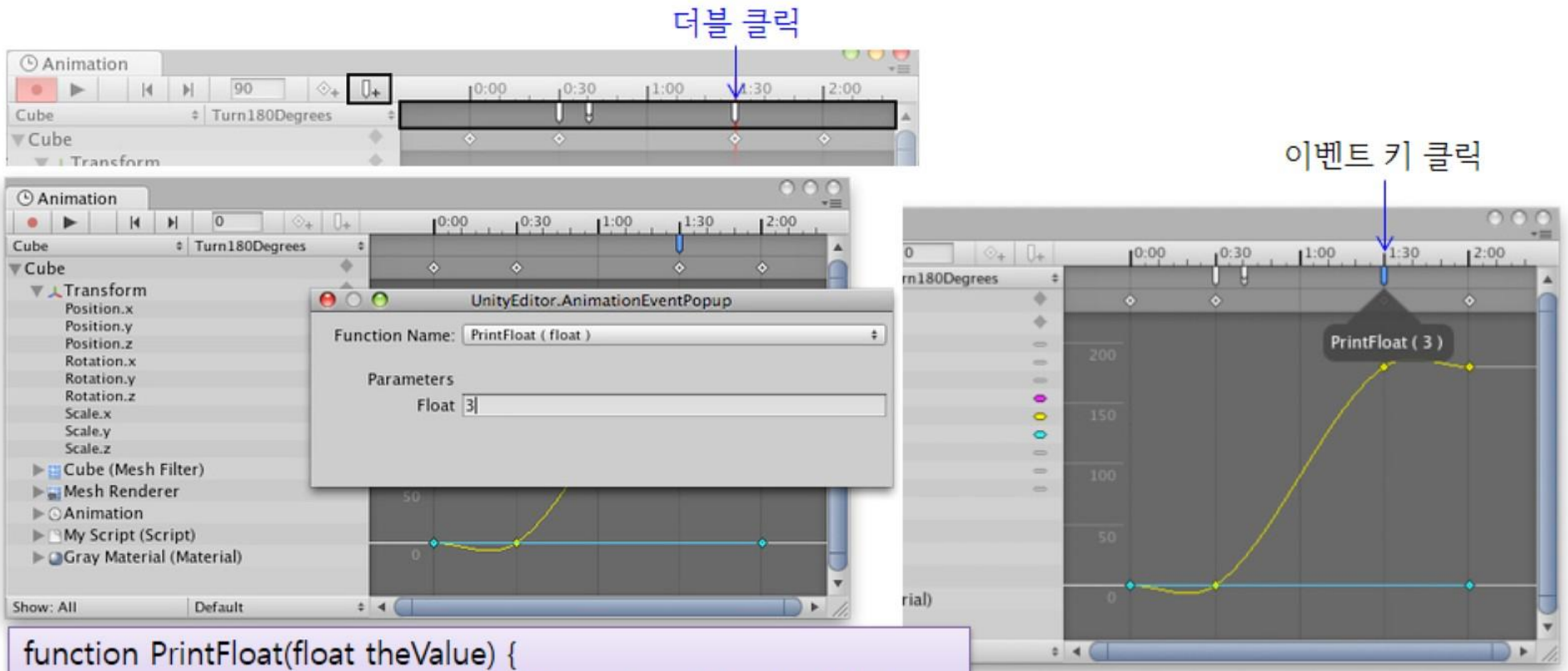


자식 객체가 애니메이션 컴포넌트를 가지면
자식 객체를 애니메이션 하기 위하여
애니메이션 컴포넌트를 선택할 수 있음



애니메이션 시스템

- **애니메이션 뷰(Animation View)**
  - 애니메이션 이벤트(Animation Events)
    특정 시점에 객체의 스크립트를 호출할 수 있음
    애니메이션 이벤트로 호출되는 함수는 하나의 파라메터를 가질 수 있음
    float, string, object, AnimationEvent
    애니메이션 클립에 애니메이션 이벤트를 추가, 이동, 삭제('Delete')




```
function PrintFloat(float theValue) {
    Debug.Log("PrintFloat is called with a value of " + theValue);
}
```

애니메이션 시스템

- **애니메이션(Animation) 클래스**
  - 애니메이션 컴포넌트에 애니메이션 클립을 지정하고 스크립트로 재생할 수 있음
    단순히 애니메이션을 재생: Animation.Play()
    애니메이션을 크로스-페이드: Animation.CrossFade()
    애니메이션 랩 모드 변경: AnimationState.wrapMode
    애니메이션 레이어 변경, 재생 속도 변경: AnimationState

clip	기본 애니메이션, AnimationClip
playAutomatically	기본 애니메이션이 자동적으로 재생을 시작하는가, bool
wrapMode	애니메이션 재생 모드, **WrapMode**
isPlaying	애니메이션이 재생되고 있는가, bool
this [string name]	문자열에 해당하는 애니메이션 상태 객체를 반환, AnimationState
animatePhysics	활성화되면 애니메이션은 물리(Physics) 루프에서 실행됨, bool
cullingType	애니메이션 컴포넌트의 컬링 유형, **AnimationCullingType**
localBounds	로컬 좌표계에서 애니메이션 컴포넌트의 AABB, Bounds

- Default
- Once
- Loop
- PingPong
- ClampForever

- AlwaysAnimate
- BasedOnRenderers
- BasedOnClipBounds
- BasedOnUserBounds

```
animation["walk"].speed = 2.0F;
animation["run"].weight = 0.5F;
```

```
foreach (AnimationState state in animation) {
   state.speed = 0.5F;
}
```

애니메이션 시스템

- **애니메이션(Animation) 클래스**

Stop	재생되고 있는 모든 애니메이션을 중지(재생 위치를 처음으로 되돌림)
Rewind	특정 애니메이션(문자열 사용) 또는 모든 애니메이션을 시작 위치로 되돌림
Sample	현재 상태에서 애니메이션을 샘플링함
IsPlaying	특정 애니메이션(문자열 사용)이 재생되고 있는가
Play	블렌딩하지 않고 애니메이션을 재생함
CrossFade	특정 애니메이션을 일정 시간동안 페이드-인을 하고 다른 애니메이션을 페이드-아웃함
Blend	특정 애니메이션을 블렌딩함
CrossFadeQueued	이전 애니메이션의 재생이 끝난 후에 애니메이션을 크로스 페이드함
PlayQueued	이전 애니메이션의 재생이 끝난 후에 애니메이션을 재생함
AddClip	애니메이션 클립을 추가함
RemoveClip	애니메이션 클립을 제거함
GetClipCount	애니메이션 클립의 개수를 반환
SyncLayer	레이어의 모든 애니메이션의 재생 속도를 일치시킴

```
bool Play(PlayMode mode=PlayMode.StopSameLayer);
bool Play(string animation, PlayMode mode=PlayMode.StopSameLayer);
```

StopSameLayer
StopAll

```
void Stop();
void Stop(string name);
```

```
animation.Play("walk", PlayMode.StopAll);
animation.Stop();
```

애니메이션 시스템

- 애니메이션(Animation) 클래스

```
void CrossFade(string animation, float fadeLength=0.3F, PlayMode mode=PlayMode.StopSameLayer);
```

```
void Update() {
   animation.CrossFade("Walk", 0.2F);
}

void Update() {
   if (Mathf.Abs(Input.GetAxis("Vertical")) > 0.1F)
      animation.CrossFade("Run");
   else
      animation.CrossFade("Idle");
}
```

CompleteOthers

PlayNow

```
AnimationState CrossFadeQueued(string animation, float fadeLength=0.3F,
            QueueMode queue=QueueMode.CompleteOthers, PlayMode mode=PlayMode.StopSameLayer);
```

```
void Update() {
   if (Input.GetButtonDown("Fire1")) animation.CrossFadeQueued("shoot", 0.3F, QueueMode.PlayNow);
}
```

```
AnimationState PlayQueued(string animation, QueueMode queue=QueueMode.CompleteOthers,
                                            PlayMode mode=PlayMode.StopSameLayer);
```

```
void Update() {
   if (Input.GetButtonDown("Fire1")) animation.PlayQueued("shoot", QueueMode.PlayNow);
}
```

애니메이션 시스템

- 애니메이션(Animation) 클래스

```
bool IsPlaying(string animation);
```

```
void Rewind(string animation);
void Rewind();
```

```
animation.Rewind("shoot");
animation.Rewind();
```

```
void Blend(string animation, float targetWeight=1.0F, float fadeLength=0.3F);
```

```
void AddClip(AnimationClip clip, string newName);
void AddClip(AnimationClip clip, string newName, int firstFrame, int lastFrame, bool addLoopFrame=false);
void RemoveClip(AnimationClip clip);
void RemoveClip(string clipName);
```

```
animation.AddClip(animation.clip, "shoot", 0, 10);
animation.RemoveClip(animation.clip);
```

```
void SyncLayer(int layer);
```

```
void Sample();
```

```
void Start() {
   animation["walk"].layer = 1;
   animation["run"].layer = 1;
   animation.SyncLayer(1);
}
```

```
void Start() {
   animation["MyClip"].time = 2.0F;
   animation["MyClip"].enabled = true;
   animation.Sample();
   animation["MyClip"].enabled = false;
}
```

애니메이션 시스템

- **애니메이션 상태(AnimationState) 클래스**

enabled	애니메이션을 활성화
weight	애니메이션 커브의 블렌드 가중치
wrapMode	애니메이션 랩 모드, WrapMode
time	애니메이션의 현재 시간
normalizedTime	애니메이션의 정규화된 시간
speed	애니메이션의 재생 속도, 1.0: 정상 속도, 음수: 역방향 재생
normalizedSpeed	애니메이션의 정규화된 재생 속도
length	애니메이션 클립의 길이(초)
layer	애니메이션의 레이어 번호 최종적인 블렌드 가중치를 계산할 때 더 높은 레이어의 가중치가 우선함(높은 레이어의 애니메이션은 자신의 가중치를 그대로 사용하고 낮은 레이어의 애니메이션은 남은 가중치를 사용)
clip	애니메이션 상태가 재생하고 있는 애니메이션 클립, AnimationClip
name	애니메이션의 이름, string
blendMode	애니메이션 블렌드 모드, AnimationBlendMode

Default
Once
Loop
PingPong
ClampForever

Blend
Additive

AddMixingTransform	애니메이션 될 변환을 더함, 생성할 애니메이션의 개수를 줄일 수 있음
RemoveMixingTransform	애니메이션 될 변환을 제거함

```
void AddMixingTransform(Transform mix, bool recursive=true);
void AddMixingTransform(Transform mix);
```

```
Transform mixTransform = transform.Find("root/upper_body/left_shoulder");
animation["wave_hand"].AddMixingTransform(mixTransform);
```

애니메이션 시스템

- 애니메이션 상태(AnimationState) 클래스

```
bool AnimationState.enabled;
```

```
animation["Walk"].enabled = true;
```

```
float AnimationState.weight;
```

```
animation["Walk"].weight = 0.5F;
```

레이어	애니메이션 이름	블렌드 가중치	하체	상체
2	Wave	20%	0%	20%
1	Walk	50%	50%	40%
0	Idle	100%	50%	40%

```
float AnimationState.time;
float AnimationState.normalizedTime;
```

```
animation["Walk"].time = 0.0f; //재생 위치를 처음으로 돌림
animation["Walk"].normalizedTime = 0.5f; //재생 위치를 중간으로 설정
```

```
float AnimationState.speed;
float AnimationState.normalizedSpeed;
```

```
animation["Walk"].speed = -1.0f; //뒤로 재생
animation["Walk"].speed = 2.0f; //2배속
```

```
int AnimationState.layer;
```

```
animation["Walk"].layer = 1;
```

애니메이션 시스템

- **애니메이션 클립(AnimationClip) 클래스**

length	애니메이션의 길이(초), (읽기 전용), float
frameRate	키프레임이 샘플링되는 프레임 레이트(애니메이션 생성시의 프레임 레이트), (읽기 전용), float
wrapMode	애니메이션 상태에서 사용되는 기본 래핑 모드, WrapMode
localBounds	애니메이션 클립의 AABB(애니메이션 컴포넌트의 로컬 좌표계), Bounds

SetCurve	애니메이션 커브를 설정함
ClearCurves	모든 애니메이션 커브를 제거함
AddEvent	애니메이션 이벤트를 추가함(애니메이션 재생시 특정 시간에 함수를 호출할 수 있음)

```
void SetCurve(string relativePath, Type type, string propertyName, AnimationCurve curve);
void AddEvent(AnimationEvent event);
```

```
[RequireComponent(typeof(Animation))]
public class example : MonoBehaviour {
   void Start() {
      AnimationCurve curve = AnimationCurve.Linear(0, 1, 2, 3);
      AnimationClip clip = new AnimationClip();
      clip.SetCurve("", typeof(Transform), "localPosition.x", curve);
      animation.AddClip(clip, "test");
      animation.Play("test");
   }
}
```

stringParameter
floatParameter
intParameter
objectReferenceParameter
functionName
time
messageOptions
animationState

```
IEnumerator Example() {
   animation.Play();
   yield return new WaitForSeconds(animation.clip.length);
}
```

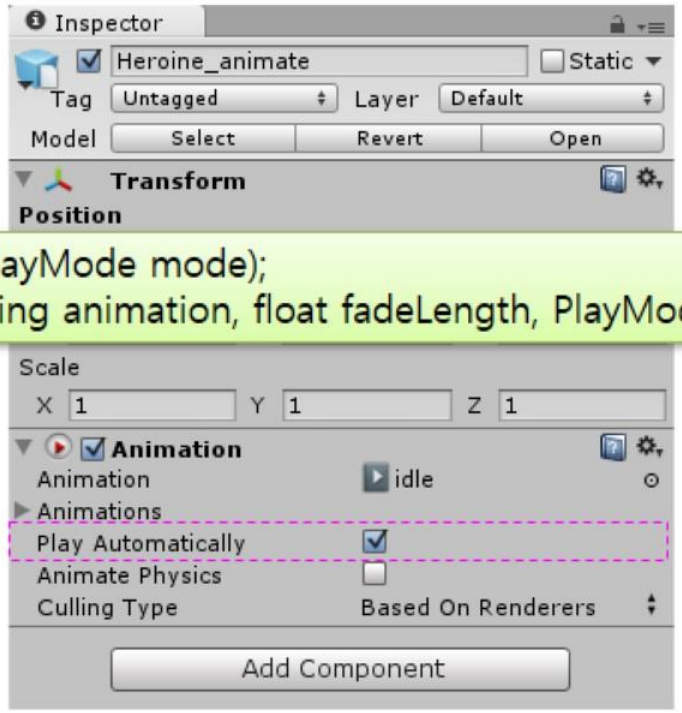
애니메이션 시스템

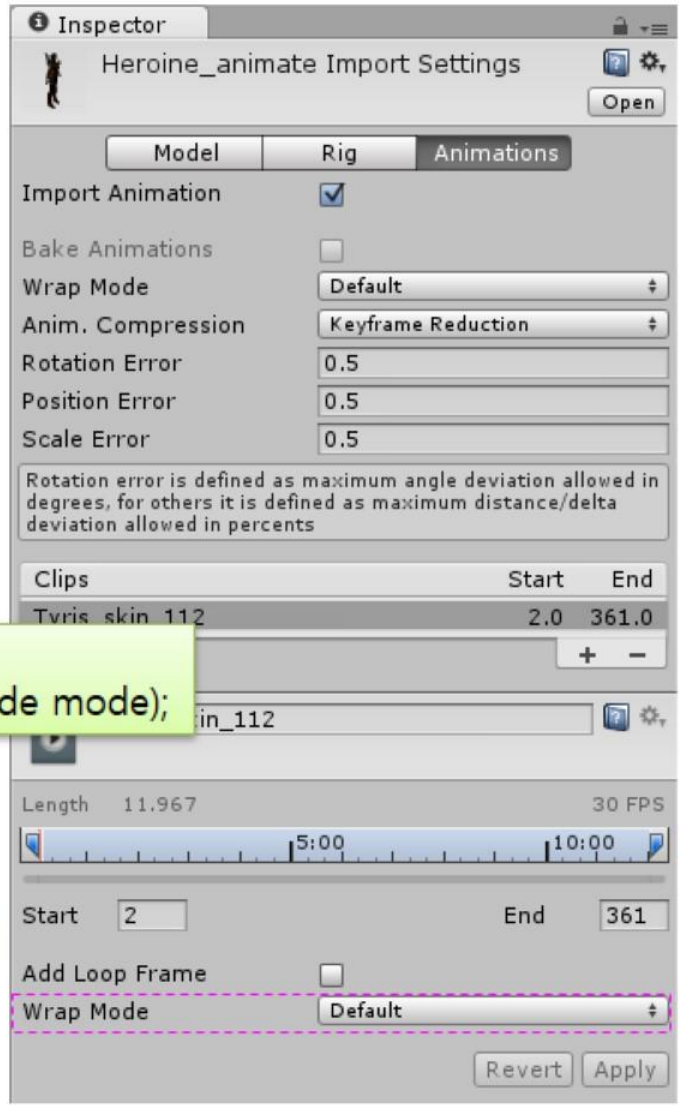
- **애니메이션 스크립팅(Animation Scripting)**
  - 애니메이션 블렌딩(Animation Blending)
    예) 걷기와 달리기 동작을 **부드럽게** 연결
    "Wrap Mode": Loop, "Play Automatically": Off

```
void Update() {
   if (Input.GetAxis("Vertical") > 0.2)
      animation.CrossFade("walk");
   else
      animation.CrossFade("idle");
}
```

```
animation.Play("walk");
animation.Play("idle");
```

```
bool Play(string animation, PlayMode mode);
void Animation.CrossFade(string animation, float fadeLength, PlayMode mode);
```





애니메이션 시스템

- **애니메이션 스크립팅(Animation Scripting)**
  - **애니메이션 레이어(Animation Layers)**
    애니메이션을 그룹화하고 블렌딩 가중치(Blending Weights)의 우선 순위를 설정
    블렌딩 가중치는 항상 정규화됨(1.0: 100%)
    예) shoot, idle, walk(속력에 따라 walk와 idle 동작은 블렌딩, shoot은 따로 실행)
      - 블렌딩 가중치를 합이 1.0이 되도록 수동적으로 설정
      - shoot을 walk와 idle 보다 높은 레이어를 부여
        블렌딩은 높은 레이어부터 적용(가중치의 합이 100%가 아니면 낮은 레이어)

    기본적으로 Play()와 CrossFade() 함수는 같은 레이어의 애니메이션을 중지시킴

```
void Start () {
   animation.wrapMode = WrapMode.Loop;
   animation["shoot"].wrapMode = WrapMode.Once;
   animation["shoot"].layer = 1;
   animation.Stop();
}

void Update () {
   if (Mathf.Abs(Input.GetAxis("Vertical")) > 0.1)
      animation.CrossFade("walk");
   else
      animation.CrossFade("idle");
   if (Input.GetButtonDown("Fire1")) animation.CrossFade("shoot");
}
```

int AnimationState.layer;

애니메이션 시스템

- **애니메이션 스크립팅(Animation Scripting)**
  - **애니메이션 믹싱(Animation Mixing)**
    캐릭터 몸체의 일부분에 애니메이션을 적용하여 애니메이션의 개수를 줄일 수 있음
    애니메이션에 애니메이션 믹싱 변환을 추가
    예) 손을 흔드는 애니메이션(idle 또는 walking)
    idle 상태에서 손 흔들기, walking 상태에서 손 흔들기 애니메이션 두 개 필요
    어깨에 애니메이션 믹싱 변환을 적용하면 손 흔들기 애니메이션은 한 개 필요

```
void AnimationState.AddMixingTransform(Transform mix, bool recursive=true);
```

```
void Start () {
   Transform mixTransform = transform.Find("root/upper_body/left_shoulder");
   animation["wave_hand"].AddMixingTransform(mixTransform);
}

void Start () {
   Transform shoulder:
   animation["wave_hand"].AddMixingTransform(shoulder);
}
```

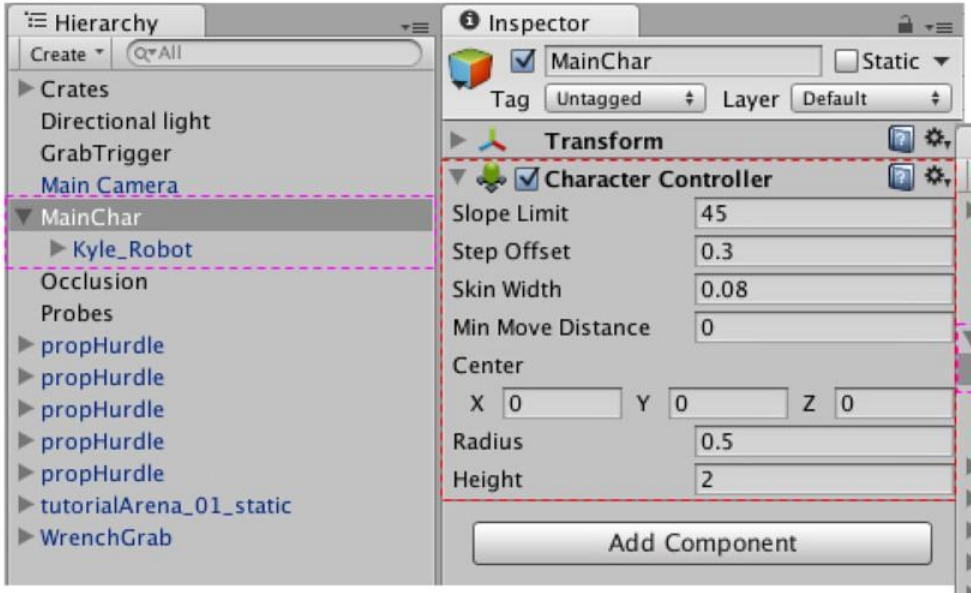
  - **중첩 애니메이션(Additive Animations)**
    하나의 애니메이션을 재생하고 있는 다른 애니메이션들 위에 중첩할 수 있음
    현재의 프레임과 애니메이션 클립의 첫 번째 프레임 사이의 차이를 계산함
    이 차이를 모든 다른 재생 애니메이션 위에 적용

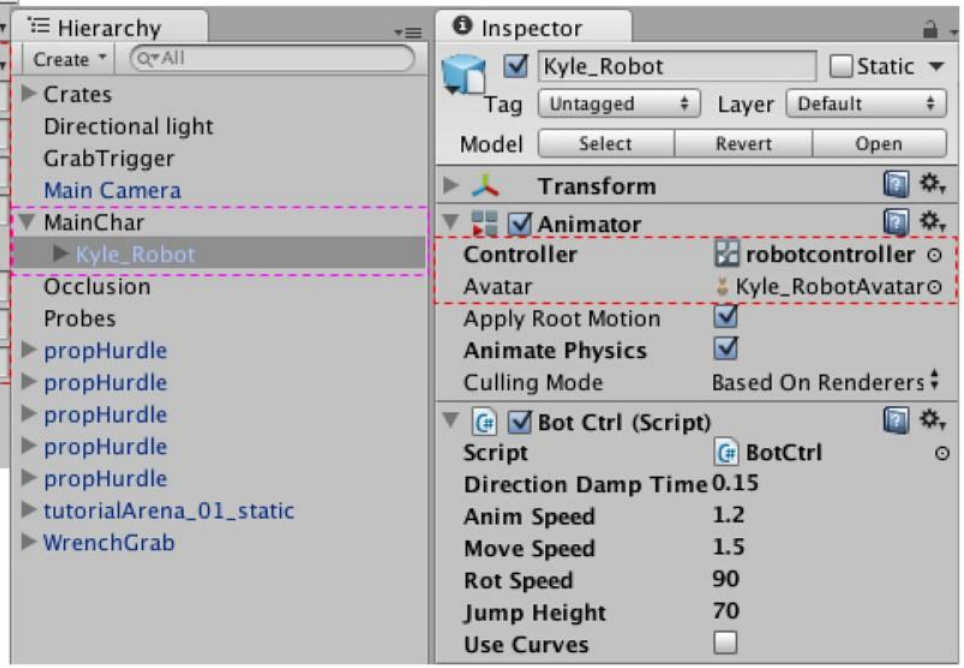
```
AnimationBlendMode.Additive
```

애니메이션 시스템

- **인간 캐릭터의 리타겟팅(Retargeting)**
  - 일반적인 "Humanoid" 애니메이션 객체
    - 캐릭터 모델(아바타를 가져야 함)
      주로 게임 객체의 자식 객체로 만듬
    - 애니메이터 컴포넌트(애니메이터 컨트롤러)
    - 애니메이션 클립
    - 스크립트 컴포넌트
    - 캐릭터 관련 컴포넌트(예: 캐릭터 컨트롤러)



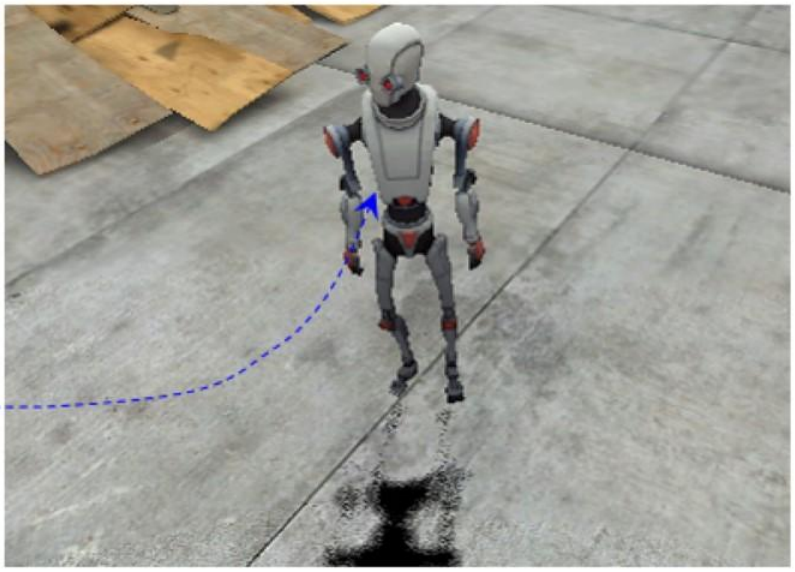


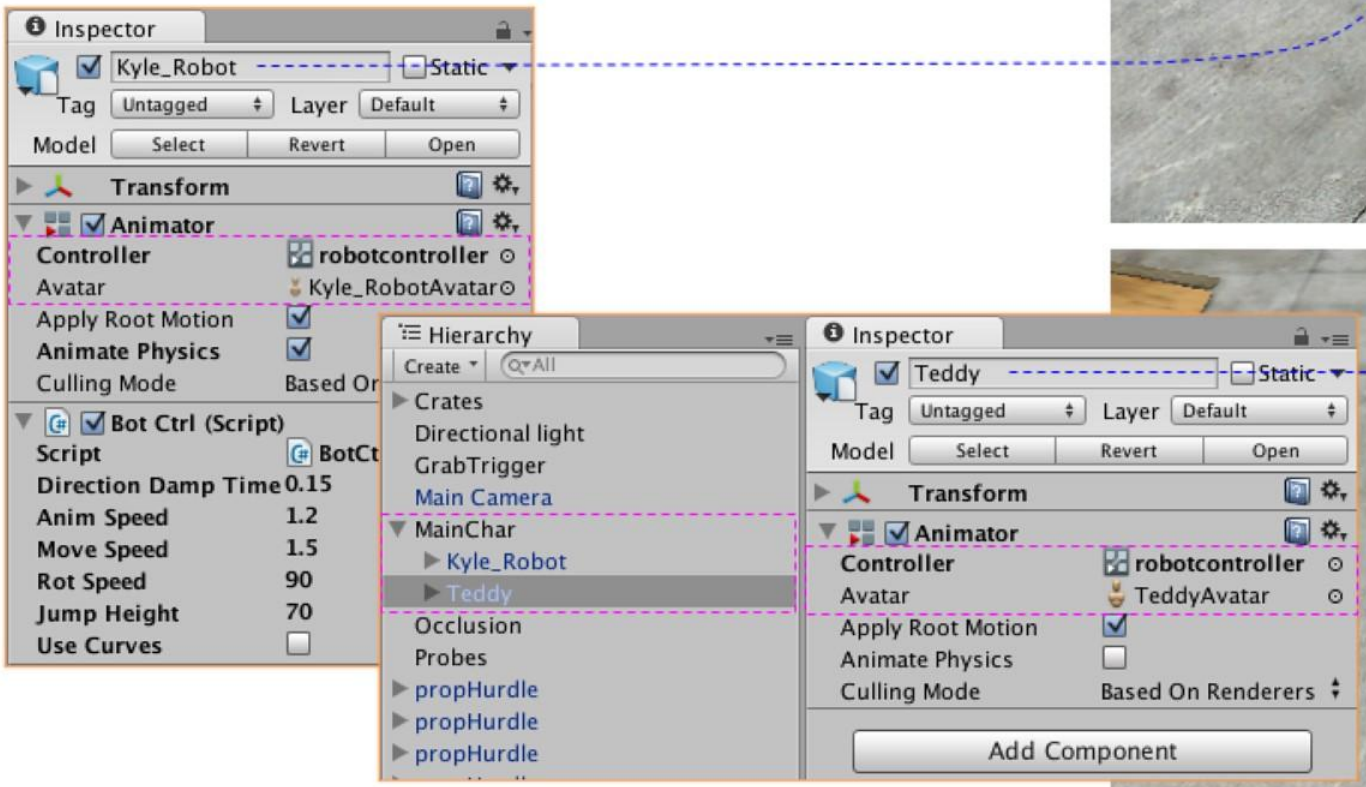


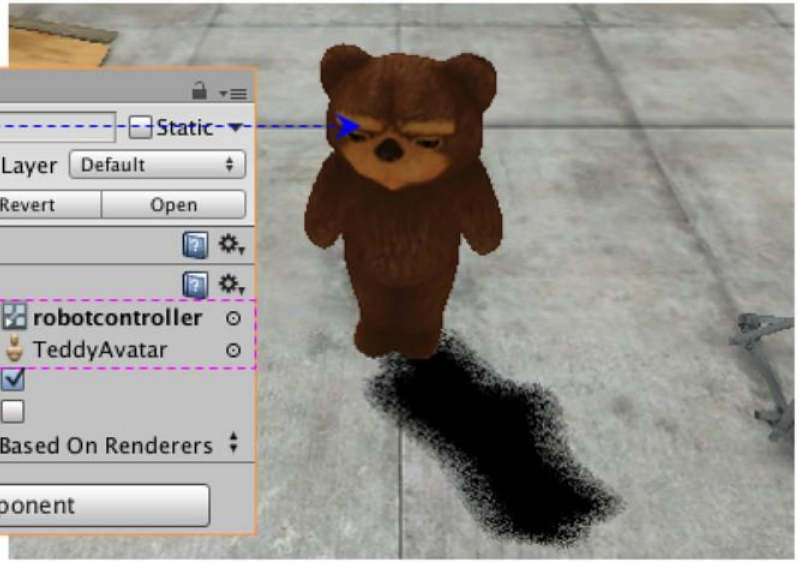
GetComponent<Animator>()
GetComponentInChildren<Animator>()

애니메이션 시스템

- **인간 캐릭터의 리타겟팅(Retargeting)**
  - 다른 모델의 애니메이션을 사용
    - 원래 캐릭터 모델을 비활성화
    - 새로운 모델을 자식 객체로 추가
      아바타 설정을 완료한 모델
    - 애니메이터 컴포넌트 설정
      같은 애니메이터 컨트롤러를 설정
    - 캐릭터 컨트롤러 등 파라메터 설정

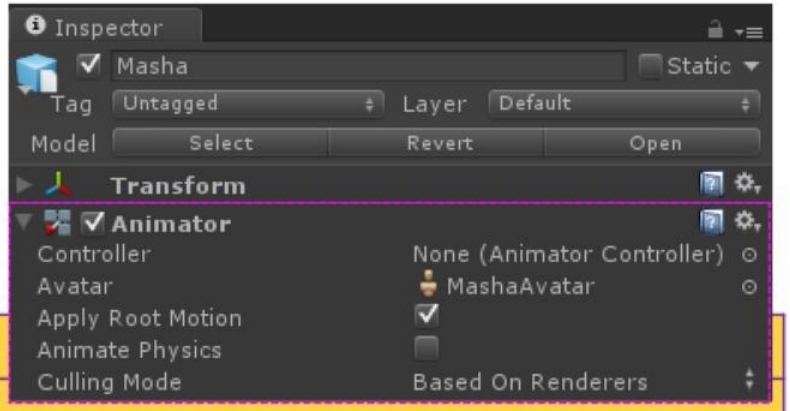






애니메이션 시스템

- **애니메이터 컴포넌트(Animator Component)**
  - 아바타를 가진 게임 객체는 자동적으로 애니메이터 컴포넌트를 가짐
  - 캐릭터와 애니메이터 컨트롤러를 연결
  - 애니메이터 컨트롤러는 캐릭터 동작을 설정
    - 상태 기계(State Machines)
    - 블렌드 트리(Blend Trees)
    - 이벤트(Events)



Controller	캐릭터에 연결된 애니메이터 컨트롤러
Avatar	캐릭터를 위한 아바타
Apply Root Motion	캐릭터의 위치를 애니메이션 자체 또는 스크립트에서 제어하는가
Animate Physics	애니메이션이 물리 엔진과 연동하는가
Culling Mode	애니메이션의 컬링 모드

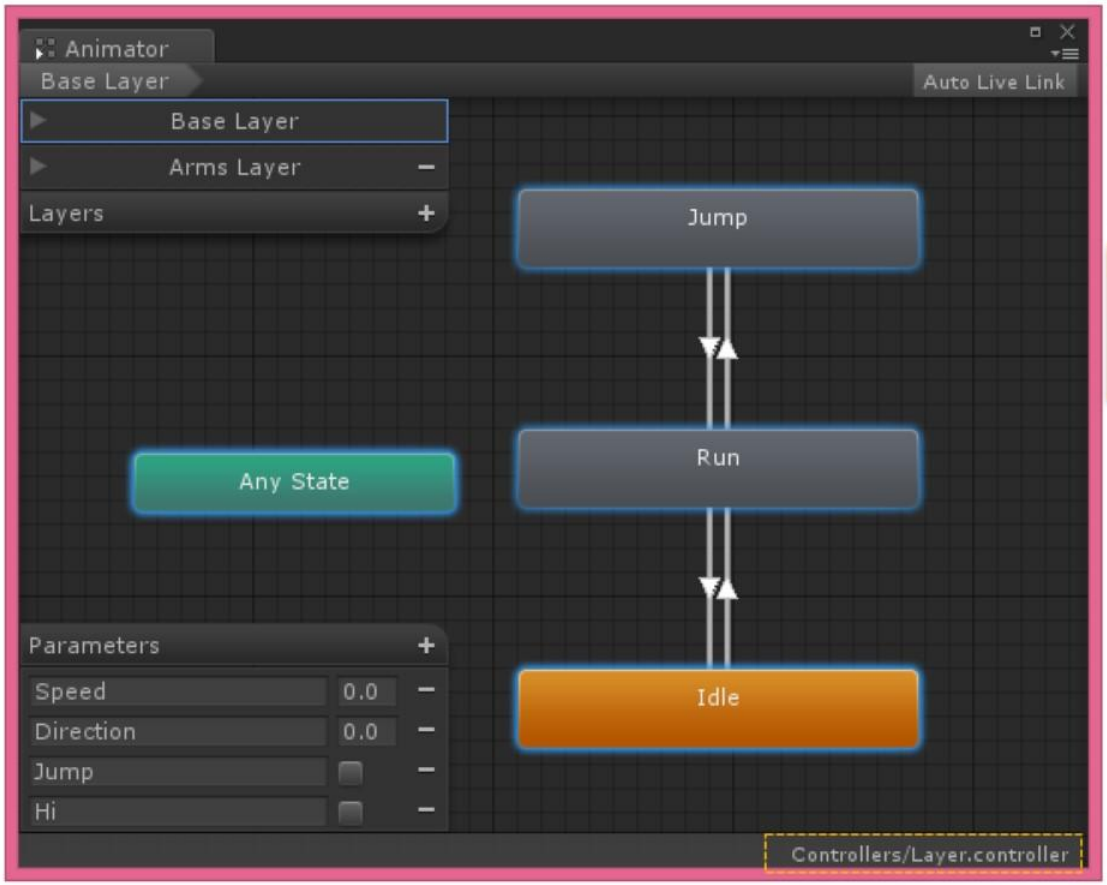
Always animate	컬링하지 않고 항상 애니메이션
Based on Renderers	카메라에 보이지 않을 때 루트 모션만 애니메이션

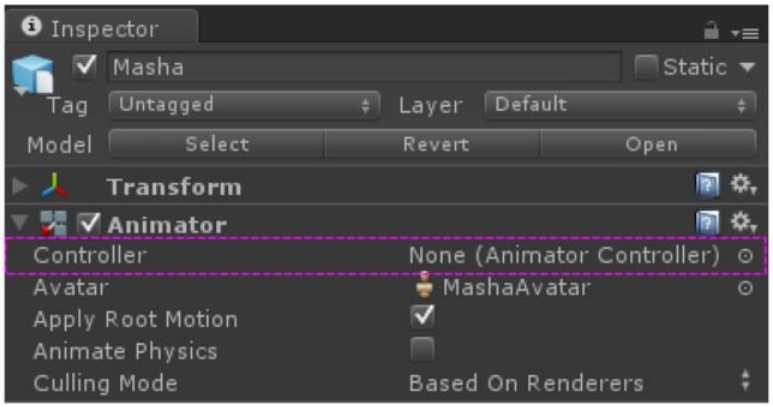
- **애니메이터 컨트롤러(Animator Controller)** 
  - Assets ▶ Create ▶ Animator Controller
  - Window ▶ Animator Controller
    씬과 상관없이 항상 가장 최근에 선택한 .controller 애셋의 상태 기계를 표시

애니메이션 시스템

- **애니메이션 상태 기계(Animation State Machines)**
  - 캐릭터 애니메이션의 제어와 시퀀싱을 쉽게하도록 지원
  - 각 동작(Action)은 상태(State)로 취급
  - 애니메이터 컨트롤러 윈도우

상태(States), 전이(Transitions), 이벤트(Events)



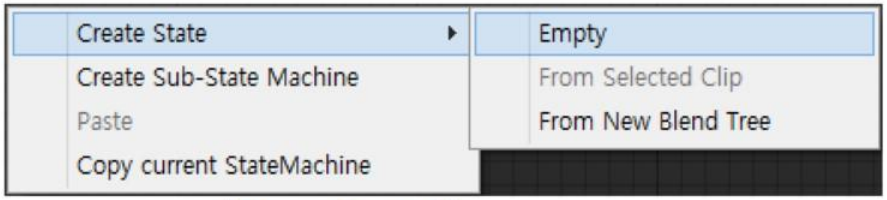


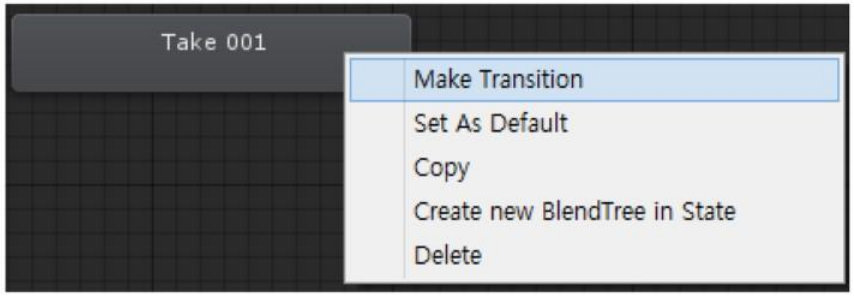
애니메이션 레이어(Animation Layer Widget)
파라메터(Event Parameters Widget)
상태 기계(State Machine)



애니메이션 시스템

- **애니메이션 상태(Animation States)**
  - 애니메이션 상태는 애니메이션의 시퀀스를 포함
    캐릭터가 그 상태에 있을 때 재생될 애니메이션 동작을 의미
    - 애니메이션 상태를 생성
      - 애니메이션 컨트롤러 윈도우의 **빈 영역을 오른쪽 클릭**
      - 애셋 폴더에서 **애니메이션 클립을 드래그**(메카님 애니메이션만 가능)
    - 기본 상태(Default State): 갈색(Brown), 처음으로 활성화되는 상태





    - "Any State"
      다른 모든 상태로 전이 가능
      상태 전이의 끝이 될 수 없음
  - 애니메이터 컨트롤러에서 "상태"를 선택하면 인스펙터 뷰에 상태의 속성이 표시됨

Speed	애니메이션의 기본 속도
Motion	이 상태에 설정된 애니메이션 클립
Foot IK	이 상태를 위하여 "Foot IK"를 고려해야 하는가
Transitions	이 상태에서 시작하는 상태 전이의 리스트

Mute	상태 전이를 비활성화, 빨간색 화살표
Solo	상태 전이를 활성화, 녹색 화살표

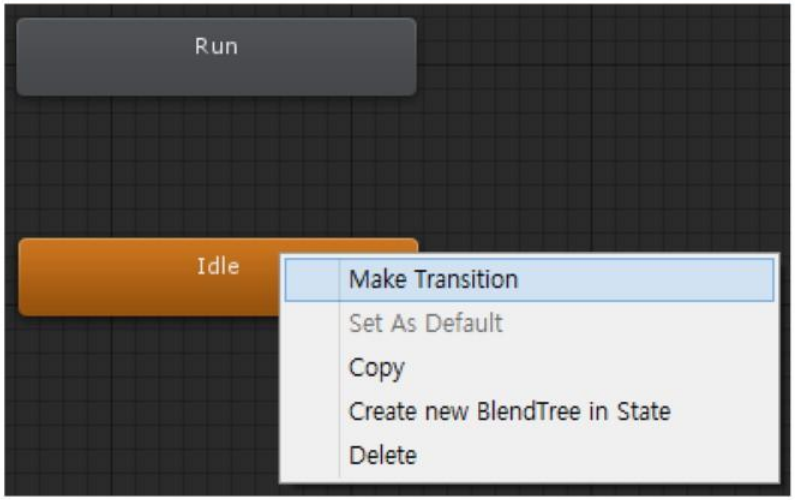


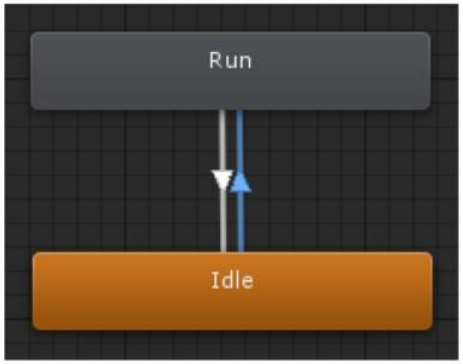
애니메이션 시스템

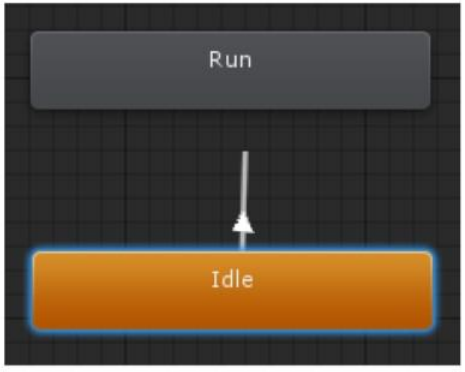
- **애니메이션 상태 전이(Animation Transitions)**
  - 하나의 상태가 다른 상태로 전환될 때의 이벤트를 정의

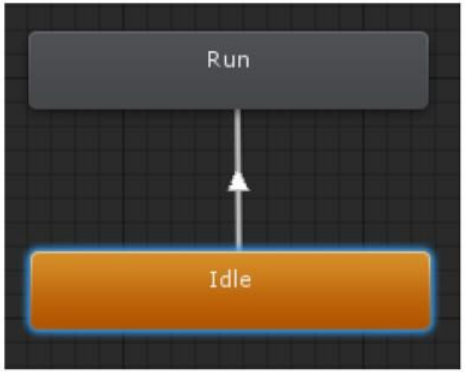
한 순간에 하나의 상태 전이만 일어날 수 있음

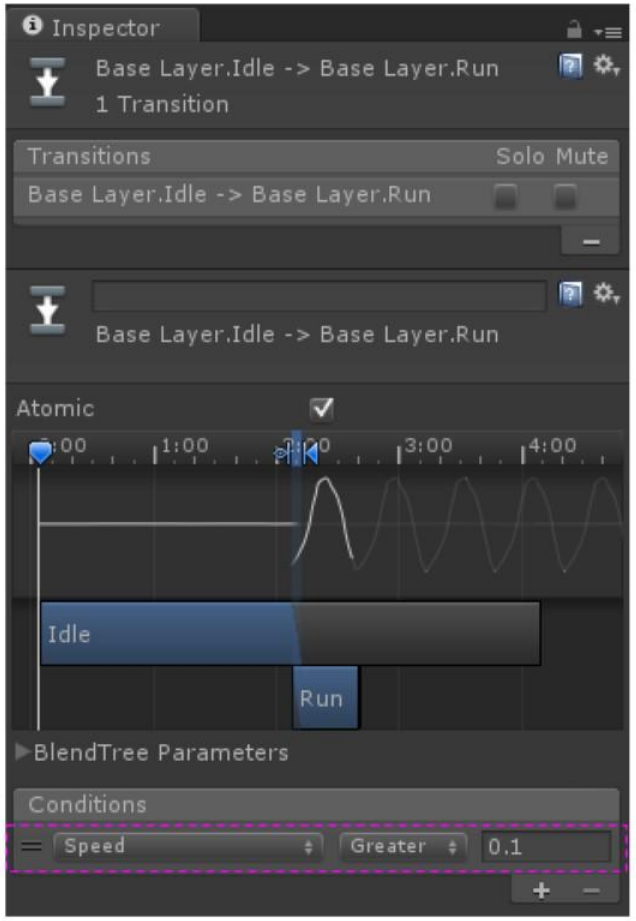
Atomic	상태 전이가 인터럽트 될 수 없음
Conditions	상태 전이가 일어나기 위한 조건
BlendTree Parameters	블렌드 트리 파라메터







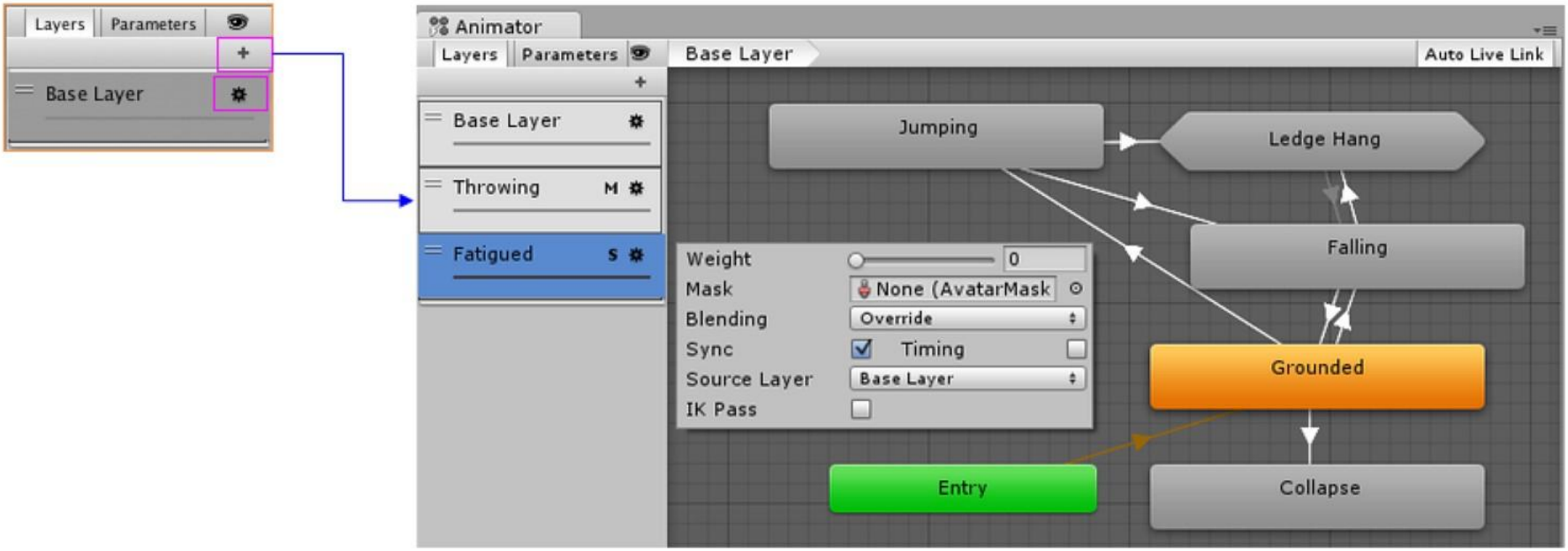




float: "Less/Greater", int: "Less/Greater/Equals/NotEqual"
"Exit Time": 상태 전이 시점(소스 상태의 클립 재생 시간(정규화))

애니메이션 시스템

- **애니메이션 레이어(Animation Layers)**
  - 애니메이션 레이어는 다른 신체 부위를 위한 복잡한 상태를 관리하기 위해 사용
    예) 하체 레이어: 걷기-점프, 상체: 던지기-총쏘기
  - 기본 레이어("Base Layer")
  - 레이어 추가하기

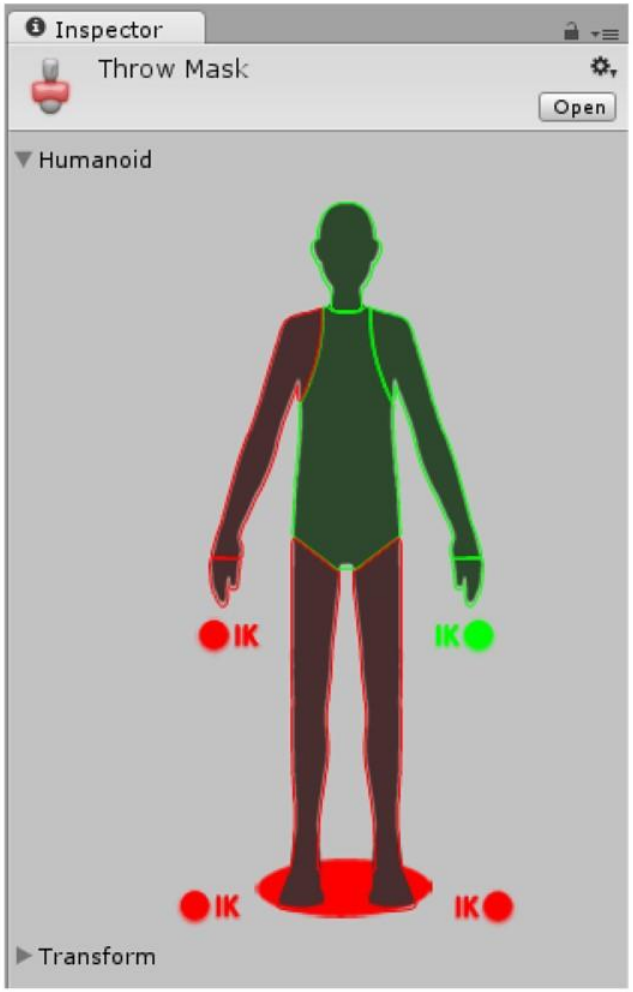


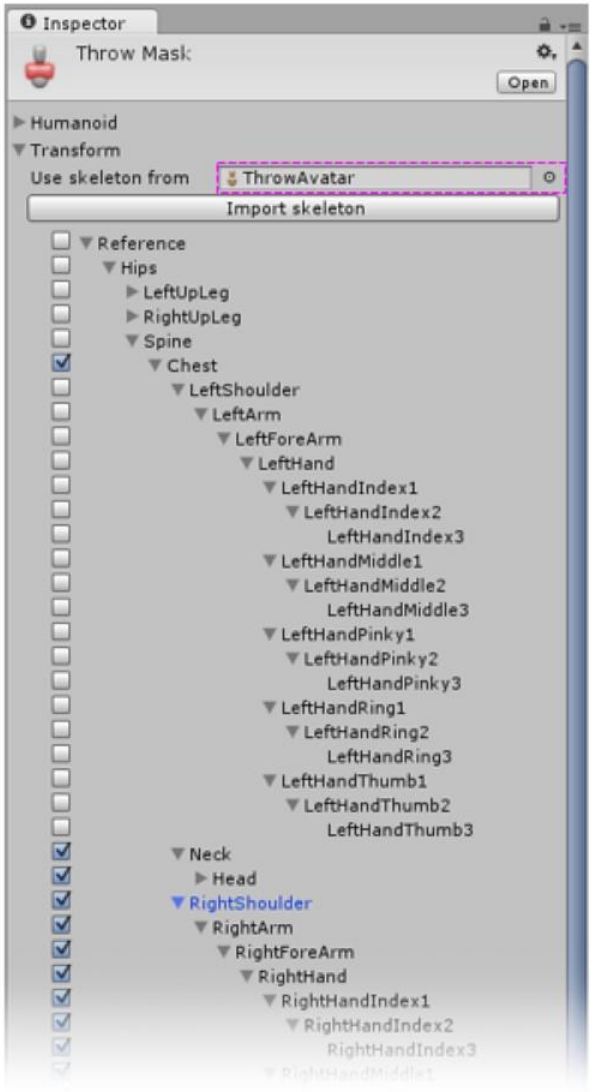
Name	레이어의 이름
Mask	아바타 매스크(애니메이션이 적용될 신체 부위)
Blending	블렌딩 유형
Sync	다른 레이어의 상태 기계를 재사용
IK Pass	IK를 적용하지 않음

Override	다른 레이어를 무시(적용하지 않음)
Additive	애니메이션이 이전 레이어의 위에 추가됨

애니메이션 시스템

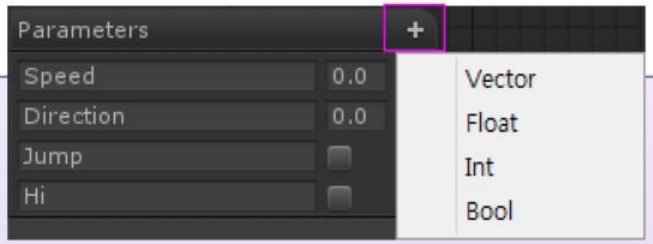
- **애니메이션 레이어(Animation Layers)**
  - 아바타 매스크(Avatar Mask)
    Assets ▶ Create ▶ Avatar Mask





애니메이션 시스템

- **애니메이션 파라메터(Animation Parameters)**
  - 애니메이션 파라메터는 애니메이션 시스템에서 정의된 변수(스크립트에서 사용)
    파라메터의 값은 애니메이션 커브에 의해 변경될 수 있음
    스크립트에서 값을 변경하면 메카님 시스템에서 사용할 수 있음
  - 애니메이터 컨트롤러에서 설정



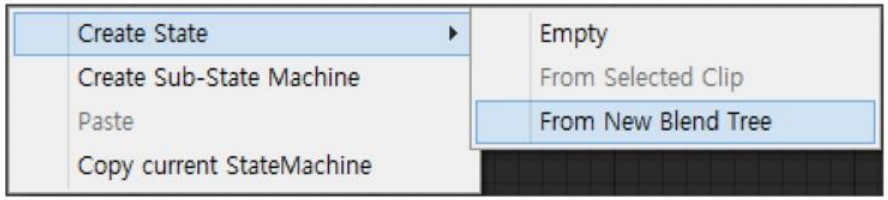

```
public class AvatarCtrl : MonoBehaviour {
   protected Animator animator;
   public float DirectionDampTime = .25f;
   void Start () {
      animator = GetComponent<Animator>();
   }
   void Update () {
      AnimatorStateInfo stateInfo = animator.GetCurrentAnimatorStateInfo(0);
      if (stateInfo.nameHash == Animator.StringToHash("Base Layer.RunBT")) {
         if (Input.GetButton("Fire1")) animator.SetBool("Jump", true );
      }
      else {
         animator.SetBool("Jump", false);
      }
      float h = Input.GetAxis("Horizontal");
      float v = Input.GetAxis("Vertical");
      animator.SetFloat("Speed", h*h+v*v);
      animator.SetFloat("Direction", h, DirectionDampTime, Time.deltaTime);
   }
}
```

파라메터 데이터 형
- 벡터(Vector)
- 정수(Int)
- 실수(Float)
- 논리형(Bool)

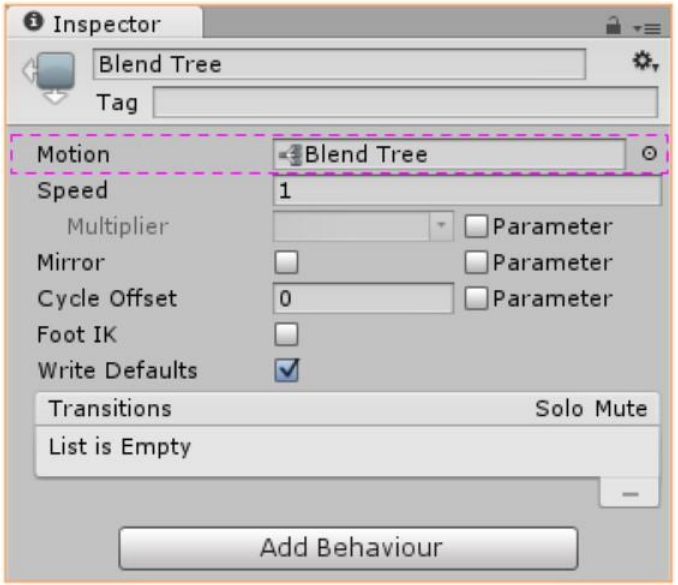
```
Animator.SetVector(string name, Vector3 value);
Animator.SetFloat(string name, float value);
Animator.SetInt(string name, int value);
Animator.SetBool(string name, bool value);
```

애니메이션 시스템

- **애니메이션 블렌드 트리(Blend Trees)**
  - 애니메이션 블렌딩
    예) 캐릭터의 속도에 따라 걷기와 달리기를 블렌딩
        달리는 동안 방향 전환할 때 캐릭터를 좌우로 기울임
    여러가지 애니메이션을 부드럽게 연결하여 부드러운 애니메이션을 생성
      상태 전이: 하나의 상태(동작)에서 다른 동작으로 전이할 때 두 동작을 블렌딩
      블렌드 트리: 여러 애니메이션을 블렌딩하여 부드러운 애니메이션을 생성
    블렌드 트리는 애니메이션 상태 기계에서 특별한 유형의 상태임
      상태의 애니메이션 동작이 여러 애니메이션들의 블렌딩의 결과
      블렌딩되는 움직임들은 유사한 특성과 타이밍을 가져야 함
      애니메이션 클립에서의 움직임은 정규화된 시간에서 같은 지점에서 일어나야 함
  - 블렌드 트리의 생성

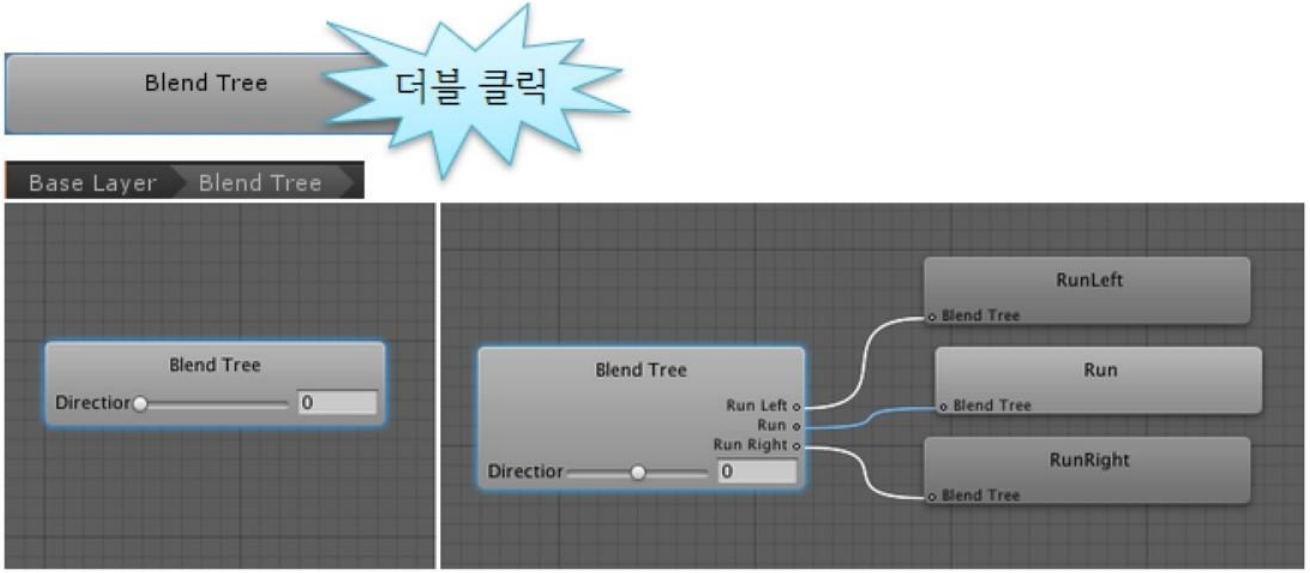






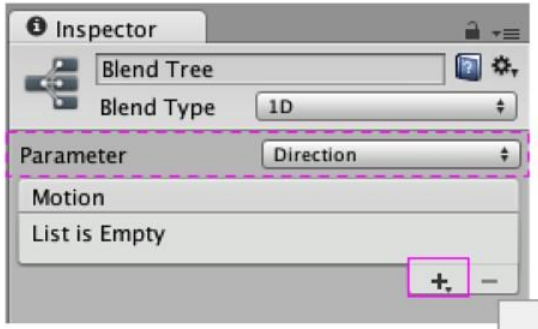
애니메이션 시스템

- **애니메이션 블렌드 트리(Blend Trees)**
  - 블렌드 트리 그래프(Blend Tree Graph)



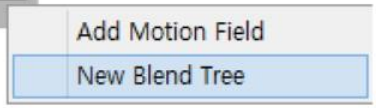


블렌드할 애니메이션 클립을 추가
블렌드 파라메터를 추가



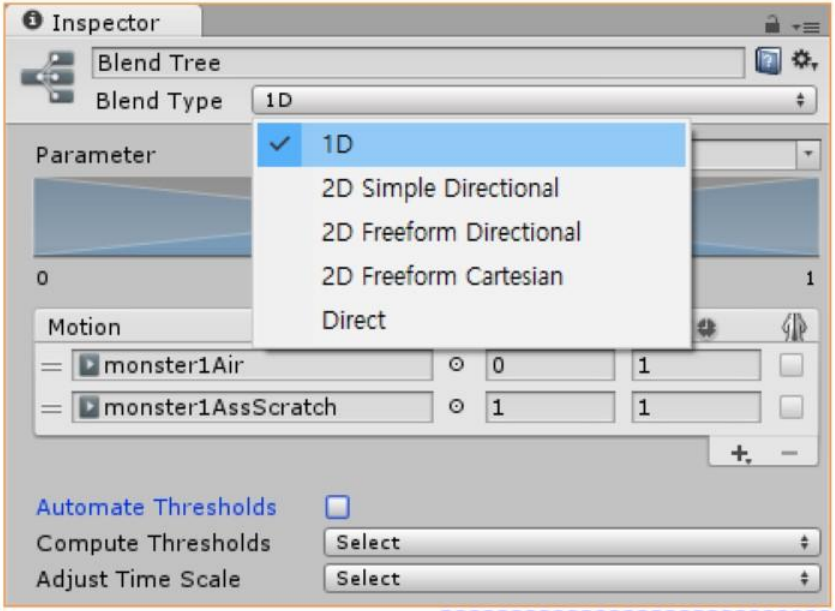
float형 파라메터

애니메이션 클립 추가



애니메이션 시스템

- **애니메이션 블렌드 트리(Blend Trees)**
  - 1D 블렌딩(1D Blending)
    하나의 파라메터로 여러 애니메이션 동작들을 블렌딩 함
    각 동작들은 파라메터 값의 위치에 삼각형으로 표시



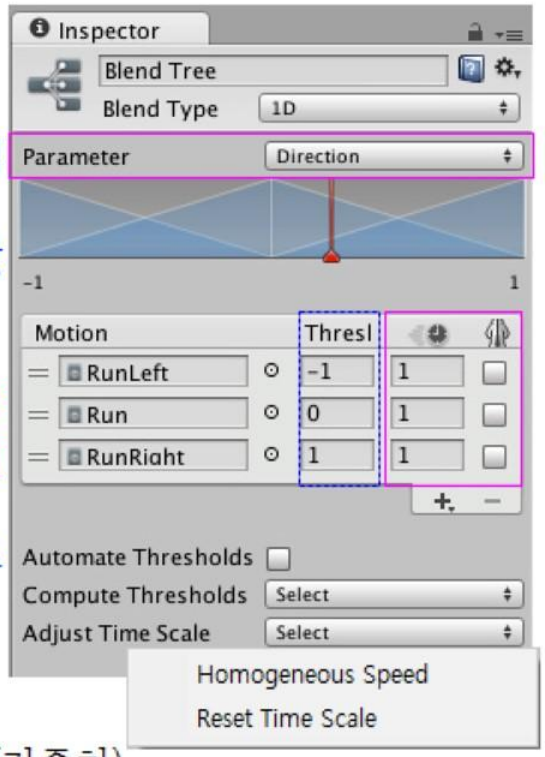
애니메이션 파라메터 선택

파라메터 임계값

Time Scale: 속도 변경

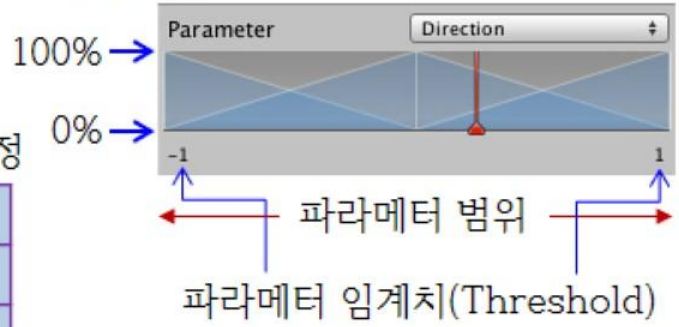
Mirroring: 대칭 애니메이션

임계값을 균일하게 분포



각 애니메이션 동작은 삼각형 모양
첫 번째와 마지막 동작은 직각 삼각형 형태
피라미드를 선택하여 이동하면 모양이 변함

영향(가중치)



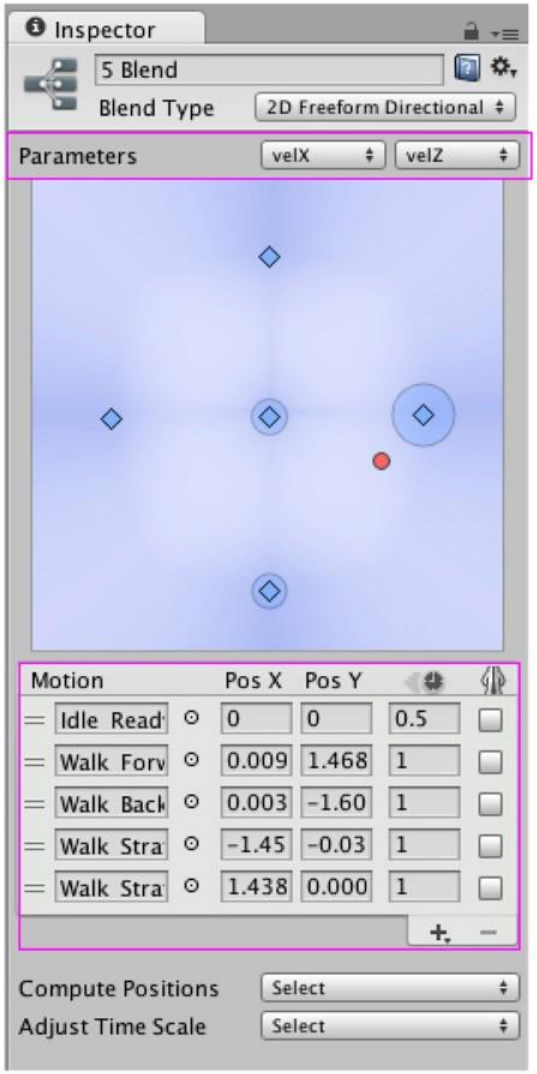
Compute Thresholds: 루트 모션에서 얻은 데이터를 사용하여 임계값을 설정

Speed	속력에 따라 각 동작의 임계치를 설정
Velocity X/Y/Z	속도(x, y, z)에 따라 각 동작의 임계치를 설정
Angular Speed (Rad/Deg)	각속도에 따라 각 동작의 임계치를 설정

애니메이션 시스템

- **애니메이션 블렌드 트리(Blend Trees)**
  - 2D 블렌딩(2D Blending)
    두 개의 파라메터에 따라 애니메이션 동작들을 블렌딩 함

2D Simple Directional	동작의 방향이 다를 때 (앞으로, 뒤로, 왼쪽으로, 오른쪽으로 걷기) 같은 방향으로 여러 동작들을 갖지 않아야 함 (앞으로 걷기, 앞으로 뛰기) 위치 (0, 0)에 하나의 동작을 가질 수 있음
2D Freeform Directional	동작의 방향이 다를 때 같은 방향으로 여러 동작들을 가질 수 있음 (앞으로 걷기, 앞으로 뛰기) 위치 (0, 0)에 하나의 동작을 가져야 함
2D Freeform Cartesian	동작들이 다른 방향을 표현하지 않을 때 X 파라메터와 Y 파라메터가 다른 개념을 표현 (X: 각속도, Y: 선형 속력)
Direct	각 노드의 가중치를 직접 조절

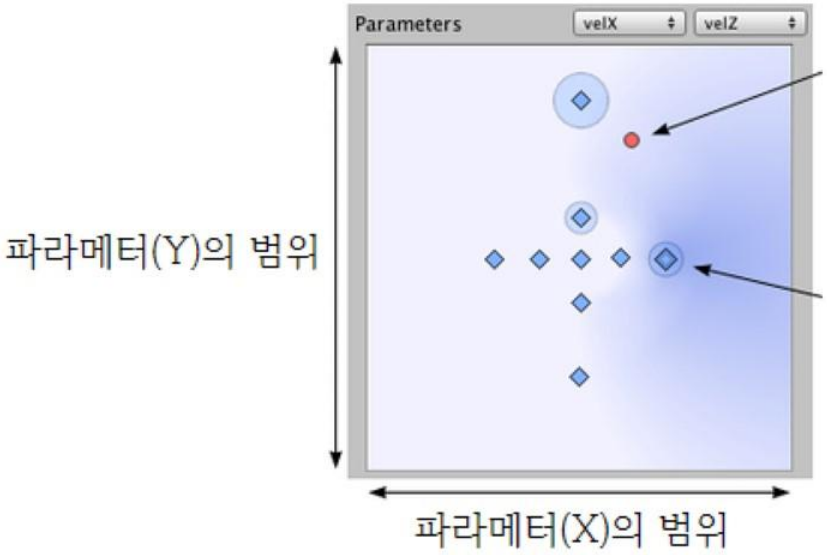


X-축: 첫 번째 파라메터
Y-축: 두 번째 파라메터

두개의 파라메터 순서쌍 (x, y)는 사각형의 한 점을 표현함
파라메터 (x, y)에 대한 동작은 인접한 동작들을 블렌딩하여 생성됨

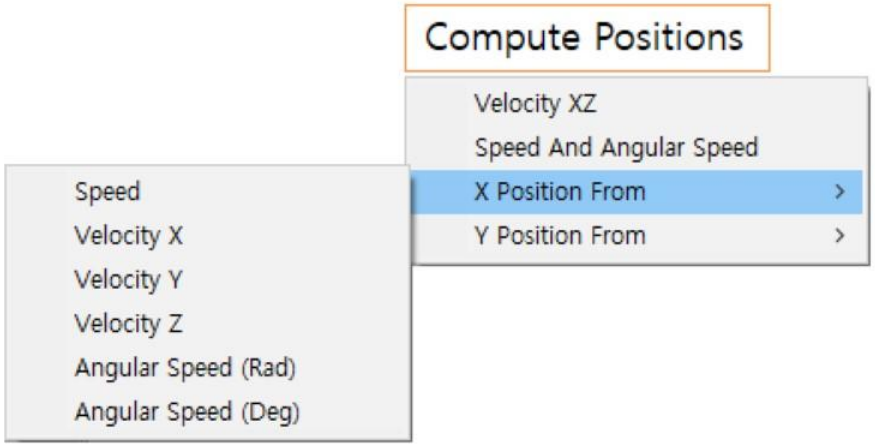
애니메이션 시스템

- **애니메이션 블렌드 트리(Blend Trees)**
  - 2D 블렌딩 다이어그램(2D Blending Diagram)
    블렌딩의 대상이 되는 동작들이 파란색 마름모로 표시됨
    마름모를 선택하여 이동하거나 (Pos X, Pos Y)의 값을 수정하여 위치를 바꿀 수 있음



현재 파라메터 (x, y)의 위치
값의 위치에서 각 동작의 영향을 받는 정도가 원으로 표시

현재 선택된 동작
(주변에 이 동작의 영향(가중치)을 파란색으로 표시함)

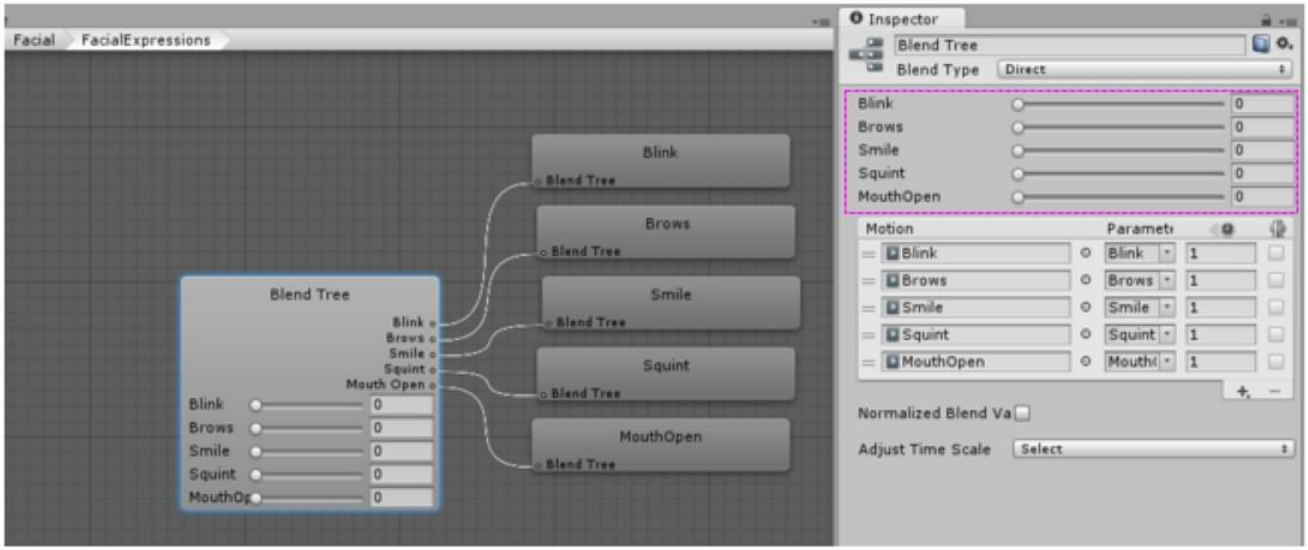


Velocity XZ: (Pos X, Pos Y) = (Velocity.x, Velocity.z)
Speed And Angular Speed: (Angular Speed, Speed)

Compute Positions: 애니메이션 클립의 루트 모션의 데이터를 사용하여 위치 (Pos X, Pos Y)를 자동 계산
파라메터가 루트 모션의 값에 해당할 경우 (Pos X, Pos Y)를 자동 계산함

애니메이션 시스템

- **애니메이션 블렌드 트리(Blend Trees)**
  - 직접 블렌딩(Direct Blending)
    블렌딩의 대상이 되는 동작의 가중치를 파라메터의 값으로 직접 설정
    하나 또는 두개의 파라메터가 아닌 여러 개의 파라메터를 사용하여 블렌딩할 수 있음
    Maya "Blend Shape" 애니메이션들을 블렌딩할 때 유용
    스크립트에서 블렌드 가중치를 설정할 수 있음







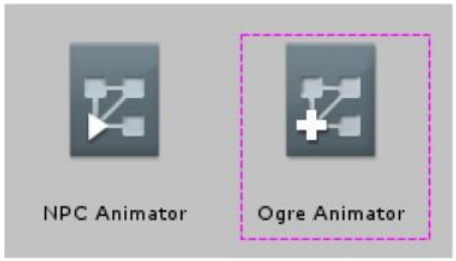
애니메이션 시스템

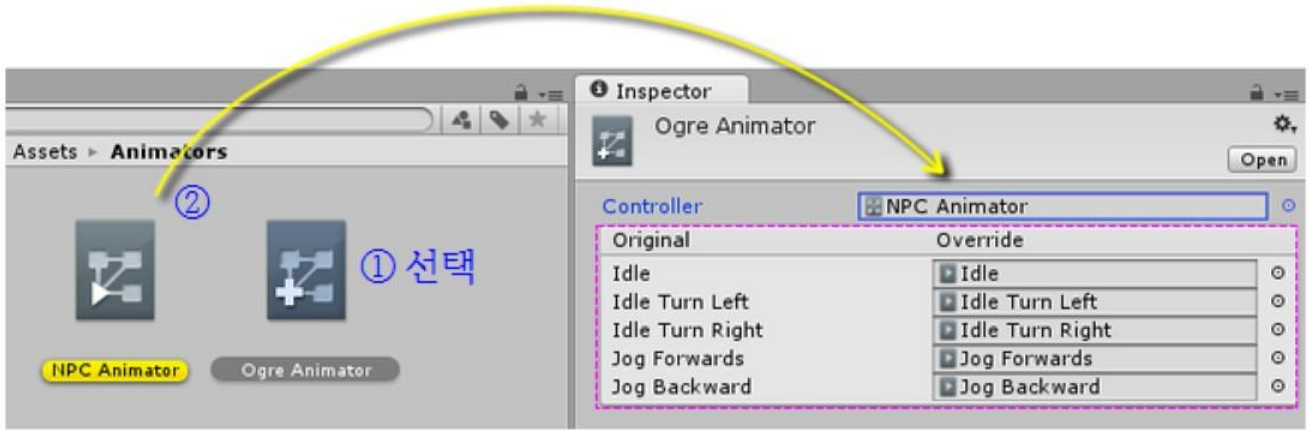
- 애니메이션 블렌드 스크립트

```
public class BlendShapeExample : MonoBehaviour {
   int blendShapeCount;
   SkinnedMeshRenderer skinnedMeshRenderer;
   Mesh skinnedMesh;
   float blendOne = 0f, blendTwo = 0f, blendSpeed = 1f;
   bool blendOneFinished = false;
   void Awake() {
      skinnedMeshRenderer = GetComponent<SkinnedMeshRenderer>();
      skinnedMesh = GetComponent<SkinnedMeshRenderer>().sharedMesh;
   }
   void Start() { blendShapeCount = skinnedMesh.blendShapeCount; }
   void Update() {
      if (blendShapeCount > 2) {
         if (blendOne < 100f) {
            skinnedMeshRenderer.SetBlendShapeWeight(0, blendOne);
            blendOne += blendSpeed;
         }
         else
            blendOneFinished = true;
         if ((blendOneFinished == true) && (blendTwo < 100f)) {
            skinnedMeshRenderer.SetBlendShapeWeight(1, blendTwo);
            blendTwo += blendSpeed;
         }
      }
   }
}
```

애니메이션 시스템

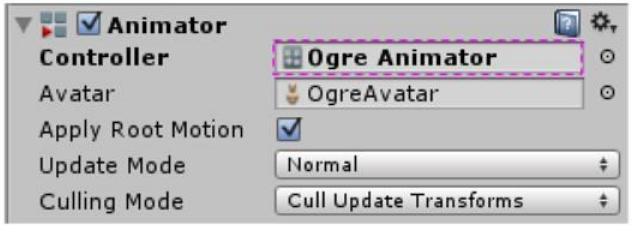
- **애니메이터 오버라이드 컨트롤러(Animator Override Controller)**
  - 애니메이터 오버라이드 컨트롤러
    하나의 애니메이터 컨트롤러를 다른 애니메이션 클립들에 적용할 수 있음
    상태 기계(State Machine), 파라메터(Parameter), 로직(Logic)을 그대로 사용
    (하나의 기본 NPC 애니메이터 컨트롤러를 여러 다른 NPC 캐릭터들에 적용)
    애니메이터 오버라이드 컨트롤러를 생성하여 애니메이션 클립들에 적용
    Assets ▶ Create ▶ Animator Override Controller

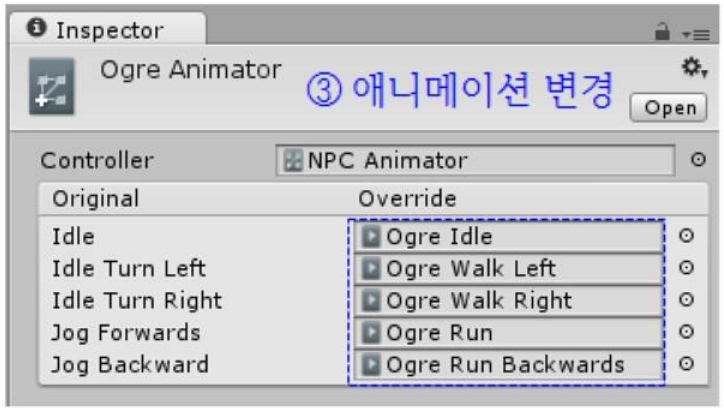




④ 게임 객체의 애니메이터 컨트롤러에 애니메이터 연결

게임 객체





애니메이션 시스템

- 애니메이터(Animator) 스크립팅

void MonoBehaviour.**OnAnimatorMove**(); //루트 모션을 수정하기 위한 애니메이션 움직임을 처리
상태 기계와 애니메이션이 계산된 후 각 프레임에서 호출됨(OnAnimatorIK() 호출 전에)

```
void OnAnimatorMove() {
    Animator animator = GetComponent<Animator>();
    if (animator) {
        Vector3 newPosition = transform.position;
        newPosition.z += animator.GetFloat("Runspeed") * Time.deltaTime;
        transform.position = newPosition;
    }
}
```

void MonoBehaviour.**OnAnimatorIK**(int layerIndex); //애니메이션 IK를 설정

```
public class Example: MonoBehaviour {
    protected Animator animator;
    void Start() {
        animator = GetComponent<Animator>();
    }
    void OnAnimatorIK(int layerIndex) {
        animator.SetIKPositionWeight(AvatarIKGoal.LeftFoot,leftFootWeightPosition);
        animator.SetIKRotationWeight(AvatarIKGoal.LeftFoot,leftFootWeightRotation);
        animator.SetIKPosition(AvatarIKGoal.LeftFoot,leftFootObj.position);
        animator.SetIKRotation(AvatarIKGoal.LeftFoot,leftFootObj.rotation);
    }
}
```

애니메이션 시스템

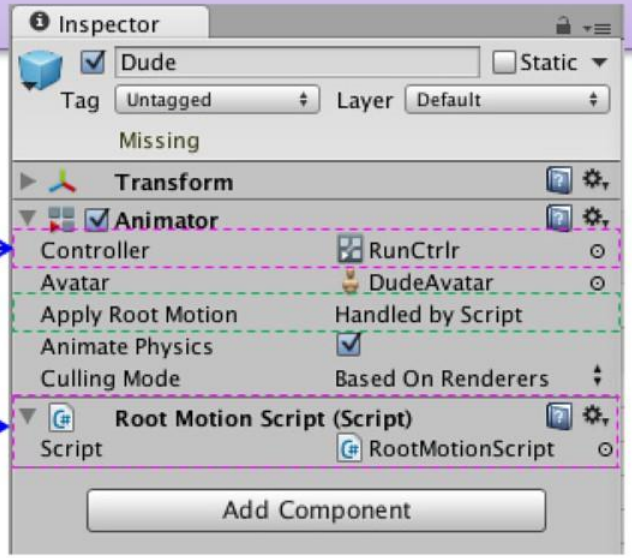
- **애니메이터(Animator) 스크립팅**
  - 제자리 애니메이션(In-Place Animation)
    애니메이션이 "Root Motion"을 포함하지 않음
  - 아바타에 "Muscle Definition"을 설정
    애니메이션 클립에 "Loop Pose"를 설정(녹색)
    캐릭터 속도를 제어하기 위한 커브를 생성
    애니메이터 컨트롤러를 생성
    애니메이션 상태와 파라메터를 추가




```
void OnAnimatorMove() {
   Animator animator = GetComponent<Animator>();
   Vector3 newPosition = transform.position;
   newPosition.z += animator.GetFloat("Runspeed") * Time.deltaTime;
   transform.position = newPosition;
}
```

애니메이터 컨트롤러 연결 →

스크립트 연결 →



애니메이션 시스템

- **루트 모션(Root Motion)**
  - 몸통 변환(Body Transform)과 몸통 방향(Body Orientation)
    - 몸통 변환은 캐릭터의 질량 중심(Mass Center)
    - 몸통 방향은 아바타의 T-포즈에 상대적인 상체와 하체 방향(Orientation)의 평균

    몸통 변환과 몸통 방향은 월드 좌표계로 애니메이션 클립에 저장됨
  - 루트 변환(Root Transform)

    몸통 변환의 XZ-평면에 대한 투영

    "미리보기(Preview)" 윈도우의 원이 루트 변환을 나타냄

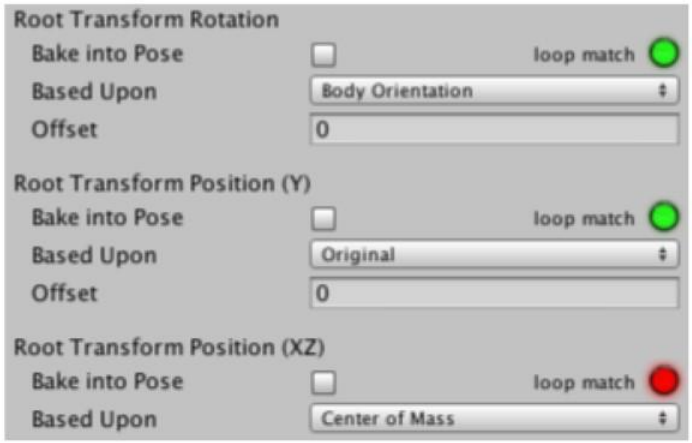
    빨강색 화살표가 루트 변환의 앞방향

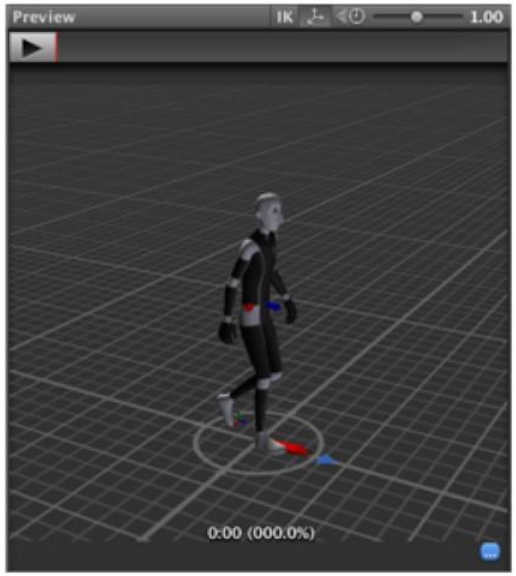
    파란색 화살표는 게임 객체의 z-축

    실행시에 루트 변환의 변화를 계산하여 게임 객체에 적용할 수 있음
  - 애니메이션 클립 설정

    몸통 변환으로부터 루트 변환을 조절할 수 있음

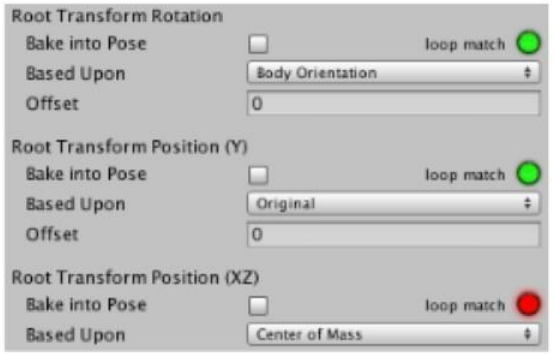
    몸통 변환의 일부분을 루트 변환으로 설정할 수 있음





애니메이션 시스템

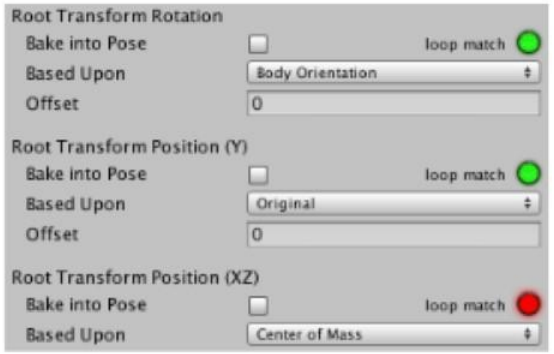
- **루트 모션(Root Motion)**
  - **Root Transform Rotation**
    - Bake into Pose
      게임 객체의 방향은 몸통 변환(포즈)을 따르고 애니메이션 클립을 전혀 따르지 않음
      루트의 시작과 끝의 방향이 유사한 애니메이션 클립에만 사용할 것(녹색으로 표시)
    - Based Upon
      애니메이션 클립의 방향을 설정
      - Body Orientation
        애니메이션 클립은 몸통의 앞쪽 방향을 따르도록 회전
      - Original
        애니메이션 클립에서 찾은 오프셋을 자동적으로 더함
        애니메이션 클립의 방향을 따름



  - **Root Transform Position (Y)**
    - Bake into Pose
      게임 객체의 Y 위치는 몸통 변환을 따르고 루트 변환의 Y-값은 상수가 될 것임
    - Based Upon
      애니메이션 클립의 방향을 설정
      - Mass Center (Body)
        애니메이션 클립은 몸통의 Y 위치를 따름
      - Original
        애니메이션 클립의 Y 위치를 따름
      - Feet
        높이가 변하는 애니메이션 클립에 적합(Bake into Pose: 비활성화)

애니메이션 시스템

- **루트 모션(Root Motion)**
  - **Root Transform Position (XZ)**
    - Bake into Pose
      위치(XZ)의 변화를 0으로 설정하고 싶은 "Idle" 동작에 적용
      게임 객체의 XZ 위치는 몸통 변환을 따르고 루트 변환의 XZ-값은 상수가 될 것임
    - Based Upon
      애니메이션 클립의 방향을 설정
      - Mass Center (Body)
        애니메이션 클립은 몸통의 XZ 위치를 따름
      - Original
        애니메이션 클립의 XZ 위치를 따름



애니메이션 시스템

- **애니메이터(Animator) 클래스**

isHuman	현재 리깅이 "Humanoid"이면 참, "Generic"이면 거짓, bool
humanScale	"Humanoid" 리깅 아바타의 스케일("Generic"이면 기본적으로 1), float
deltaPosition	마지막 프레임동안 아바타의 위치 이동량, Vector3
deltaRotation	마지막 프레임동안 아바타의 회전 이동량, Quaternion
rootPosition	루트 위치, 게임 객체의 위치, Vector3
rootRotation	루트 회전, 게임 객체의 회전, Quaternion
applyRootMotion	루트가 애니메이션으로 컨트롤됨, bool
animatePhysics	애니메이션이 물리 루프에서 실행, bool
gravityWeight	현재 재생되는 애니메이션에 기반한 중력 가중치, float
bodyPosition	몸체의 무게 중심의 위치, Vector3
bodyRotation	몸체의 무게 중심의 회전(게임 객체에 상대적), Quaternion
stabilizeFeet	이동과 블렌딩하는 동안 자동적으로 발을 안정화시킴, bool
layerCount	애니메이터 컨트롤러 레이어 개수, int
feetPivotActive	몸체의 무게중심과 발의 피봇 사이에서 피봇 지점을 블렌딩(0: 무게중심), float
pivotWeight(Position)	피봇의 가중치(현재 위치), float(Vector3)
isMatchingTarget	자동 매칭이 활성화되었는가, bool
speed	애니메이터의 재생 속도(1: 정상 속도), float
targetPosition	목표 위치(SetTarget 함수로 설정), Vector3
targetRotation	목표 회전(SetTarget 함수로 설정), Quaternion
cullingMode	애니메이터 컴포넌트의 컬링 방법, AnimatorCullingMode
playbackTime	레코딩 버퍼에서 재생 위치를 설정, float
recorderStart(Stop)Time	버퍼의 첫 번째 프레임의 시작(끝) 시간
runtimeAnimatorController	애니메이터를 제어하는 애니메이터 컨트롤러, RuntimeAnimatorController
avatar	현재 아바타

애니메이션 시스템

- 애니메이터(Animator) 클래스

Get(Set)Float/Bool/Integer	실수/논리/정수 파라메터의 값을 반환(설정)
Get(Set)Vector	벡터 파라메터의 값을 반환(설정)
Get(Set)Quaternion	쿼터니언 파라메터의 값을 반환(설정)
IsParameterControlledByCurve	파라메터가 애니메이션의 추가적인 커브로 제어되는가, bool
Get(Set)IKPosition(Rotation)	IK 목표 위치(회전)를 반환(설정)
Get(Set)IKPositionWeight	IK 목표 위치 이동의 가중치를 반환(설정), 0: 원래 애니메이션, 1: 목표 위치
Get(Set)IKRotationWeight	IK 목표의 회전 가중치를 반환(설정), 0: 원래 애니메이션, 1: IK 목표의 회전
SetLookAtPosition(Weight)	바라보는 지점(가중치)을 설정
GetLayerName	레이어의 이름을 반환
Get(Set)LayerWeight	레이어의 현재 가중치를 반환(설정)
GetCurrentAnimatorStateInfo	애니메이터 컨트롤러 레이어의 현재 상태 정보를 반환, AnimatorStateInfo
GetNextAnimatorStateInfo	애니메이터 컨트롤러 레이어의 다음 상태 정보를 반환, AnimatorStateInfo
GetAnimatorTransitionInfo	애니메이터 컨트롤러 레이어의 상태 전이 정보를 반환, AnimatorTransitionInfo
GetCurrentAnimationClipState	현재 상태에서 재생중인 애니메이션 정보의 리스트를 반환, AnimationInfo[]
GetNextAnimationClipState	다음 상태에서 재생할 애니메이션 정보의 리스트를 반환, AnimationInfo[]
IsInTransition	애니메이터 컨트롤러 레이어가 상태 전이중인가, bool
MatchTarget	게임 객체의 위치와 회전을 자동적으로 조정
InterruptMatchTarget	자동적인 타겟 일치를 인터럽트
SetTarget	현재 상태를 위한 아바타 타겟과 시간을 설정
IsControlled	애니메이터가 변환을 제어하고 있는가
GetBoneTransform	뼈대 ID로 매핑되는 변환을 반환
Start(Stop)Playback	애니메이터를 재생 모드로 설정(중지)
StringToHash	문자열에서 파라메터 ID를 생성

애니메이션 시스템

- 애니메이터(Animator) 클래스

```
void SetVector(string name, Vector3 value);
void SetVector(int id, Vector3 value);
```

```
Vector3 GetVector(string name);
Vector3 GetVector(int id);
```

```
void SetFloat(string name, float value);
void SetFloat(string name, float value, float dampTime, float deltaTime);
void SetFloat(int id, float value);
void SetFloat(int id, float value, float dampTime, float deltaTime);
```

dampTime	설정 값에 도달할 시간

```
void SetInt(string name, int value);
void SetInt(int id, int value);
```

```
bool IsName(string name);
bool IsTag(string tag);
```

```
void SetBool(string name, bool value);  void SetBool(int id, bool value);
void SetTrigger(string name); void SetTrigger(int id);
```

```
void SetQuaternion(string name, Quaternion value);
void SetQuaternion(int id, Quaternion value);
```

```
Vector3 GetIKPosition(AvatarIKGoal goal);
Quaternion GetIKRotation(AvatarIKGoal goal);
```

```
void SetIKPosition(AvatarIKGoal goal, Vector3 goalPosition);
void SetIKRotation(AvatarIKGoal goal, Quaternion goalRotation);
void SetIKPositionWeight(AvatarIKGoal goal, float value);
void SetIKRotationWeight(AvatarIKGoal goal, float value);
```

```
void CrossFade(string state, float Duration, int layer=-1, float normalizedTime=float.NegativeInfinity);
void CrossFade(int stateHash, float Duration, int layer=-1, float normalizedTime=float.NegativeInfinity);
```

```
void Play(string state, int layer=-1, float normalizedTime=float.NegativeInfinity);
void Play(int stateHash, int layer=-1, float normalizedTime=float.NegativeInfinity);
```

애니메이션 시스템

- 애니메이터(Animator) 클래스

```
AnimatorStateInfo GetCurrentAnimatorStateInfo(int layerIndex);
AnimatorStateInfo GetNextAnimatorStateInfo(int layerIndex);
AnimatorTransitionInfo GetAnimatorTransitionInfo(int layerIndex);
AnimationInfo[] GetCurrentAnimationClipState(int layerIndex);
AnimationInfo[] GetNextAnimationClipState(int layerIndex);
```

애니메이터 상태 구조체(AnimatorStateInfo)

nameHash	상태의 이름, int
normalizedTime	상태의 정규화 시간, float
length	상태의 현재 길이, float
tagHash	상태의 태그, int
loop	상태가 반복되는가, bool
IsName	이름이 상태 기계의 활성화 상태의 이름과 일치하는가, bool
IsTag	태그가 상태 기계의 활성화 상태의 태그와 일치하는가, bool

애니메이터 상태 전이 구조체(AnimatorTransitionInfo)

nameHash	상태 전이의 이름, int
userNameHash	상태 전이의 사용자 정의 이름, int
normalizedTime	상태 전이의 정규화 시간, float
IsName	이름이 상태 기계의 활성화 상태 전이의 이름과 일치하는가, bool
IsUserName	이름이 상태 기계의 활성화 상태 전이의 이름과 일치하는가, bool

애니메이션 정보 구조체(AnimationInfo)

clip	재생하고 있는 애니메이션 클립, AnimationClip
weight	애니메이션 클립의 가중치, float

애니메이션 시스템

- **애니메이터(Animator) 클래스**
  - **타겟 매칭(Target Matching)**
    캐릭터가 어떤 시각에 특정 지점에 손 또는 발이 위치하도록 하는 방법

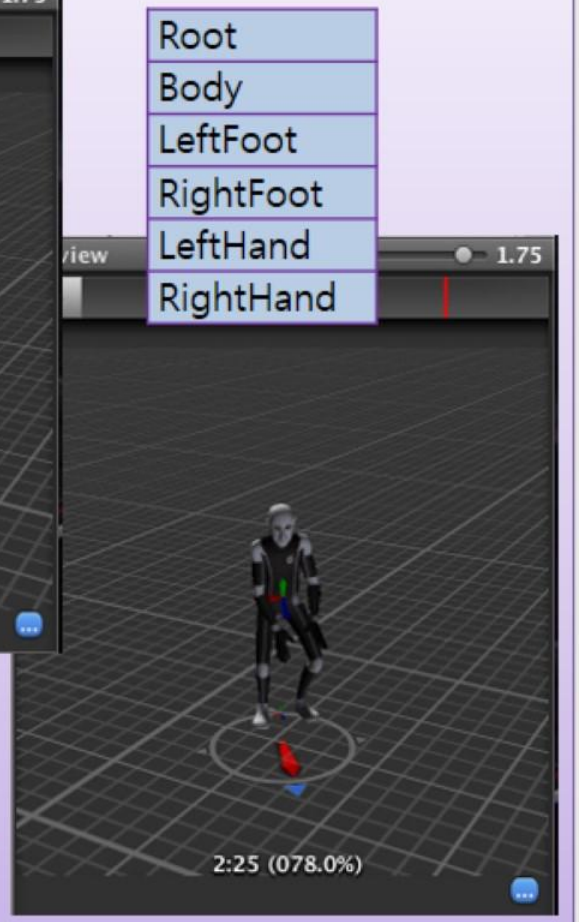
```
void Animator.MatchTarget(Vector3 matchPosition, Quaternion matchRotation, AvatarTarget targetBodyPart,
        MatchTargetWeightMask weightMask, float startNormalizedTime, float targetNormalizedTime = 1);
```

```
[RequireComponent(typeof(Animator))]
public class TargetCtrl : MonoBehaviour {
   protected Animator animator;
   public Transform jumpTarget = null;

   void Start () {
      animator = GetComponent<Animator>();
   }
   void Update () {
      if(animator) {
         if(Input.GetButton("Fire1")) {
            animator.MatchTarget(
                  jumpTarget.position,
                  jumpTarget.rotation,
                  AvatarTarget.LeftFoot,
                  new MatchTargetWeightMask(Vector3.one, 1f),
                  0.141f, 0.78f);
         }
      }
   }
}
```

weightMask	위치와 방향을 일치하기 위한 가중치
startNormalizedTime	애니메이션 클립의 시작 시간(0: 시작, 1.0: 끝)
targetNormalizedTime	애니메이션 클립의 끝 시간(0: 시작, 1.0: 끝)





애니메이션 시스템

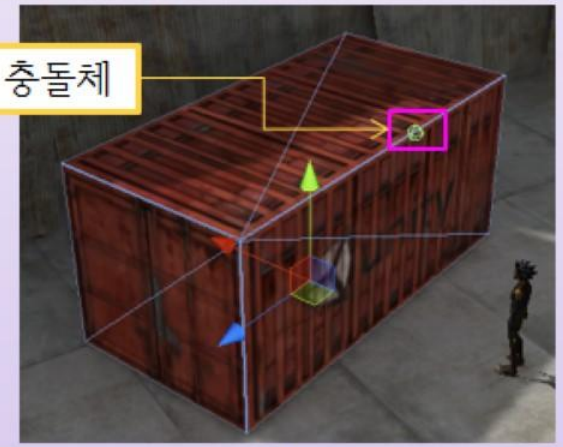
- 애니메이터(Animator) 클래스

```
public class TargetMatching : MonoBehaviour {
   private Animator animator;
   public Transform RightHand;
   bool hasJumped = false;

   void Start () {
      animator = GetComponent<Animator>();
   }
   void Update () {
      AnimatorStateInfo state = animator.GetCurrentAnimatorStateInfo(0);
      if (Input.GetButton("Fire1")) animator.SetBool("Jump", true);
      if (state.IsName("Base Layer.JumpUp")) {
         animator.SetBool("Jump", false);
         animator.MatchTarget(RightHand.position, RightHand.rotation,
            AvatarTarget.RightHand,
            new MatchTargetWeightMask(new Vector3(1, 1, 1), 0),
            animator.GetFloat("MatchStart"),
            animator.GetFloat("MatchEnd"));
         hasJumped = true;
      }
      if (hasJumped && state.normalizedTime > 1.2) {
         hasJumped = false;
         Application.LoadLevel(0);
      }
   }
}
```

MatchTargetWeightMask(Vector3 positionXYZWeight, float rotationWeight);









애니메이션 시스템

- **애니메이터(Animator) 클래스**
  - 애니메이션: 미리 정해진 계층적 관절을 정해진 값 만큼 회전(Forward Kinematics)
  - **IK(Inverse Kinematics)**
    물건을 집으려면(?), 울퉁 불퉁한 길에 서려면(?), 시선(LookAt)을 바꾸려면(?)
    IK는 스크립트로 구현(Animator 클래스)




```
[RequireComponent(typeof(Animator))]
public class IKCtrl : MonoBehaviour {
   protected Animator animator;
   public bool ikActive = false;
   public Transform rightHandObj = null;
   void Start () { animator = GetComponent<Animator>(); }
   void OnAnimatorIK(int layerIndex) {
      if (ikActive) {
         animator.SetIKPositionWeight(AvatarIKGoal.RightHand, 1.0f);
         animator.SetIKRotationWeight(AvatarIKGoal.RightHand, 1.0f);
         if (rightHandObj != null) {
            animator.SetIKPosition(AvatarIKGoal.RightHand, rightHandObj.position);
            animator.SetIKRotation(AvatarIKGoal.RightHand, rightHandObj.rotation);
         }
      }
      else {
         animator.SetIKPositionWeight(AvatarIKGoal.RightHand, 0.0f);
         animator.SetIKRotationWeight(AvatarIKGoal.RightHand, 0.0f);
      }
   }
}
```

1.0: 최종 목표는 IK 위치

```
void SetIKPosition(AvatarIKGoal goal, Vector3 goalPosition);
void SetIKRotation(AvatarIKGoal goal, Quaternion goalRotation);
```

AvatarIKGoal

LeftFoot
RightFoot
LeftHand
RightHand

```
void SetIKPositionWeight(AvatarIKGoal goal, float weight); //weight: 0(애니메이션)~1(목표)
void SetIKRotationWeight(AvatarIKGoal goal, float weight);
```

애니메이션 시스템

- 애니메이터(Animator) 클래스
  - IK(Inverse Kinematics)

```
protected Animator animator;
public bool ikActive = false;
public Transform rightFootObj = null;
public Transform lookAtObj = null;
public float rightFootPositionWeight = 1.0f;
public float rightFootRotationWeight = 1.0f;
public float lookAtWeight = 1.0f;
```

```
[RequireComponent(typeof(Animator))]
public class IKControl : MonoBehaviour {
    ...
    void Start () { animator = GetComponent<Animator>(); }

    void OnAnimatorIK(int layerIndex) {
        if (ikActive) {
            animator.SetIKPositionWeight(AvatarIKGoal.RightFoot, rightFootPositionWeight);
            animator.SetIKRotationWeight(AvatarIKGoal.RightFoot, rightFootRotationWeight);
            animator.SetIKPosition(AvatarIKGoal.RightFoot,rightFootObj.position);
            animator.SetIKRotation(AvatarIKGoal.RightFoot,rightFootObj.rotation);
            animator.SetLookAtWeight(lookAtWeight, 0.3f, 0.6f, 1.0f, 0.5f);
            animator.SetLookAtPosition(lookAtObj.position);
        }
        else {
            animator.SetIKPositionWeight(AvatarIKGoal.RightFoot, 0.0f);
            animator.SetIKRotationWeight(AvatarIKGoal.RightFoot, 0.0f);
            animator.SetLookAtWeight(0.0f); //body=0.0f, head=1.0f, eyes=0.0f
        }
    }
}
```

lookAtObj



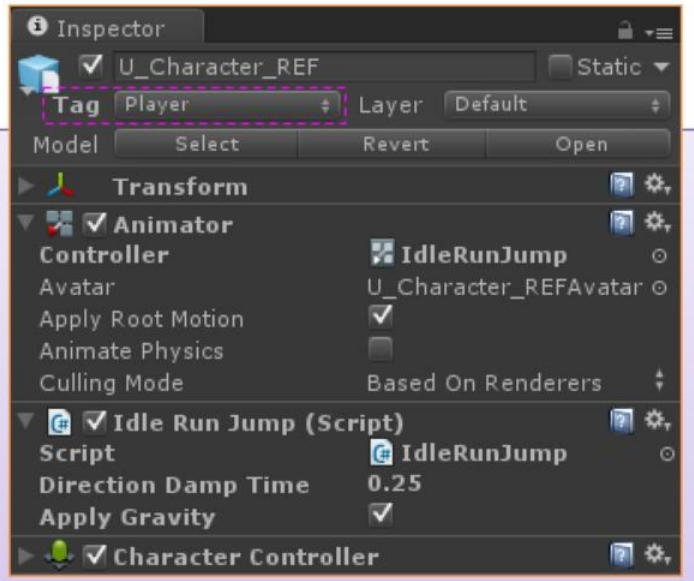
```
void SetLookAtPosition(Vector3 lookAtPosition);
void SetLookAtWeight(float weight, float body, float head, float eyes, float clamp=0.5f);
```

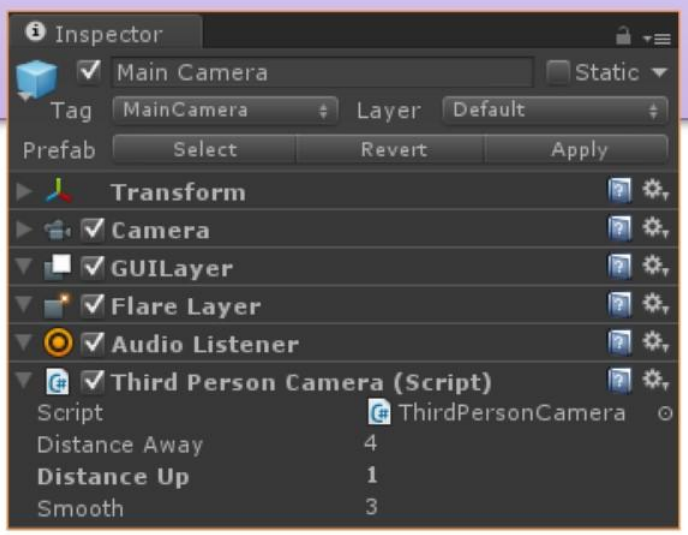
weight: 다른 가중치에 곱해짐, clamp: 1.0(LookAt 적용되지 않음)

애니메이션 시스템

- 애니메이터(Animator) 클래스
  - 군중 시뮬레이션(Crowd Simulation)

```
public class ThirdPersonCamera : MonoBehaviour {
    public float distanceAway;
    public float distanceUp;
    public float smooth;
    private GameObject hovercraft;
    private Transform follow;
    void Start() {
        follow = GameObject.FindWithTag("Player").transform;
    }
    void LateUpdate() {
        Vector3 targetPosition = follow.position + Vector3.up * distanceUp - follow.forward * distanceAway;
        transform.position = Vector3.Lerp(transform.position, targetPosition, Time.deltaTime * smooth);
        transform.LookAt(follow);
    }
}
```







애니메이션 시스템

- 애니메이터(Animator) 클래스
  - 군중 시뮬레이션(Crowd Simulation)



